THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. R.MOHANAPRIYA. Reg. No:20524UCSC057 a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Incharge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		

14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01	CREATION OF DDL OPERATIONS USING VEHICLES TABLE
DATE:28/07/2026	

Aim:

To create a *Vehicles* table, insert vehicle records, display all records, and retrieve vehicles based on the required year.

Software used:

Oracle / MySQL / sqlite3 **Description:**

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL commands:

- Create
- Alter
- Drop
- Truncate

Create: Create command is used to create a new table and also use constraints.

Syntax:

CREATE TABLE vehicle (vehicle_id

INTEGER PRIMARY KEY,

vehicle_name TEXT, model_year

INTEGER

);

Table Created

Alter: Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP

Add: Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

ALTER TABLE vehicle ADD brand VARCHAR(100);

Table Altered

MODIFY: Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype; **Query:**

ALTER TABLE vehicle MODIFY vehicle_name VARCHAR(150); DROP:Drop

command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name; **Query:**

ALTER TABLE vehicle DROP COLUMN brand;

RENAME:Rename command is used to re-change the table name.

ALTER TABLE vehicle RENAME TO vehicles;

Table renamed.

Truncate: Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE vehicles;

DROP: Deletes the table and all its data permanently.

Syntax:DROP TABLE tablename;

Query:

DROP TABLE vehicles;

Explanation

- Create a **Vehicle** table to store vehicle details.

- Insert vehicle records into the table.
- Use **ALTER** to add, modify, or remove columns.
- Use **TRUNCATE** to delete all records and **DROP** to delete the table.
- Display vehicle details and retrieve vehicles based on the given condition.

Create Table

```
CREATE TABLE vehicle (     vehicle_id
INTEGER PRIMARY KEY,
vehicle_name TEXT,     model_year
INTEGER
);
```

Sample Data

```
INSERT INTO vehicle VALUES (1, 'Toyota', 2020);

INSERT INTO vehicle VALUES (2, 'Honda', 2018);

INSERT INTO vehicle VALUES (3, 'BMW', 2022);

INSERT INTO vehicle VALUES (4, 'Hyundai', 2021);

INSERT INTO vehicle VALUES (5, 'Ford', 2019);

INSERT INTO vehicle VALUES (6, 'Tesla', 2023);
```

SQL Query

Query 1: Display all vehicle records

```
SQL> SELECT * FROM vehicle;
```

Resultant Query

vehicle_id	vehicle_name	model_year
1	Toyota	2020
2	Honda	2018
3	BMW	2022
4	Hyundai	2021
5	Ford	2019
6	Tesla	2023

Query 2: Vehicles from 2020 onwards

```
SELECT *

FROM vehicle

WHERE model_year >= 2020

ORDER BY model_year;
```

Resultant Query

vehicle_id	vehicle_name	model_year
1	**Toyota**	**2020**
4	**Hyundai**	**2021**
3	**BMW**	**2022**
6	**Tesla**	**2023**

Result

Thus, the Vehicle table was created successfully, records were inserted, all vehicle details were displayed, and vehicles from the year 2020 onwards were retrieved successfully.

EX. NO:02	**CREATION OF DML OPERATIONS USING FLOWERS TABLE**
DATE:29/07/2026	

Aim:

To insert flower records, update the flower details, delete flower records, and display the records from the *Flowers* table.

Software Used: Oracle

/ MySQL / sqlite3

Description:

DML (Data Manipulation Language), it is used to insert, modify, delete, and retrieve data from a database table.

DML commands:

- Insert
- Update
- Delete
- Select

Insert:

Insert command is used to add new records into the table.

Syntax:

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query:

INSERT INTO flowers VALUES (1, 'Rose', 'Red');

INSERT INTO flowers VALUES (2, 'Lily', 'White');

INSERT INTO flowers VALUES (3, 'Sunflower', 'Yellow');

INSERT INTO flowers VALUES (4, 'Jasmine', 'White');

INSERT INTO flowers VALUES (5, 'Lotus', 'Pink');

Update:

Update command is used to modify existing records in the table.

Syntax:

UPDATE table_name

SET column_name = value

WHERE condition;

Query

UPDATE flowers

SET color = 'Pink' WHERE

flower_id = 1;

Delete:

Delete command is used to remove existing records from the table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

DELETE FROM flowers

WHERE flower_id = 5;

Explanation

- DML is used to manipulate the data in a database table.
- INSERT is used to add new records to the table.
- UPDATE is used to modify existing records.
- DELETE is used to remove records from the table.
- SELECT is used to display or retrieve records from the table.

SQL Query

Query 1: Display all flowers records

SQL> SELECT * FROM flowers;

flower_id	flower_name	color
1	Rose	Red
2	Lily	White
3	Sunflower	Yellow
4	Jasmine	White
5	Lotus	Pink

UPDATE Command

```
UPDATE flowers
SET color = 'Pink'
WHERE flower_id = 1;
```

flower_id	flower_name	color
1	Rose	Pink

DELETE

```
DELETE FROM flowers
WHERE flower_id = 5;
```

Result:

Thus, the **DML operations — INSERT, UPDATE, DELETE, and SELECT — were successfully executed on the flowers table.**

EX.NO:03	CREATION OF DQL OPERATIONS USING PRODUCTS TABLE
DATE:01/08/2026	

Aim:

To create a **Products** table, insert product records, and perform DQL operations such as **SELECT, WHERE, ORDER BY, GROUP BY, and HAVING** to retrieve and display data.

Software Used:

Oracle / MySQL / SQLite3 **Description:**

DQL (Data Query Language) is used to retrieve and display data from database tables.

DQL COMMANDS:

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING **SELECT:**

SELECT command is used to retrieve and display records from a table.

Syntax:

SELECT column_name

FROM table_name;

Query

SELECT * FROM products;

WHERE

WHERE clause is used to retrieve records based on a specified condition.

Syntax:

SELECT column_name

FROM table_name

WHERE condition;

Query:

SELECT *

FROM products

WHERE price > 2000;

Result

product_id	product_name	category	price
101	Laptop	Electronics	55000
103	Chair	Furniture	4500
105	Table	Furniture	7000
106	Headphone	Electronics	2500

ORDER BY

ORDER BY is used to sort records in ascending or descending order.

Syntax:

SELECT column_name

FROM table_name

ORDER BY column_name ASC/DESC; **Query:**

SQL>SELECT *

FROM products

ORDER BY price DESC;

Result

product_id	product_name	category	price
101	Laptop	Electronics	55000
102	Mouse	Electronics	800
103	Chair	Furniture	4500

104	Keyboard	Electronics	1500
105	Table	Furniture	7000
106	Headphone	Electronics	2500

GROUP BY

GROUP BY is used to group records having the same value in a column.

Syntax:

SELECT column_name, COUNT(*)

FROM table_name GROUP

BY column_name;

Query:

SQL>SELECT category, COUNT(*) AS total_products

FROM products

GROUP BY category;

Result

category	total_products
Electronics	4
Furniture	2

HAVING

HAVING is used to filter groups created using the GROUP BY clause.

Syntax:

SELECT column_name, COUNT(*)

FROM table_name

GROUP BY column_name

HAVING condition; **Query:**

SQL>SELECT category, COUNT(*) AS total_products

FROM products

GROUP BY category

HAVING COUNT(*) > 2;

Result

category	total_products
Electronics	4

CREATE TABLE Syntax:

CREATE TABLE products(product_id

INTEGER PRIMARY KEY,

product_name TEXT, category TEXT,

price INTEGER

);

Table Created

SAMPLE DATA

INSERT INTO products VALUES (101, 'Laptop', 'Electronics', 55000);

INSERT INTO products VALUES (102, 'Mouse', 'Electronics', 800);

INSERT INTO products VALUES (103, 'Chair', 'Furniture', 4500);

INSERT INTO products VALUES (104, 'Keyboard', 'Electronics', 1500);

INSERT INTO products VALUES (105, 'Table', 'Furniture', 7000);

INSERT INTO products VALUES (106, 'Headphone', 'Electronics', 2500);

Records Inserted

product_id	product_name	category	price
101	Laptop	Electronics	55000
102	Mouse	Electronics	800
103	Chair	Furniture	4500
104	Keyboard	Electronics	1500
105	Table	Furniture	7000
106	Headphone	Electronics	2500

Explanation – Products Table

1. The **Products** table stores information about different products.
2. product_id uniquely identifies each product.
3. product_name stores the name of the product.
4. category stores the product category.
5. price stores the price of each product.

Result

Thus, the **Products table** was created successfully and the DQL commands **SELECT, WHERE, ORDER BY, GROUP BY, and HAVING** were executed successfully to retrieve, filter, sort, group, and display the product records.

EX.NO:04	**CREATION OF TRANSACTION OPERATIONS USING FOOD TABLE**
DATE:02/08/2026	

Aim:

To create a Food table, insert food records, and perform transaction operations using **COMMIT, ROLLBACK, and SAVEPOINT**, and verify the changes made to the database.

Software Used:

Oracle / MySQL / SQLite3 **Description:**

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS

- COMMIT
- ROLLBACK
- SAVEPOINT **COMMIT:**

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax:

COMMIT; **Query:**

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax:

ROLLBACK; **Query:**

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax:

SAVEPOINT savepoint_name; **Query:**

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without ending the entire transaction.

Syntax:

ROLLBACK TO SAVEPOINT savepoint_name; **Query:**

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

CREATE TABLE

SQL>CREATE TABLE food(

food_id INTEGER PRIMARY KEY,

food_name TEXT, price INTEGER

);

Table Created

SAMPLE DATA

INSERT INTO food VALUES (101, 'Pizza', 250);

INSERT INTO food VALUES (102, 'Burger', 180);

INSERT INTO food VALUES (103, 'Biryani', 300);

INSERT INTO food VALUES (104, 'Pasta', 220);

INSERT INTO food VALUES (105, 'Sandwich', 150);

Records Inserted

food_id	food_name	price
101	Pizza	250
102	Burger	180
103	Biryani	300
104	Pasta	220
105	Sandwich	150

SQL QUERY

Query 1: Display all food records

SQL> SELECT * FROM food;

Resultant Query

food_id	food_name	price
101	Pizza	250
102	Burger	180
103	Biryani	300
104	Pasta	220
105	Sandwich	150

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed.

The inserted food records are now permanently saved.

Query 3: Update Pizza's price

SQL> UPDATE food

SET price = 300

WHERE food_id = 101; **1**

row updated.

Updated Record

food_id	food_name	price
101	Pizza	300

T

Query 4: Create a SAVEPOI

SQL> SAVEPOINT sp1;

Savepoint created. **e**

Query 5: Update Burger's pri

SQL> UPDATE food

SET price = 220

WHERE food_id = 102;

1 row updated.

Updated Record

food_id	food_name	price
102	Burger	220

Query 6: Rollback to SAVEPOINT SQL>

ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The change made to **Burger after the savepoint is undone**.

Resultant Query

food_id	food_name	price
101	Pizza	300
102	Burger	180

Query 7: Rollback an uncommitted change

SQL> UPDATE food

SET price = 350

WHERE food_id = 103;

1 row updated.

SQL> ROLLBACK;

Transaction rolled back.

The price of **Biryani** is restored to its previous value.

Resultant Query

food_id	food_name	price
103	Biryani	300

Explanation

- Create a table named **food**.
- Insert food details into the table.
- Use **COMMIT** to permanently save inserted records.
- Update the price of Pizza and create a **SAVEPOINT**.
- Make another change after the savepoint.
- Use **ROLLBACK TO SAVEPOINT** to undo changes made after the savepoint.
- Use **ROLLBACK** to undo uncommitted changes.

Result

Thus, the **Food table** was created successfully, food records were inserted, and the transaction operations **COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT** were performed successfully.

EX.NO:05	**CREATION OF JOIN OPERATIONS USING BIRD AND BIRD_ORDERS TABLE**
DATE:04/08/2026	

Aim:

To create Bird and Bird_Orders tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used:

Oracle / MySQL / SQLite **Description:**

Joins are used to combine records from two or more tables based on a related column between them.

Join Commands:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

RIGHT JOIN
Right Join returns all records from the right table and the matching records from the left table.

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax

```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

SELF JOIN

Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax

```
SELECT A.column, B.column

FROM table_name A

JOIN table_name B

ON A.column = B.column;
```

Explanation

- Create Bird and Bird_Orders tables.
- Insert bird and order details into the tables.
- Display matching bird and order records using Inner Join.
- Display all birds and their matching orders using Left Join.
- Display all orders and their matching birds using Right Join.
- Display all matching and non-matching records using Full Join.
- Use Self Join to display parent and child birds.

Create Tables

```
CREATE TABLE bird (     bird_id
INTEGER PRIMARY KEY,
   bird_name TEXT,
   species TEXT
);
```

Table Created

Sample Data

```
INSERT INTO bird VALUES (1, 'Parrot', 'Macaw');
INSERT INTO bird VALUES (2, 'Peacock', 'Indian Peafowl');
INSERT INTO bird VALUES (3, 'Sparrow', 'House Sparrow');
INSERT INTO bird VALUES (4, 'Eagle', 'Golden Eagle');
INSERT INTO bird VALUES (5, 'Owl', 'Barn Owl');
CREATE TABLE bird_orders (     order_id
INTEGER PRIMARY KEY,     bird_id INTEGER,
food TEXT,     amount INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO bird_orders VALUES (101, 1, 'Seeds', 550);
INSERT INTO bird_orders VALUES (102, 2, 'Grains', 250);
INSERT INTO bird_orders VALUES (103, 1, 'Fruits', 300);
INSERT INTO bird_orders VALUES (104, 3, 'Millet', 200);
INSERT INTO bird_orders VALUES (105, 6, 'Worms', 100);
```

Records Inserted

SQL Query

Query 1: Inner Join

```
SELECT bird.bird_name, bird_orders.food, bird_orders.amount
FROM bird
INNER JOIN bird_orders
ON bird.bird_id = bird_orders.bird_id;
```

Resultant Query

bird_name	food	amount
Parrot	Seeds	550
Peacock	Grains	250
Parrot	Fruits	300
Sparrow	Millet	200

Query 2: Left Join

```
SELECT bird.bird_name, bird_orders.food, bird_orders.amount
FROM bird
LEFT JOIN bird_orders
ON bird.bird_id = bird_orders.bird_id;
```

Resultant Query

bird_name	food	amount
Parrot	Seeds	550
Peacock	Grains	250
Parrot	Fruits	300
Sparrow	Millet	200
Eagle	NULL	NULL
Owl	NULL	NULL

Query 3: Right Join

```
SELECT bird.bird_name, bird_orders.food, bird_orders.amount
FROM bird
RIGHT JOIN bird_orders
ON bird.bird_id = bird_orders.bird_id;
```

Resultant Query

bird_name	food	amount
Parrot	Seeds	550
Peacock	Grains	250
Parrot	Fruits	300
Sparrow	Millet	200
NULL	Worms	100

Query 4: Full Join

```
SELECT bird.bird_name, bird_orders.food, bird_orders.amount
FROM bird
FULL JOIN bird_orders
ON bird.bird_id = bird_orders.bird_id;
```

Resultant Query

bird_name	food	amount
Parrot	Seeds	550
Peacock	Grains	250
Parrot	Fruits	300
Sparrow	Millet	200
Eagle	NULL	NULL
Owl	NULL	NULL
NULL	Worms	100

Query 5: Self Join

Create Bird Table with Parent ID

```
CREATE TABLE birds (     bird_id
INTEGER PRIMARY KEY,
   bird_name TEXT,
parent_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO birds VALUES (1, 'Parrot', NULL);
INSERT INTO birds VALUES (2, 'Peacock', 1);
```

```
INSERT INTO birds VALUES (3, 'Sparrow', 1);
INSERT INTO birds VALUES (4, 'Owl', 2);
INSERT INTO birds VALUES (5, 'Eagle', 2);
```

Query

```
SELECT A.bird_name AS Bird,
    B.bird_name AS Parent
FROM birds A
LEFT JOIN birds B
ON A.parent_id = B.bird_id;
```

Resultant Query

Bird	Parent
Parrot	NULL
Peacock	Parrot
Sparrow	Parrot
Owl	Peacock
Eagle	Peacock

Result:

Thus, the Bird and Bird_Orders tables were created successfully, and the required records were retrieved successfully using **Inner Join, Left Join, Right Join, Full Join, and Self Join operations.**

EX. NO: 06	CREATION OF SQL FUNCTIONS USING COLORS TABLE
DATE:05/08/2026	

Aim

To create a Colors table, insert color records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL Function Types:

- Aggregate Functions
- String Functions
- Date Functions

Aggregate Functions

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

```
SELECT aggregate_function(column_name)
FROM table_name;
```

String Functions

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

Syntax

```
SELECT string_function(column_name)
FROM table_name;
```

Date Functions

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named Colors.
- Insert color details into the table.
- Display all color records.
- Calculate the total number of colors using COUNT().
- Calculate the total and average price using SUM() and AVG().
- Find the highest and lowest price using MAX() and MIN().
- Display color names in uppercase and lowercase using UPPER() and LOWER().
- Find the length of color names using LENGTH().
- Retrieve color details based on the date.

Create Table

```
CREATE TABLE colors (     color_id
INTEGER PRIMARY KEY,
   color_name TEXT,
color_type TEXT,     price
INTEGER,
   purchase_date DATE
);
```

Table Created

Sample Data

INSERT INTO colors VALUES (1, 'Red', 'Primary', 500, '2026-01-15');

INSERT INTO colors VALUES (2, 'Blue', 'Primary', 800, '2026-02-10');

INSERT INTO colors VALUES (3, 'Green', 'Secondary', 1200, '2026-03-05');

INSERT INTO colors VALUES (4, 'Yellow', 'Primary', 600, '2026-04-12');

INSERT INTO colors VALUES (5, 'Purple', 'Secondary', 1500, '2026-05-20');

INSERT INTO colors VALUES (6, 'Orange', 'Secondary', 1000, '2026-06-18');

Records Inserted

SQL Query

Query 1: Display all color records

SELECT * FROM colors;

Resultant Query

color_id	color_name	color_type	price	purchase_date
1	Red	Primary	500	2026-01-15

2	Blue	Primary	800	2026-02-10
3	Green	Secondary	1200	2026-03-05
4	Yellow	Primary	600	2026-04-12
5	Purple	Secondary	1500	2026-05-20
6	Orange	Secondary	1000	2026-06-18

AGGREGATE FUNCTIONS

Query 2: Count the total number of colors

SELECT COUNT(*) AS Total_Colors

FROM colors;

Resultant Query

Total_Colors
6

Query 3: Calculate the total price of all colors

SELECT SUM(price) AS Total_Price

FROM colors;

Resultant Query

Total_Price
5600

Query 4: Calculate the average price of colors

SELECT AVG(price) AS Average_Price

FROM colors;

Resultant Query

Average_Price
933.33

Query 5: Find the highest and lowest color price

SELECT MAX(price) AS Highest_Price,

MIN(price) AS Lowest_Price

Highest_Price	Lowest_Price
1500	500

FROM colors;

Resultant Query

STRING FUNCTIONS

Query 6: Display color names in uppercase

SELECT color_name, UPPER(color_name) AS Upper_Name

FROM colors;

Resultant Query

color_name	Upper_Name
Red	RED
Blue	BLUE
Green	GREEN
Yellow	YELLOW
Purple	PURPLE
Orange	ORANGE

Query 7: Display color names in lowercase

SELECT color_name, LOWER(color_name) AS Lower_Name

FROM colors;

Resultant Query

color_name	Lower_Name
Red	red
Blue	blue
Green	green
Yellow	yellow
Purple	purple
Orange	orange

Query 8: Display the length of each color name

```
SELECT color_name, LENGTH(color_name) AS Name_Length
FROM colors;
```

Resultant Query

color_name	Name_Length
Red	3
Blue	4
Green	5
Yellow	6
Purple	6
Orange	6

DATE FUNCTIONS

Query 9: Display the current date

```
SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date
Current system date

Query 10: Display colors purchased after March 2026

```
SELECT color_name, purchase_date
FROM colors
WHERE purchase_date > '2026-03-31';
```

Resultant Query

color_name	purchase_date
Yellow	2026-04-12
Purple	2026-05-20
Orange	2026-06-18

RESULT:

Thus, the **Colors table** was created successfully, color records were inserted, and the required data was retrieved successfully using **Aggregate Functions, String Functions, and Date Functions**.

EX.NO:07	Creation of Subqueries and Nested Queries using Bridal Makeup
DATE:06/08/2026	

Aim

To create a Bridal Makeup table, insert makeup records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / SQLite3

Description

A **Subquery** is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A **Nested Query** is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as >, <, >=, <=, =.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

Multiple Row Subquery

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name
                      FROM table_name
                      WHERE condition);
```

Nested Subquery

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

Explanation

- Create a table named bridal_makeup.
- Insert bridal makeup details into the table.
- Display all bridal makeup records.
- Find makeup whose price is greater than the average price.
- Find makeup belonging to selected categories.
- Find the makeup with the highest price.
- Retrieve makeup based on the result of another query.

Create Table

```
CREATE TABLE bridal_makeup (
makeup_id INTEGER PRIMARY KEY,
   makeup_name TEXT,
category TEXT,     price
INTEGER
);
```

Table Created

Sample Data

INSERT INTO bridal_makeup VALUES (1, 'Traditional Bridal', 'South Indian', 4500);

INSERT INTO bridal_makeup VALUES (2, 'HD Bridal', 'North Indian', 3200);

INSERT INTO bridal_makeup VALUES (3, 'Engagement Makeup', 'Engagement', 3800);

INSERT INTO bridal_makeup VALUES (4, 'Airbrush Bridal', 'Airbrush', 5000);

INSERT INTO bridal_makeup VALUES (5, 'Reception Makeup', 'Reception', 2800);

INSERT INTO bridal_makeup VALUES (6, 'Premium Bridal', 'Luxury', 4200);

Records Inserted

SQL Query

Query 1: Display all bridal makeup records

SELECT * FROM bridal_makeup;

Resultant Query

makeup_id	makeup_name	category	price
1	Traditional Bridal	South Indian	4500
2	HD Bridal	North Indian	3200
3	Engagement Makeup	Engagement	3800
4	Airbrush Bridal	Airbrush	5000
5	Reception Makeup	Reception	2800

6	Premium Bridal	Luxury	4200

Query 2: Display makeup whose price is greater than the average price

```
SELECT makeup_name, price
FROM bridal_makeup
WHERE price > (SELECT AVG(price)
        FROM bridal_makeup);
```

Resultant Query

makeup_name	**price**
Traditional Bridal	4500
Airbrush Bridal	5000
Premium Bridal	4200

Query 3: Display makeup belonging to Airbrush or North Indian category

```
SELECT makeup_name, category
FROM bridal_makeup
WHERE category IN (SELECT category
        FROM bridal_makeup
        WHERE category IN ('Airbrush', 'North Indian'));
```

Resultant Query

makeup_name	**category**
HD Bridal	North Indian
Airbrush Bridal	Airbrush

Query 4: Display the bridal makeup with the highest price

```
SELECT makeup_name, price
FROM bridal_makeup
WHERE price = (SELECT MAX(price)
        FROM bridal_makeup);
```

Resultant Query

makeup_name	price
Airbrush Bridal	5000

Query 5: Display makeup whose price is greater than HD Bridal's price

```
SELECT makeup_name, price
FROM bridal_makeup
WHERE price > (SELECT price
        FROM bridal_makeup
        WHERE makeup_name = 'HD Bridal');
```

Resultant Query

makeup_name	price
Traditional Bridal	4500
Engagement Makeup	3800
Airbrush Bridal	5000
Premium Bridal	4200

Result:

Thus, the **Bridal Makeup table** was created successfully, makeup records were inserted, and the required details were retrieved successfully using **Subqueries and Nested Queries**.

EX.NO:08	CREATION OF VIEW,INDEX AND SEQUENCE OPERATIONS USING CLOTHES
DATE:06/08/2026	

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:

A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:

An Index is used to improve the speed of data retrieval from a table.

Sequence:

A Sequence is a database object used to generate unique numeric values automatically.

1.VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;

Create Table

```
CREATE TABLE clothes (     clothes_id
NUMBER PRIMARY KEY,
clothes_name VARCHAR2(50),
category VARCHAR2(30),
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO clothes VALUES (101, 'Saree', 'Traditional', 4500);

INSERT INTO clothes VALUES (102, 'Kurti', 'Casual', 3200);

INSERT INTO clothes VALUES (103, 'Lehenga', 'Bridal', 3800);

INSERT INTO clothes VALUES (104, 'Gown', 'Party', 5000);

INSERT INTO clothes VALUES (105, 'Jeans', 'Casual', 4200);
COMMIT;
```

Records Inserted

Create View

```
CREATE VIEW it_clothes AS
SELECT clothes_id, clothes_name, price
FROM clothes
WHERE category = 'Casual';
```

View Created

Display View

```
SELECT * FROM it_clothes;
```

Resultant Query

clothes_id	clothes_name	price
102	Kurti	3200
105	Jeans	4200

Explanation

The it_clothes view displays only the clothes belonging to the **Casual** category.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_clothes_name

ON clothes(clothes_name);

Index Created

Use Index in Search

SELECT *

FROM clothes

WHERE clothes_name = 'Saree';

Resultant Query

clothes_id	clothes_name	category	price
101	Saree	Traditional	4500

Explanation

The index idx_clothes_name is created on the clothes_name column. It helps improve the performance of searches based on clothes names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE clothes_seq

START WITH 106

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

INSERT INTO clothes

VALUES (clothes_seq.NEXTVAL, 'Jacket', 'Winter', 2800);

COMMIT;

Record Inserted

Display Sequence Value

SELECT clothes_seq.CURRVAL AS current_sequence

FROM dual;

Insert Record Using Sequence

INSERT INTO clothes

VALUES (clothes_seq.NEXTVAL, 'Jacket', 'Winter', 2800);

COMMIT;

Record Inserted

Display Sequence Value

SELECT clothes_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence
106

Insert Another Record Using Sequence

INSERT INTO clothes

VALUES (clothes_seq.NEXTVAL, 'Frock', 'Kids', 2500);

COMMIT;

Record Inserted

Display All Clothes Records SELECT

* FROM clothes

ORDER BY clothes_id;

Resultant Query

clothes_id	clothes_name	category	price
101	Saree	Traditional	4500
102	Kurti	Casual	3200
103	Lehenga	Bridal	3800
104	Gown	Party	5000
105	Jeans	Casual	4200
106	Jacket	Winter	2800
107	Frock	Kids	2500

Important Commands

Drop View

DROP VIEW it_clothes;

View Dropped

Drop Index

DROP INDEX idx_clothes_name;

Index Dropped

Drop Sequence

DROP SEQUENCE clothes_seq;

Sequence Dropped

Result

Thus, the **View, Index, and Sequence** were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO: 09	Creation of Stored Procedures and Functions
DATE:07/08/2026	

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:

A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:

A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name

(parameter_name IN datatype)

IS

BEGIN

   statements;

END;

/
```

Create Table

```
CREATE TABLE cars (
   car_id NUMBER PRIMARY KEY,
car_name VARCHAR2(50),
car_type VARCHAR2(30),     price
NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO cars VALUES (101, 'Swift', 'Hatchback', 750000);

INSERT INTO cars VALUES (102, 'Creta', 'SUV', 1200000);

INSERT INTO cars VALUES (103, 'City', 'Sedan', 1100000);

INSERT INTO cars VALUES (104, 'Nexon', 'SUV', 950000);

INSERT INTO cars VALUES (105, 'Verna', 'Sedan', 1300000);
COMMIT;
```

Records Inserted

Display Car Records

```
SELECT * FROM cars;
```

Resultant Query

car_id	car_name	car_type	price
101	Swift	Hatchback	750000
102	Creta	SUV	1200000
103	City	Sedan	1100000
104	Nexon	SUV	950000
105	Verna	Sedan	1300000

Create Stored Procedure

The following procedure displays the details of a car based on the Car ID.

```
CREATE OR REPLACE PROCEDURE display_car (
p_car_id IN NUMBER
) IS     v_name
cars.car_name%TYPE;     v_price
cars.price%TYPE;
BEGIN
   SELECT car_name, price
   INTO v_name, v_price
   FROM cars
   WHERE car_id = p_car_id;

   DBMS_OUTPUT.PUT_LINE('Car Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Price: ' || v_price);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;
EXEC display_car(103);
```

Resultant Query Car

Name: City

Price: 1100000

Output

Car Name: City

Price: 1100000

Explanation

The procedure display_car accepts the Car ID as an input parameter and retrieves the corresponding car name and price from the cars table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
statements;
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the car category based on the car price.

```
CREATE
OR REPLACE FUNCTION get_category (
   p_price IN NUMBER
)
RETURN VARCHAR2
IS    v_category
VARCHAR2(10);
BEGIN
   IF p_price >= 1200000 THEN
v_category := 'Luxury';
```

```
    ELSIF p_price >= 1000000 THEN
v_category := 'Premium';

    ELSIF p_price >= 800000 THEN
        v_category := 'Mid';

    ELSE        v_category :=
'Budget';
    END IF;

    RETURN v_category;
END;
/
```

Function Created

Execute Function

```
SELECT car_id, car_name, price,
get_category(price) AS category
FROM cars;
```

Resultant Query

car_id	car_name	price	category
101	Swift	750000	Budget
102	Creta	1200000	Luxury
103	City	1100000	Premium
104	Nexon	950000	Mid
105	Verna	1300000	Luxury

Explanation

The function get_category accepts the car price as an input and returns the corresponding category. The function is called inside the SELECT statement for each car.

Important Commands

Display Stored Procedure

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_CAR';

Resultant Query

object_name	object_type
DISPLAY_CAR	PROCEDURE

Display Stored Function

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_CATEGORY';

Resultant Query

object_name	object_type
GET_CATEGORY	FUNCTION

Drop Procedure

DROP PROCEDURE display_car;

Procedure Dropped

Drop Function

DROP FUNCTION get_category;

Function Dropped

Result:

Thus, the **Stored Procedure and Function** were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve car details, and the Function was used to determine the car category based on its price.

EX. NO: 10	CREATION OF TRIGGERS METHOD USING APP TABLE
DATE:08/08/2026	

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as inserting, updating, or maintaining records when a particular event takes place.

1. CREATE TABLE CREATE

TABLE app(app_id NUMBER

PRIMARY KEY, app_name

VARCHAR2(50), stock NUMBER,

price NUMBER

);

Table Created

Insert Records

INSERT INTO app VALUES (201, 'Laptop', 10, 55000);

INSERT INTO app VALUES (202, 'Mobile', 20, 25000);

INSERT INTO app VALUES (203, 'Keyboard', 15, 2000);

INSERT INTO app VALUES (204, 'Headphone', 12, 3000);

INSERT INTO app VALUES (205, 'Mouse', 25, 1000);

COMMIT;

Records Inserted

Display App Records

SELECT * FROM app;

Resultant Query

app_id	app_name	stock	price
201	Laptop	10	55000
202	Mobile	20	25000
203	Keyboard	15	2000
204	Headphone	12	3000
205	Mouse	25	1000

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW BEGIN
   statements;
END;
/
```

Create Trigger

The following trigger checks whether the stock is less than 0 before inserting a record.

```
CREATE OR REPLACE TRIGGER check_app_stock
BEFORE INSERT ON app
FOR EACH ROW
BEGIN
```

```
    IF :NEW.stock < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Stock cannot be negative');
    END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO app VALUES (206, 'Printer', -5, 12000);
```

Output

ORA-20001: Stock cannot be negative

The trigger prevents the insertion of an invalid stock value.

Correct Insert

```
INSERT INTO app VALUES (206, 'Printer', 8, 12000);
COMMIT;
```

Record Inserted

3. AFTER UPDATE TRIGGER

The following trigger updates the app name to **'Out of Stock'** when the stock becomes zero.

```
CREATE OR REPLACE TRIGGER update_stock
AFTER UPDATE ON app
FOR EACH ROW
BEGIN
    IF :NEW.stock = 0 THEN
        UPDATE app
        SET app_name = 'Out of Stock'
        WHERE app_id = :NEW.app_id;
    END IF;
END;
/
```

Trigger Created

Execute Trigger

UPDATE app SET stock = 0 WHERE app_id = 202;

COMMIT;

Display Updated Record

SELECT * FROM app WHERE app_id = 202;

Resultant Query

app_id	app_name	stock	price
202	Out of Stock	0	25000

4. AFTER DELETE TRIGGER

The following trigger maintains an audit table when a record is deleted.

Create Audit Table CREATE TABLE

app_audit(audit_id NUMBER

PRIMARY KEY, app_id NUMBER,

app_name VARCHAR2(50), action

VARCHAR2(20), deleted_date

DATE

);

Table Created

Create Trigger

CREATE OR REPLACE TRIGGER delete_audit

AFTER DELETE ON app

FOR EACH ROW

BEGIN

INSERT INTO app_audit

VALUES (:OLD.app_id, :OLD.app_id, :OLD.app_name, 'Deleted', SYSDATE);

END;

/

Trigger Created

Execute Trigger

DELETE FROM app WHERE app_id = 205;

COMMIT;

Record Deleted

Display Audit Records

SELECT * FROM app_audit;

Resultant Query

audit_id	app_id	app_name	action	deleted_date
1	205	Mouse	Deleted	2026-10-02

5. BEFORE UPDATE TRIGGER

The following trigger validates the price before updating a record.

```
CREATE OR REPLACE TRIGGER check_price
BEFORE UPDATE OF price ON app
FOR EACH ROW
BEGIN
   IF :NEW.price < 1000 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Price cannot be less than 1000');
   END IF;
END;
/
```

Trigger Created

Execute Trigger — Invalid Update

UPDATE app SET price = 500 WHERE app_id = 201;

Resultant Query

ORA-20002: Price cannot be less than 1000

Correct Update

UPDATE app SET price = 1200 WHERE app_id = 201;

COMMIT;

Record Updated

Display Updated Record

SELECT * FROM app WHERE app_id = 201;

Resultant Query

app_id	app_name	stock	price
201	Laptop	10	1200

6. DROP TRIGGER

DROP TRIGGER check_app_stock;

Trigger Dropped

DROP TRIGGER update_stock;

Trigger Dropped

DROP TRIGGER delete_audit;

Trigger Dropped

DROP TRIGGER check_price;

Trigger Dropped

Important Commands

Display Triggers

SELECT trigger_name, triggering_event, status

FROM user_triggers;

Resultant Query

trigger_name	triggering_event	status
UPDATE_STOCK	UPDATE	ENABLED
DELETE_AUDIT	DELETE	ENABLED
CHECK_PRICE	UPDATE	ENABLED

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated app stock and price before modification and recorded deleted app details in the audit table.

EX.NO:11	CREATION OF CURSOR IMPLEMENTATION USING PLATE SHOP TABLE
DATE:09/08/2026	

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an **Explicit Cursor** is implemented.

1. CREATE TABLE

```
CREATE TABLE plate_shop (
plate_id NUMBER PRIMARY KEY,
plate_name VARCHAR2(50),
material VARCHAR2(50),     price
NUMBER
);
```

Table Created

Insert Records

INSERT INTO plate_shop VALUES (301, 'Dinner Plate', 'Ceramic', 550);

INSERT INTO plate_shop VALUES (302, 'Serving Plate', 'Steel', 650);

INSERT INTO plate_shop VALUES (303, 'Dessert Plate', 'Glass', 450);

INSERT INTO plate_shop VALUES (304, 'Decorative Plate', 'Porcelain', 700);

INSERT INTO plate_shop VALUES (305, 'Melamine Plate', 'Melamine', 600);

COMMIT;

Records Inserted

Display Plate Records

SELECT * FROM plate_shop;

Resultant Query

plate_id	plate_name	material	price
301	Dinner Plate	Ceramic	550
302	Serving Plate	Steel	650
303	Dessert Plate	Glass	450
304	Decorative Plate	Porcelain	700
305	Melamine Plate	Melamine	600

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;
```

```
   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;

   CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each plate name and price from the plate_shop table.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR plate_cursor IS
      SELECT plate_name, price
      FROM plate_shop;

   v_plate_name plate_shop.plate_name%TYPE;
v_price plate_shop.price%TYPE;

BEGIN
   OPEN plate_cursor;

   LOOP
      FETCH plate_cursor INTO v_plate_name, v_price;
```

```
        EXIT WHEN plate_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Plate: ' || v_plate_name || ' | Price: ₹' || v_price
        );
    END LOOP;

    CLOSE plate_cursor;
END;
/
```

Output

Plate: Dinner Plate | Price: ₹550

Plate: Serving Plate | Price: ₹650

Plate: Dessert Plate | Price: ₹450

Plate: Decorative Plate | Price: ₹700 Plate:

Melamine Plate | Price: ₹600

3. CURSOR WITH CONDITION

The following cursor displays only the plates whose price is greater than ₹500.

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR plate_cursor IS
        SELECT plate_id, plate_name, price
        FROM plate_shop
        WHERE price > 500;
```

```
    v_plate_id plate_shop.plate_id%TYPE;
v_plate_name plate_shop.plate_name%TYPE;
v_price plate_shop.price%TYPE;

BEGIN
    OPEN plate_cursor;

    LOOP
        FETCH plate_cursor INTO v_plate_id, v_plate_name, v_price;

        EXIT WHEN plate_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Plate ID: ' || v_plate_id ||
            ' | Plate: ' || v_plate_name ||
            ' | Price: ₹' || v_price
        );
    END LOOP;

    CLOSE plate_cursor;
END;
/
```

Output

Plate ID: 301 | Plate: Dinner Plate | Price: ₹550

Plate ID: 302 | Plate: Serving Plate | Price: ₹650

Plate ID: 304 | Plate: Decorative Plate | Price: ₹700

Plate ID: 305 | Plate: Melamine Plate | Price: ₹600

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of plates processed by the cursor.

```
SET SERVEROUTPUT ON;


DECLARE
   CURSOR plate_cursor IS
      SELECT plate_name, price
      FROM plate_shop;


   v_plate_name plate_shop.plate_name%TYPE;
v_price plate_shop.price%TYPE;


BEGIN
   OPEN plate_cursor;


   LOOP
      FETCH plate_cursor INTO v_plate_name, v_price;


      EXIT WHEN plate_cursor%NOTFOUND;


      DBMS_OUTPUT.PUT_LINE(
         'Plate: ' || v_plate_name ||
```

```
            ' | Price: ₹' || v_price
        );
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(
        'Total Plates Processed: ' || plate_cursor%ROWCOUNT
    );

    CLOSE plate_cursor;
END;
/
```

Output

Plate: Dinner Plate | Price: ₹550

Plate: Serving Plate | Price: ₹650

Plate: Dessert Plate | Price: ₹450

Plate: Decorative Plate | Price: ₹700

Plate: Melamine Plate | Price: ₹600

Total Plates Processed: 5

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process plate records one by one, apply conditions, and count the number of processed records.

EX. NO: 12	CREATION OF PL/SQL EXCEPTION HANDLING USING MOBILE TABLE
DATE:11/08/2026	

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
```

/

Types of Exceptions

Type Description
Predefined Exception Built-in exceptions provided by Oracle
User-Defined Exception Exception created by the programmer

1. CREATE TABLE CREATE TABLE

mobile (mobile_id NUMBER

PRIMARY KEY, mobile_name

VARCHAR2(50), price NUMBER

);

Table Created

Insert Records

INSERT INTO mobile VALUES (401, 'Samsung', 25000);

INSERT INTO mobile VALUES (402, 'iPhone', 30000);

INSERT INTO mobile VALUES (403, 'Redmi', 20000);

INSERT INTO mobile VALUES (404, 'OnePlus', 35000);

INSERT INTO mobile VALUES (405, 'Vivo', 40000);

COMMIT;

Records Inserted

Display Mobile Records

SELECT * FROM mobile;

Resultant Query

mobile_id	mobile_name	price
401	Samsung	25000
402	iPhone	30000
403	Redmi	20000
404	OnePlus	35000
405	Vivo	40000

2. NO_DATA_FOUND EXCEPTION

Definition

Program

SET SERVEROUTPUT ON;

```
DECLARE
```

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

```
v_mobile_name mobile.mobile_name%TYPE;     v_price mobile.price%TYPE;

BEGIN
   SELECT mobile_name, price
   INTO v_mobile_name, v_price
   FROM mobile
   WHERE mobile_id = 499;

   DBMS_OUTPUT.PUT_LINE('Mobile Name: ' || v_mobile_name);
   DBMS_OUTPUT.PUT_LINE('Price: ₹' || v_price);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Mobile not found.');
END;
/
```

Output

Exception: Mobile not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE



TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.
v_mobile_name mobile.mobile_name%TYPE;


BEGIN
   SELECT mobile_name
   INTO v_mobile_name
   FROM mobile
   WHERE price > 20000;


   DBMS_OUTPUT.PUT_LINE('Mobile Name: ' || v_mobile_name);


EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: More than one mobile found.'
      );
END;
/
```

Output

Exception: More than one mobile found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

Program

```
SET SERVEROUTPUT ON;
```

DECLARE

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE


   v_price NUMBER := 25000;
v_divisor NUMBER := 0;     v_result
NUMBER;


BEGIN
   v_result := v_price / v_divisor;


   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);


EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, a mobile purchase is allowed only when sufficient budget is available.

Program

```
SET SERVEROUTPUT ON;


DECLARE
```

```
   v_budget NUMBER := 25000;
v_price NUMBER := 35000;
insufficient_budget EXCEPTION;

BEGIN
   IF v_price > v_budget THEN
      RAISE insufficient_budget;
   END IF;

   EXCEPTION
      WHEN insufficient_budget THEN
         DBMS_OUTPUT.PUT_LINE(
            'Exception: Insufficient budget.'
         );
END;
/
```

Output

Exception: Insufficient budget.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid mobile purchase when sufficient budget is available.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_budget NUMBER := 50000;
v_price NUMBER := 5000; BEGIN
```

```
    IF v_price > v_budget THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient budget'
        );
    END IF;

    UPDATE mobile
    SET price = price
    WHERE mobile_id = 401;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Purchase Successful.');
    DBMS_OUTPUT.PUT_LINE(
        'Mobile Price: ₹' || v_price
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Purchase Successful.

Mobile Price: ₹5000

Display Updated Mobile

SELECT * FROM mobile

WHERE mobile_id = 401;

Resultant Query

mobile_id	mobile_name	price
401	Samsung	25000

Important Predefined Exceptions

Exception Description
NO_DATA_FOUND No record is found
TOO_MANY_ROWS More than one record is returned
ZERO_DIVIDE Division by zero occurs
DUP_VAL_ON_INDEX Duplicate value is inserted into a unique/primary key column
VALUE_ERROR Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented. The **NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions** were handled successfully using appropriate exception-handling mechanisms.

EX. NO: 13	**DATABASE NORMALIZATION OF DOLL**

DATE:12/08/2026	STORE

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) for a Mobile Store database to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Mobile Store table.

The table contains repeated information. For example, the category name and supplier name may be repeated for different mobiles.

Unnormalized Mobile Store Table

mobile_id	mobile_name	category_id	category_name	supplier_id	supplier_name	price
M01	Samsung Galaxy	M01	Android	S01	ABC Mobiles	25000
M02	iPhone 15	M02	iOS	S02	Apple Store	60000
M03	Redmi Note	M01	Android	S01	ABC Mobiles	18000
M04	OnePlus 12	M01	Android	S03	Smart Mobiles	45000

Problem:

Category and supplier information is repeated. This causes data redundancy and update anomalies.

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above Mobile Store table contains atomic values, so it can be considered as the 1NF table.

1NF Table

mobile_id	mobile_name	category_id	category_name	supplier_id	supplier_name	price
M01	Samsung Galaxy	C01	Android	S01	ABC Mobiles	25000
M02	iPhone 15	C02	iOS	S02	Apple Store	60000
M03	Redmi Note	C01	Android	S01	ABC Mobiles	18000
M04	OnePlus 12	C01	Android	S03	Smart Mobiles	45000

Primary Key: mobile_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

1. It is already in 1NF.
2. There is no partial dependency.
3. Every non-key attribute depends on the complete primary key.

For a Mobile Store transaction table, consider the composite key:

Primary Key = (order_id, mobile_id)

Some attributes depend only on mobile_id, such as:

- mobile_name
- category_id
- price

Therefore, these partial dependencies should be removed.

Functional Dependencies mobile_id → mobile_name,

category_id, price

category_id → category_name

supplier_id → supplier_name

Therefore, the table is divided into separate tables.

Mobile Table

mobile_id	mobile_name	category_id	price
M01	Samsung Galaxy	C01	25000
M02	iPhone 15	C02	60000
M03	Redmi Note	C01	18000
M04	OnePlus 12	C01	45000

Category Table

category_id	**category_name**
C01	Android
C02	iOS

Supplier Table

supplier_id	**supplier_name**
S01	ABC Mobiles
S02	Apple Store
S03	Smart MobilesSupplier Table

Order/Mobile Enrollment Table

order_id	mobile_id

O01	M01
O02	M02
O03	M03
O04	M04

The partial dependencies are removed.

Therefore, the tables are in **2NF**.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes depend only on the primary key.

Consider the Mobile table:

mobile_id → category_id and

category_id → category_name

Therefore:

mobile_id → category_id → category_name

Here, category_name has a transitive dependency on mobile_id.

To remove this dependency, category information is separated into another table.

Mobile Table

mobile_id	mobile_name	category_id	price
M01	Samsung Galaxy	C01	25000
M02	iPhone 15	C02	60000
M03	Redmi Note	C01	18000
M04	OnePlus 12	C01	45000

Category Table

category_id	category_name
C01	Android
C02	iOS

Supplier Table

supplier_id	supplier_name
S01	ABC Mobiles
S02	Apple Store
S03	Smart Mobiles

Order Table

order_id	mobile_id
O01	M01
O02	M02
O03	M03
O04	M04

Therefore, the tables satisfy **3NF**.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider the following Mobile_Store relation:

customer_id	mobile_id	staff_id
CU01	M01	E01
CU01	M02	E02
CU02	M01	E01
CU03	M03	E03

Assume that each staff member handles only one particular mobile.

Therefore:

staff_id → mobile_id

- But staff_id is not a candidate key of the original relation.

So the relation violates BCNF.

Therefore, it is decomposed into separate relations.

Staff-Mobile Table

staff_id	mobile_id
E01	M01
E02	M02
E03	M03

Customer-Staff Table

customer_id	staff_id
CU01	E01
CU01	E02
CU02	E01
CU03	E03

After decomposition, the determinant staff_id becomes a key in the Staff-Mobile relation.

Therefore, the relations satisfy **BCNF**.

FINAL NORMALIZED MOBILE STORE DATABASE

1. Mobile Table

mobile_id	mobile_name	category_id	price
M01	Samsung Galaxy	C01	25000
M02	iPhone 15	C02	60000
M03	Redmi Note	C01	18000
M04	OnePlus 12	C01	45000

2. Category Table

category_id	category_name
C01	Android
C02	iOS

3. Supplier Table

supplier_id	supplier_name
S01	ABC Mobiles
S02	Apple Store
S03	Smart Mobiles

4. Order Table

order_id	mobile_id
O01	M01
O02	M02
O03	M03
O04	M04

5. Staff-Mobile Table

staff_id	mobile_id
E01	M01
E02	M02
E03	M03

6. Customer-Staff Table

customer_id	staff_id
CU01	E01
CU01	E02
CU02	E01
CU03	E03

Result

Thus, the Database Normalization process for a Mobile Store was studied and implemented up to **1NF, 2NF, 3NF and BCNF**. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

EX. NO:14	MINI PROJECT
DATE:23/08/2026	HOSPITAL PATIENT REGISTRATION AND BILLING

Aim

To develop a Diabetes Risk Analysis and Patient Monitoring System using SQL and PL/SQL to store, manage, process, and analyze patient glucose, BMI, blood pressure, and diabetes outcome records.

Software Used

- GitHub Codespaces
- Oracle Database
- SQL
- PL/SQL

Description

The **Diabetes Risk Analysis and Patient Monitoring System** is developed to examine glucose, BMI, blood pressure, and diabetes outcome records of patients using the Pima Indians Diabetes dataset.

The system stores patient records using SQL. SQL queries are used to display patient records, identify patients with a positive diabetes outcome, calculate average glucose levels for each outcome group, retrieve patients with high glucose readings, and compare average BMI. PL/SQL functions and procedures are used to classify glucose levels, retrieve the measurements and outcome of a patient record, and generate a dataset summary while handling invalid patient IDs and missing measurements.

Database Tables

Column	Data Type	Constraint
PATIENT_ID	**NUMBER**	**PRIMARY KEY**
AGE	**NUMBER(3)**	**NOT NULL**
GLUCOSE	**NUMBER(5,1)**	**—**
BLOOD_PRESSURE	**NUMBER(5,1)**	**—**
BMI	**NUMBER(4,1)**	**—**
DIABETES_OUTCOME	**NUMBER(1)**	**NOT NULL (0 or 1)**

1. Table Creation

```
CREATE TABLE diabetes_patients (
  patient_id NUMBER PRIMARY KEY,
  age NUMBER(3) NOT NULL,
  glucose NUMBER(5,1),
  blood_pressure NUMBER(5,1),
  bmi NUMBER(4,1),
  diabetes_outcome NUMBER(1) NOT NULL
    CHECK (diabetes_outcome IN (0,1))
);
```

Table Created

The DIABETES_PATIENTS table was created successfully with primary key, NOT NULL and CHECK constraints.

2.Dataset Insertion — 25 Records

```
INSERT INTO diabetes_patients VALUES (101, 50, 148, 72, 33.6, 1);
INSERT INTO diabetes_patients VALUES (102, 31, 85, 66, 26.6, 0);
INSERT INTO diabetes_patients VALUES (103, 32, 183, 64, 23.3, 1);
INSERT INTO diabetes_patients VALUES (104, 21, 89, 66, 28.1, 0);
INSERT INTO diabetes_patients VALUES (105, 33, 137, 40, 43.1, 1);
INSERT INTO diabetes_patients VALUES (106, 30, 116, 74, 25.6, 0);
INSERT INTO diabetes_patients VALUES (107, 26, 78, 50, 31.0, 1);
INSERT INTO diabetes_patients VALUES (108, 29, 115, 70, 35.3, 0);
INSERT INTO diabetes_patients VALUES (109, 53, 197, 70, 30.5, 1);
INSERT INTO diabetes_patients VALUES (110, 54, 125, 96, 32.3, 1);
INSERT INTO diabetes_patients VALUES (111, 30, 110, 92, 37.6, 0);
INSERT INTO diabetes_patients VALUES (112, 34, 168, 74, 38.0, 1);
INSERT INTO diabetes_patients VALUES (113, 57, 139, 80, 27.1, 0);
INSERT INTO diabetes_patients VALUES (114, 59, 189, 60, 30.1, 1);
INSERT INTO diabetes_patients VALUES (115, 51, 166, 72, 25.8, 1);
INSERT INTO diabetes_patients VALUES (116, 32, 100, 88, 30.0, 0);
INSERT INTO diabetes_patients VALUES (117, 31, 118, 84, 45.8, 1);
INSERT INTO diabetes_patients VALUES (118, 31, 107, 74, 29.6, 1);
```

```
INSERT INTO diabetes_patients VALUES (119, 33, 103, 30, 43.3, 0);
INSERT INTO diabetes_patients VALUES (120, 27, 115, 70, 34.6, 0);
INSERT INTO diabetes_patients VALUES (121, 50, 126, 88, 39.3, 0);
INSERT INTO diabetes_patients VALUES (122, 41, 99, 84, 35.4, 0);
INSERT INTO diabetes_patients VALUES (123, 29, 196, 90, 39.8, 1);
INSERT INTO diabetes_patients VALUES (124, 51, 119, 80, 29.0, 1);
INSERT INTO diabetes_patients VALUES (125, 43, 143, 94, 36.6, 1);
COMMIT;
```

Records Inserted

A total of 25 patient records were inserted into the DIABETES_PATIENTS table.

Query 1 — Display all patients

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
ORDER BY patient_id;
```

Output

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     101    50     148           72  33.6       1
     102    31      85           66  26.6       0
     103    32     183           64  23.3       1
     104    21      89           66  28.1       0
     105    33     137           40  43.1       1
     106    30     116           74  25.6       0
     107    26      78           50  31.0       1
     108    29     115           70  35.3       0
     109    53     197           70  30.5       1
     110    54     125           96  32.3       1
     111    30     110           92  37.6       0
     112    34     168           74  38.0       1
     113    57     139           80  27.1       0
     114    59     189           60  30.1       1
     115    51     166           72  25.8       1
     116    32     100           88  30.0       0
     117    31     118           84  45.8       1
     118    31     107           74  29.6       1
     119    33     103           30  43.3       0
     120    27     115           70  34.6       0
     121    50     126           88  39.3       0
     122    41      99           84  35.4       0
     123    29     196           90  39.8       1
     124    51     119           80  29.0       1
     125    43     143           94  36.6       1


25 rows selected.
```

Query 2 — Identify records with a positive diabetes outcome

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
WHERE diabetes_outcome = 1
ORDER BY patient_id;
```

Output

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     101    50     148            72  33.6       1
     103    32     183            64  23.3       1
     105    33     137            40  43.1       1
     107    26      78            50  31.0       1
     109    53     197            70  30.5       1
     110    54     125            96  32.3       1
     112    34     168            74  38.0       1
     114    59     189            60  30.1       1
     115    51     166            72  25.8       1
     117    31     118            84  45.8       1
     118    31     107            74  29.6       1
     123    29     196            90  39.8       1
     124    51     119            80  29.0       1
     125    43     143            94  36.6       1

14 rows selected.
```

Query 3: Calculate average glucose level for each diabetes outcome group

```
SELECT diabetes_outcome,
    COUNT(*) AS total_patients,
    ROUND(AVG(glucose), 2) AS avg_glucose
FROM diabetes_patients
GROUP BY diabetes_outcome
ORDER BY diabetes_outcome;
```

Output

```
DIABETES_OUTCOME  TOTAL_PATIENTS  AVG_GLUCOSE
----------------  --------------  -----------
              0              11       108.82
              1              14       148.14

2 rows selected.
```

Query 4 — Retrieve patients with glucose readings above 140

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
WHERE glucose > 140
ORDER BY glucose DESC;
```

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     109    53     197            70  30.5       1
     123    29     196            90  39.8       1
     114    59     189            60  30.1       1
     103    32     183            64  23.3       1
     112    34     168            74  38.0       1
     115    51     166            72  25.8       1
     101    50     148            72  33.6       1
     125    43     143            94  36.6       1

8 rows selected.
```

Query 5: Calculate average BMI for patients with positive and negative diabetes outcomes

```
SELECT diabetes_outcome,
    ROUND(AVG(bmi), 2) AS avg_bmi
FROM diabetes_patients
GROUP BY diabetes_outcome
ORDER BY diabetes_outcome;
```

Output

```
DIABETES_OUTCOME  AVG_BMI
----------------  -------
              0    32.99
              1    33.46

2 rows selected.
```

PL/SQL Procedure 1:

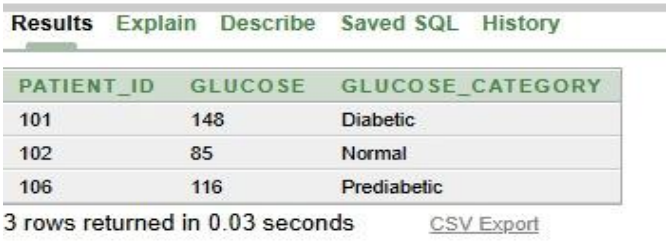
```
CREATE OR REPLACE FUNCTION classify_glucose (
  p_glucose IN NUMBER
)
RETURN VARCHAR2
IS
```

```
BEGIN
  IF p_glucose IS NULL THEN
    RETURN 'Not Recorded';
  ELSIF p_glucose < 100 THEN
    RETURN 'Normal';
  ELSIF p_glucose <= 125 THEN
    RETURN 'Prediabetic';
  ELSE
    RETURN 'Diabetic';
  END IF;
END;
/
```

Execution

```
SELECT patient_id,
       glucose,
       classify_glucose(glucose) AS glucose_category
FROM diabetes_patients
WHERE patient_id IN (101, 102, 106);
```

Output :



Results Explain Describe Saved SQL History

PATIENT_ID	GLUCOSE	GLUCOSE_CATEGORY
101	148	Diabetic
102	85	Normal
106	116	Prediabetic

3 rows returned in 0.03 seconds CSV Export

PL/SQL Function 2: Retrieve Patient Measurements and Outcome

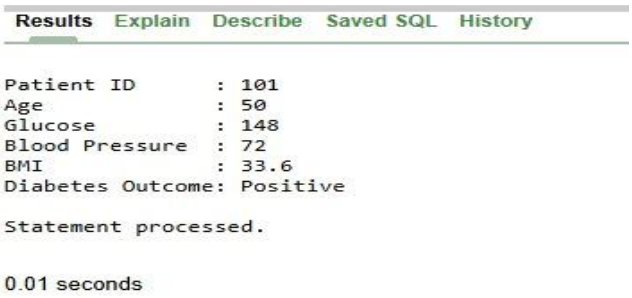
```
CREATE OR REPLACE PROCEDURE get_patient_record (
  p_patient_id IN NUMBER
)
IS
  v_age     diabetes_patients.age%TYPE;
  v_glucose diabetes_patients.glucose%TYPE;
  v_bp      diabetes_patients.blood_pressure%TYPE;
```

```
  v_bmi     diabetes_patients.bmi%TYPE;
  v_outcome diabetes_patients.diabetes_outcome%TYPE;
BEGIN
  SELECT age, glucose, blood_pressure, bmi, diabetes_outcome
  INTO v_age, v_glucose, v_bp, v_bmi, v_outcome
  FROM diabetes_patients
  WHERE patient_id = p_patient_id;
  DBMS_OUTPUT.PUT_LINE('Patient ID      : ' || p_patient_id);
  DBMS_OUTPUT.PUT_LINE('Age             : ' || v_age);
  DBMS_OUTPUT.PUT_LINE('Glucose         : ' || NVL(TO_CHAR(v_glucose), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('Blood Pressure  : ' || NVL(TO_CHAR(v_bp), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('BMI             : ' || NVL(TO_CHAR(v_bmi), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('Diabetes Outcome: ' ||
    CASE v_outcome WHEN 1 THEN 'Positive' ELSE 'Negative' END);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Patient ID not found.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
SET SERVEROUTPUT ON;
BEGIN
  get_patient_record(101);
END;
/
```

Output



PL/SQL Procedure 3: Generate Diabetes Dataset Summary

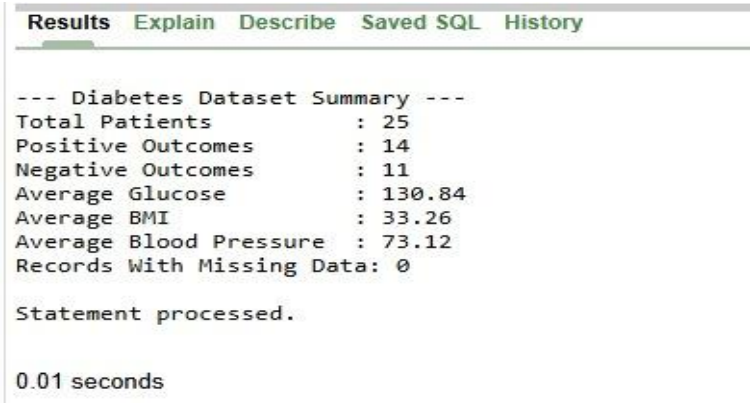
```
CREATE OR REPLACE PROCEDURE generate_diabetes_summary (
  p_patient_id IN NUMBER DEFAULT NULL
)
IS
  v_total    NUMBER;
  v_positive NUMBER;
  v_negative NUMBER;
  v_avg_glu  NUMBER;
  v_avg_bmi  NUMBER;
  v_avg_bp   NUMBER;
  v_missing  NUMBER;
  v_exists   NUMBER;
BEGIN
  -- Validate patient ID when one is supplied
  IF p_patient_id IS NOT NULL THEN
    SELECT COUNT(*) INTO v_exists
    FROM diabetes_patients
    WHERE patient_id = p_patient_id;
    IF v_exists = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Patient ID not found.');
      RETURN;
    END IF;
  END IF;
  SELECT COUNT(*),
        NVL(SUM(CASE WHEN diabetes_outcome = 1 THEN 1 ELSE 0 END), 0),
        NVL(SUM(CASE WHEN diabetes_outcome = 0 THEN 1 ELSE 0 END), 0),
        ROUND(AVG(glucose), 2),
        ROUND(AVG(bmi), 2),
        ROUND(AVG(blood_pressure), 2),
        NVL(SUM(CASE WHEN glucose IS NULL
                  OR bmi IS NULL
                  OR blood_pressure IS NULL
                 THEN 1 ELSE 0 END), 0)
  INTO v_total, v_positive, v_negative,
```

```
    v_avg_glu, v_avg_bmi, v_avg_bp, v_missing
  FROM diabetes_patients;
  IF v_total = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No patient records available.');
    RETURN;
  END IF;
  DBMS_OUTPUT.PUT_LINE('--- Diabetes Dataset Summary ---');
  DBMS_OUTPUT.PUT_LINE('Total Patients          : ' || v_total);
  DBMS_OUTPUT.PUT_LINE('Positive Outcomes       : ' || v_positive);
  DBMS_OUTPUT.PUT_LINE('Negative Outcomes       : ' || v_negative);
  DBMS_OUTPUT.PUT_LINE('Average Glucose         : ' || NVL(TO_CHAR(v_avg_glu), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Average BMI             : ' || NVL(TO_CHAR(v_avg_bmi), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Average Blood Pressure  : ' || NVL(TO_CHAR(v_avg_bp), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Records With Missing Data: ' || v_missing);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
BEGIN
  generate_diabetes_summary;
END;
/
```

Output



1. Glucose Category Report

Query

```
SELECT classify_glucose(glucose) AS glucose_category,
    COUNT(*) AS total_patients,
    ROUND(AVG(bmi), 2) AS avg_bmi
FROM diabetes_patients
GROUP BY classify_glucose(glucose)
ORDER BY total_patients DESC;
```

Output



Results Explain Describe Saved SQL History

GLUCOSE_CATEGORY	TOTAL_PATIENTS	AVG_BMI
Diabetic	11	33.38
Prediabetic	10	34.31
Normal	4	30.28

3 rows returned in 0.00 seconds CSV Export

2. Exception Handling

Test Case 1 – Invalid Patient ID for Patient Record Retrieval

```
BEGIN
  get_patient_record(999);
END;
/
```

Output



Results Explain Describe Saved SQL History

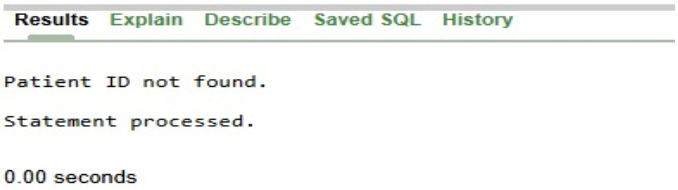
```
Patient ID not found.
Statement processed.

0.00 seconds
```

Test Case 2 – Invalid Patient ID for Dataset Summary

```
BEGIN
  generate_diabetes_summary(999);
END;
/
```

Output



Test Case 3 – Missing Glucose Measurement

```
SELECT classify_glucose(NULL) AS glucose_category
FROM dual;
```

Output



Dataset Insights

- The dataset contains 25 patient records from the Pima Indians Diabetes dataset.
- 14 patients have a positive diabetes outcome and 11 have a negative outcome.
- The average glucose of positive patients (148.14) is clearly higher than that of negative patients (108.82).
- The average BMI is similar for both groups (33.46 positive, 32.99 negative).
- 8 patients have a glucose reading above 140, and all of them have a positive outcome.
- Glucose categories help classify patients as Normal, Prediabetic, or Diabetic for monitoring.

4. Result

Thus, the Diabetes Risk Analysis and Patient Monitoring System was successfully implemented using SQL and PL/SQL. The diabetes patient table was created, 25 patient records were inserted, SQL queries were executed, PL/SQL functions and procedures were implemented, and exception handling for invalid patient IDs and missing measurements was tested successfully.

KHAN ACADEMY

PRACTICE PROGRAM

DATABASE TABLE FOR MY FAVORITE BOOKS

QUESTION

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.

Now, add three of your favorite books into the table.

ANSWERS

CREATE TABLE books (id

INTEGER PRIMARY KEY,

name TEXT NOT NULL,

rating INTEGER

);

INSERT INTO books (id, name, rating)

VALUES

(1, '1984', 10),

(2, 'To Kill a Mockingbird', 9),

(3, 'The Hobbit', 9);

OUTPUT



BOX OFFICE MOVIE EXPLORER

QUESTIONS

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step

ANSWERS

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994); INSERT

INTO movies VALUES (6, "Disney's Up", 2009); select*from

movies;

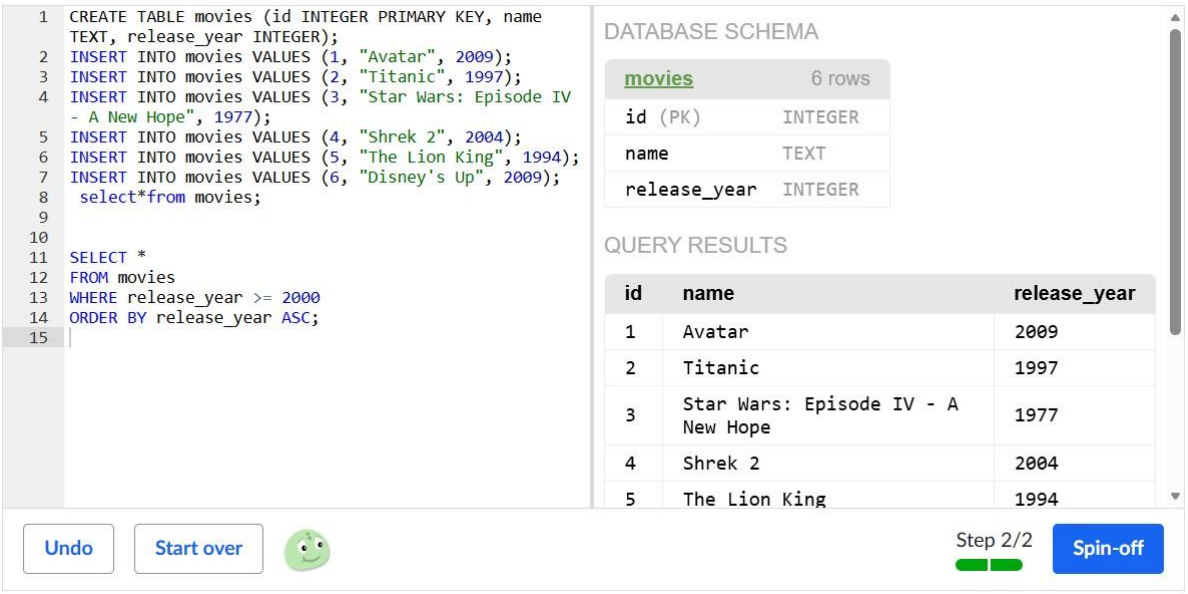
SELECT *

FROM movies

WHERE release_year >= 2000

ORDER BY release_year ASC;

OUTPUT



DATABASE FOR TODO LIST

QUESTIONS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take.

Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

ANSWER**S**

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

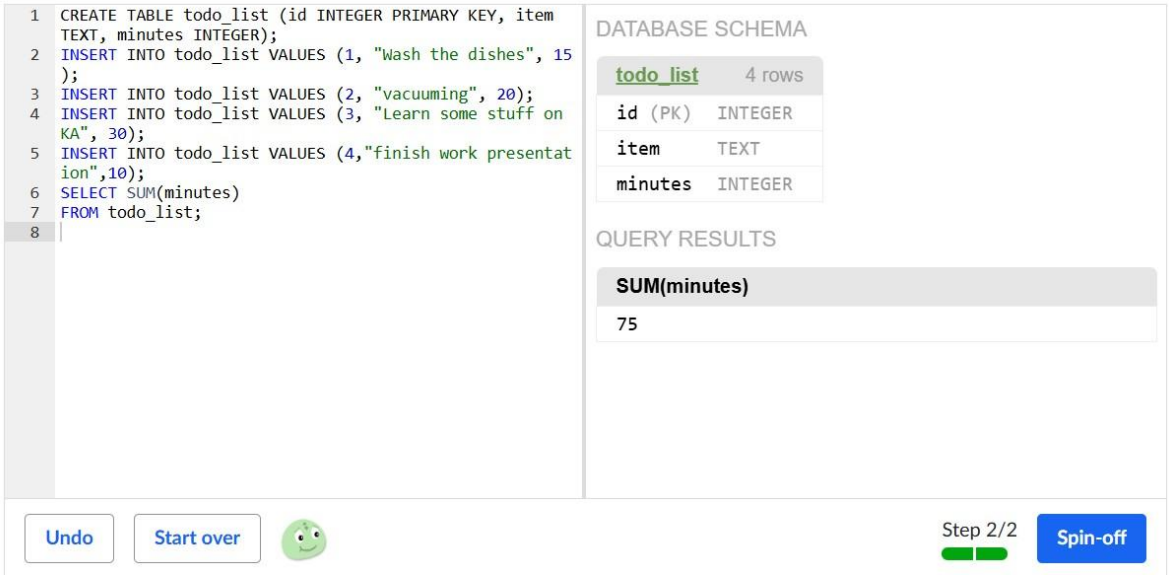
INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4,"finish work presentation",10);

SELECT SUM(minutes)

FROM todo_list;

OUTPUT



DATABASE FOR CLOTHING STORE

QUESTIONS

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

ANSWERS

CREATE TABLE clothing_store (

id INTEGER PRIMARY KEY,

item_name TEXT, category

TEXT, size TEXT, price

INTEGER, quantity INTEGER

);

INSERT INTO clothing_store VALUES (1, 'T-Shirt', 'Casual', 'M', 500, 20);

INSERT INTO clothing_store VALUES (2, 'Jeans', 'Casual', 'L', 1200, 15);

INSERT INTO clothing_store VALUES (3, 'Shirt', 'Formal', 'M', 900, 10);

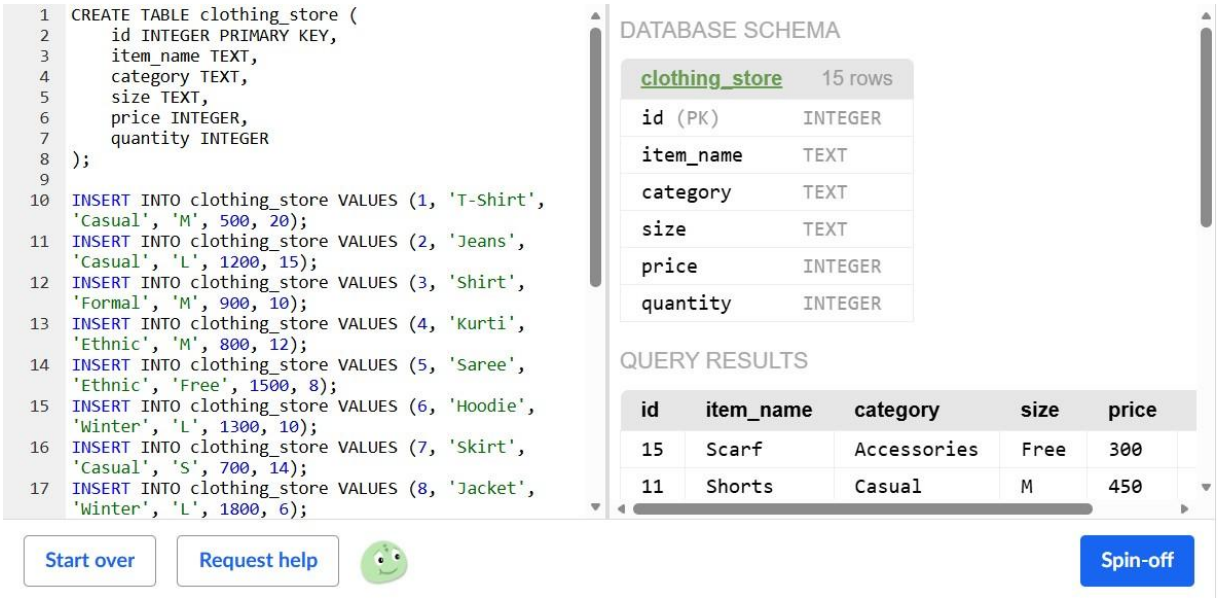
INSERT INTO clothing_store VALUES (4, 'Kurti', 'Ethnic', 'M', 800, 12);

```
INSERT INTO clothing_store VALUES (5, 'Saree', 'Ethnic', 'Free', 1500, 8);
INSERT INTO clothing_store VALUES (6, 'Hoodie', 'Winter', 'L', 1300, 10);
INSERT INTO clothing_store VALUES (7, 'Skirt', 'Casual', 'S', 700, 14);
INSERT INTO clothing_store VALUES (8, 'Jacket', 'Winter', 'L', 1800, 6);
INSERT INTO clothing_store VALUES (9, 'Top', 'Casual', 'S', 600, 18);
INSERT INTO clothing_store VALUES (10, 'Blazer', 'Formal', 'L', 2200, 5);
INSERT INTO clothing_store VALUES (11, 'Shorts', 'Casual', 'M', 450, 16);
INSERT INTO clothing_store VALUES (12, 'Sweater', 'Winter', 'M', 1100, 9);
INSERT INTO clothing_store VALUES (13, 'Lehenga', 'Ethnic', 'M', 2500, 4);
INSERT INTO clothing_store VALUES (14, 'Trousers', 'Formal', 'L', 1000, 11);
INSERT INTO clothing_store VALUES (15, 'Scarf', 'Accessories', 'Free', 300, 25);

SELECT * FROM clothing_store
ORDER BY price;

SELECT AVG(price) AS average_price
FROM clothing_store;
```

OUTPUT



KARAOKE SONG SELECTOR

QUESTIONS

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990.

(Tip: If you're not sure how to select rows where a column equals a value, remember that you can check the documentation below.)

People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', **and** released after 1990, **and** less than 4 minutes long. Note that the duration column is measured in seconds.

ANSWERS

CREATE TABLE clothing_store (

id INTEGER PRIMARY KEY,

item_name TEXT, category

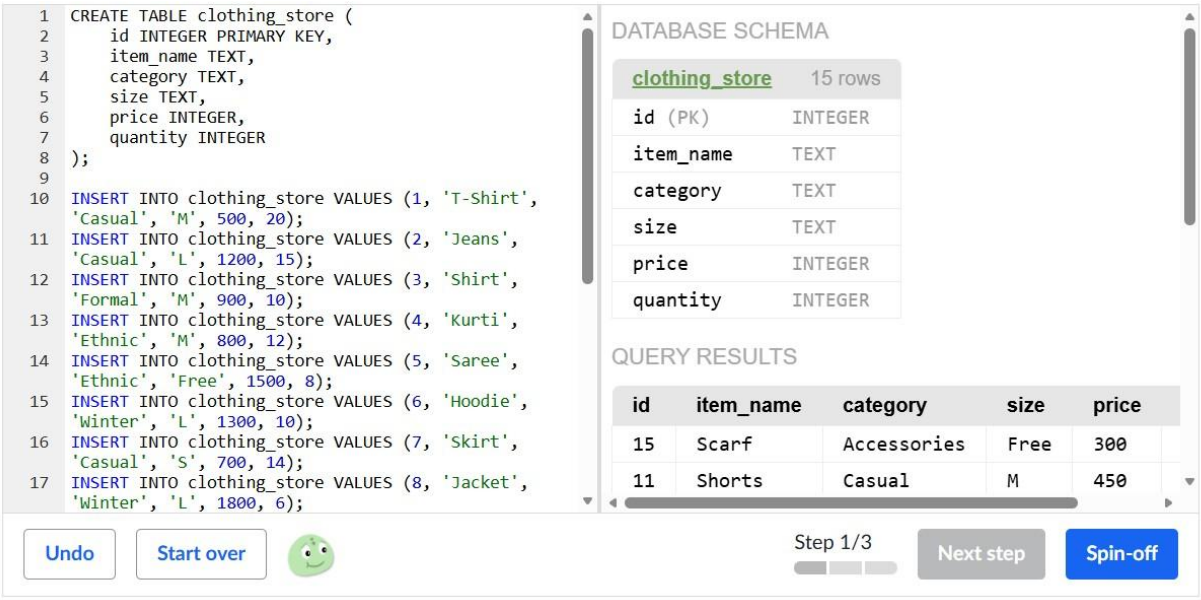
TEXT, size TEXT,

```
    price INTEGER,
quantity INTEGER
);

INSERT INTO clothing_store VALUES (1, 'T-Shirt', 'Casual', 'M', 500, 20);
INSERT INTO clothing_store VALUES (2, 'Jeans', 'Casual', 'L', 1200, 15);
INSERT INTO clothing_store VALUES (3, 'Shirt', 'Formal', 'M', 900, 10);
INSERT INTO clothing_store VALUES (4, 'Kurti', 'Ethnic', 'M', 800, 12);
INSERT INTO clothing_store VALUES (5, 'Saree', 'Ethnic', 'Free', 1500, 8);
INSERT INTO clothing_store VALUES (6, 'Hoodie', 'Winter', 'L', 1300, 10);
INSERT INTO clothing_store VALUES (7, 'Skirt', 'Casual', 'S', 700, 14);
INSERT INTO clothing_store VALUES (8, 'Jacket', 'Winter', 'L', 1800, 6);
INSERT INTO clothing_store VALUES (9, 'Top', 'Casual', 'S', 600, 18);
INSERT INTO clothing_store VALUES (10, 'Blazer', 'Formal', 'L', 2200, 5);
INSERT INTO clothing_store VALUES (11, 'Shorts', 'Casual', 'M', 450, 16);
INSERT INTO clothing_store VALUES (12, 'Sweater', 'Winter', 'M', 1100, 9);
INSERT INTO clothing_store VALUES (13, 'Lehenga', 'Ethnic', 'M', 2500, 4);
INSERT INTO clothing_store VALUES (14, 'Trousers', 'Formal', 'L', 1000, 11);
INSERT INTO clothing_store VALUES (15, 'Scarf', 'Accessories', 'Free', 300, 25);

SELECT * FROM clothing_store
ORDER BY price;

SELECT AVG(price) AS average_price
FROM clothing_store;
OUTPUT
```



DATABASE OF SONGS AND ARTISTS

QUESTIONS

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.

Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre.
(Tip: Make sure you type it 'Pop', SQL considers that different from 'pop'.)

To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

ANWERS

CREATE TABLE artists (id INTEGER PRIMARY

KEY AUTOINCREMENT,

name TEXT,

country TEXT,

genre TEXT);

INSERT INTO artists (name, country, genre)

VALUES ("Taylor Swift", "US", "Pop");

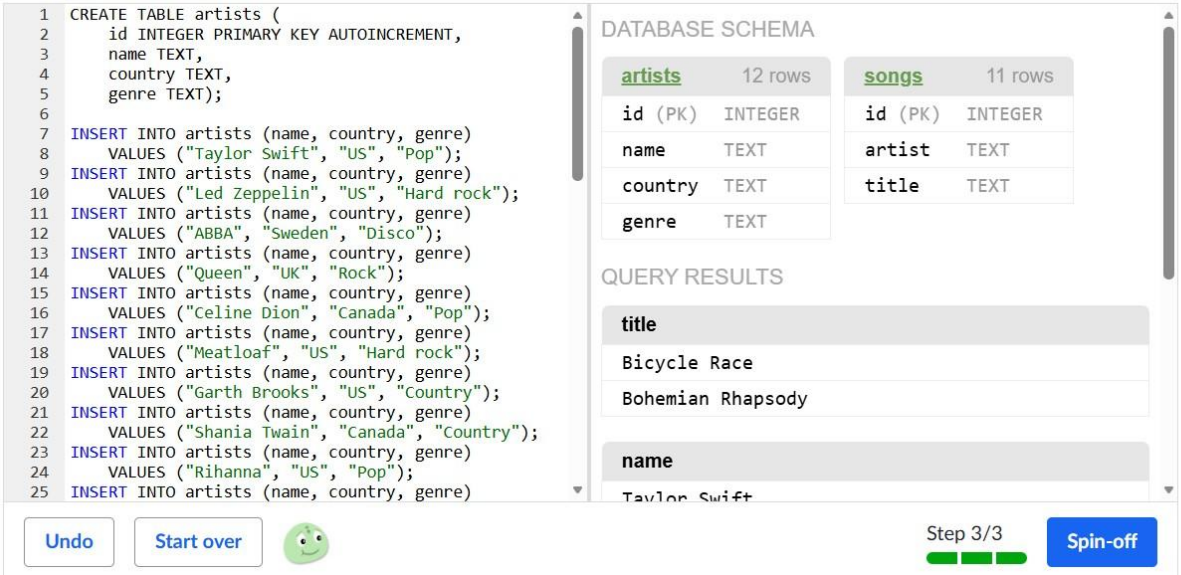
```
INSERT INTO artists (name, country, genre)
   VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
   VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Bob Marley", "Jamaica", "Reggae");

CREATE TABLE songs (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   artist TEXT,
title TEXT);
```

```
INSERT INTO
songs (artist, title)
   VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
   VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
   VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
   VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
   VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
   VALUES ("ABBA", "Mamma mia"); INSERT
INTO songs (artist, title)
   VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
   VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
   VALUES ("Guns N' Roses", "Don't cry");
   SELECT title
FROM songs
WHERE artist = 'Queen';
SELECT name
FROM artists
```

```
WHERE genre = 'Pop';
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```

OUTPUT



DATABASE OF A FEW POPULAR AUTHORS AND THEIR BOOKS

QUESTIONS

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

ANSWERS

```
CREATE TABLE books (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   author        TEXT,
title TEXT,     words
INTEGER);

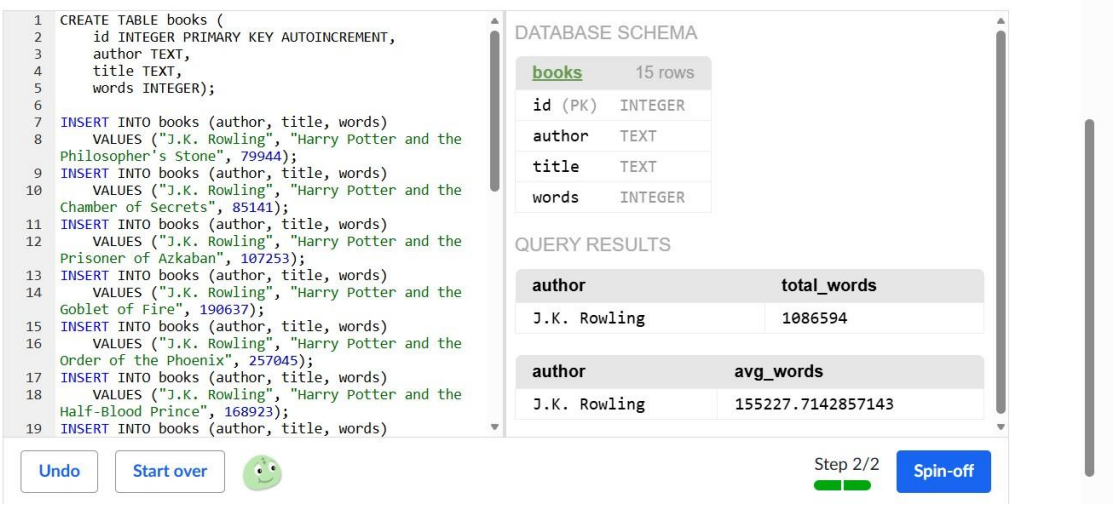

INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);


INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501);
```

```
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
     SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
SELECT author,
     AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```

OUTPUT



DATABASE TO TRACK STUDENT GRADES

QUESTIONS

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

ANSWERS

CREATE TABLE student_grades (id INTEGER

PRIMARY KEY AUTOINCREMENT,

name TEXT,

number_grade INTEGER,

fraction_completed REAL);

INSERT INTO student_grades (name, number_grade, fraction_completed)

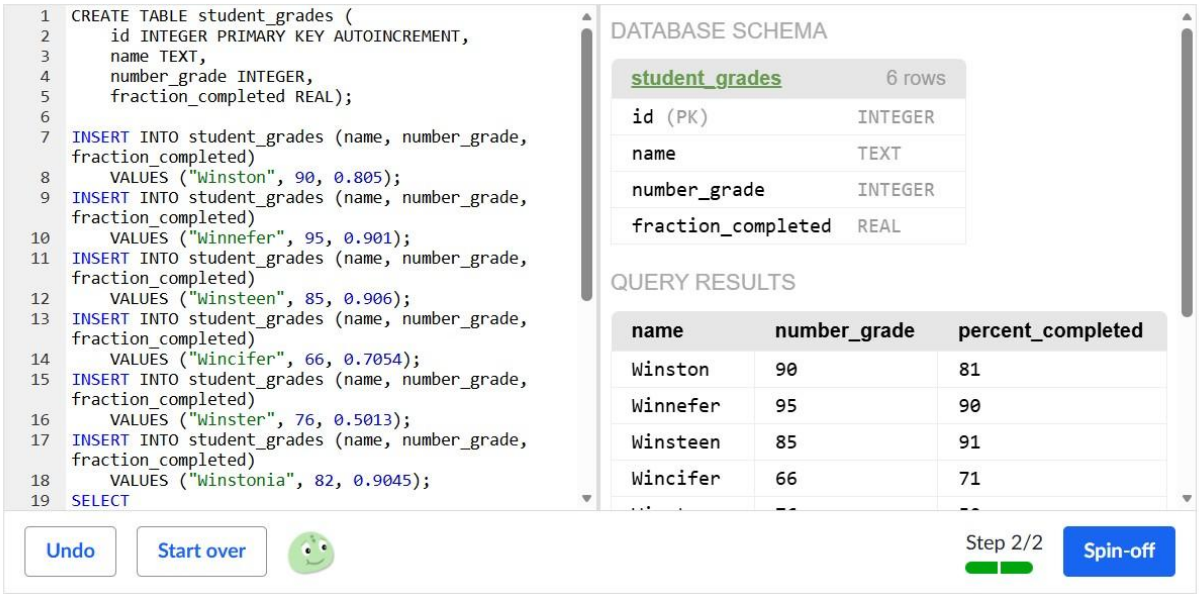
VALUES ("Winston", 90, 0.805);

INSERT INTO student_grades (name, number_grade, fraction_completed)

```
   VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);
SELECT
   name,
number_grade,
   ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;

/* Step 2 Query */
SELECT COUNT(*),
   CASE
      WHEN number_grade > 90 THEN "A"
      WHEN number_grade > 80 THEN "B"
WHEN number_grade > 70 THEN "C"
      ELSE "F"
   END AS "letter_grade"
FROM student_grades
GROUP BY letter_grade;

OUTPUT
```



POKEMON DATA ANALYSIS

QUESTIONS

We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.

Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

- What are average, max, and min values in the data?
- What about those numbers per category in the data (using HAVING)?
- What ways are there to group the data values that don't exist yet (using CASE)?
- What interesting ways are there to filter the data (using AND/OR)?

ANSWERS

CREATE TABLE pokemon (

id INTEGER PRIMARY KEY,

name TEXT, type TEXT, hp

INTEGER,

```
   attack INTEGER,
defense INTEGER,     speed
INTEGER
);

INSERT INTO pokemon VALUES
(1,'Bulbasaur','Grass',45,49,49,45),
(2,'Ivysaur','Grass',60,62,63,60),
(3,'Venusaur','Grass',80,82,83,80),
(4,'Charmander','Fire',39,52,43,65),
(5,'Charmeleon','Fire',58,64,58,80),
(6,'Charizard','Fire',78,84,78,100),
(7,'Squirtle','Water',44,48,65,43),
(8,'Wartortle','Water',59,63,80,58),
(9,'Blastoise','Water',79,83,100,78),
(10,'Pikachu','Electric',35,55,40,90);

-- 1. Average, Maximum, and Minimum Attack
SELECT AVG(attack) AS Avg_Attack,
      MAX(attack) AS Max_Attack,
      MIN(attack) AS Min_Attack
FROM pokemon;

-- 2. Average HP by Type
SELECT type, AVG(hp) AS Avg_HP
FROM pokemon
GROUP BY type;

-- 3. Types with Average Attack Greater Than 60
```

```
SELECT type, AVG(attack) AS Avg_Attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 60;

-- 4. Categorize Pokémon by Speed
SELECT name, speed,
CASE
   WHEN speed >= 90 THEN 'Very Fast'
   WHEN speed >= 70 THEN 'Fast'
   WHEN speed >= 50 THEN 'Medium'
   ELSE 'Slow'
END AS Speed_Category
FROM pokemon;

-- 5. High Attack and High Speed
SELECT *
FROM pokemon
WHERE attack >= 80 AND speed >= 80;

-- 6. Fire or Electric Pokémon
SELECT *
FROM pokemon
WHERE type = 'Fire' OR type = 'Electric';

-- 7. Strongest Attack by Type
SELECT type, MAX(attack) AS Strongest_Attack FROM pokemon
GROUP BY type;
```

-- 8. Count Pokémon by Type

SELECT type, COUNT(*) AS Total_Pokemon

FROM pokemon

GROUP BY type;

-- 9. Pokémon with High Defense

SELECT name, defense

FROM pokemon

WHERE defense > 75;

-- 10. Total Power Score

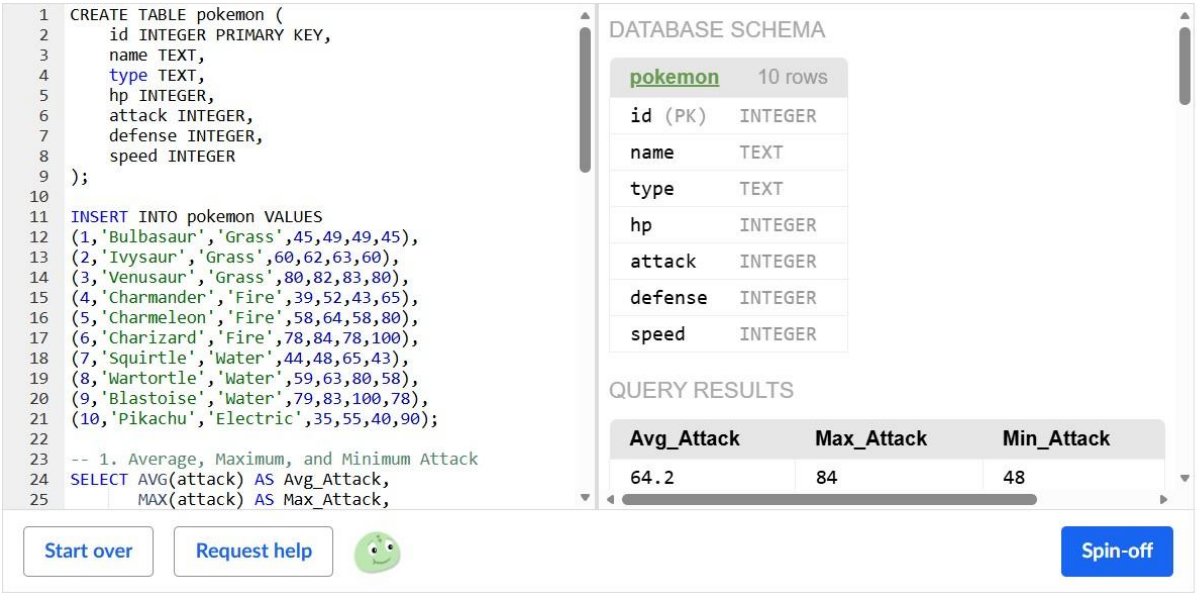
SELECT name,

(hp + attack + defense + speed) AS Total_Stats

FROM pokemon

ORDER BY Total_Stats DESC;

OUTPUT



DATABASE OF PEOPLE AND HOBBIES

QUESTIONS

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

Now, select the 2 tables with a join so that you can see each person's name next to their hobby.

Now, add an additional query that shows **only** the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

ANSWERS

```
CREATE TABLE persons (     id INTEGER
PRIMARY KEY AUTOINCREMENT,
   name TEXT,
age INTEGER);


INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);

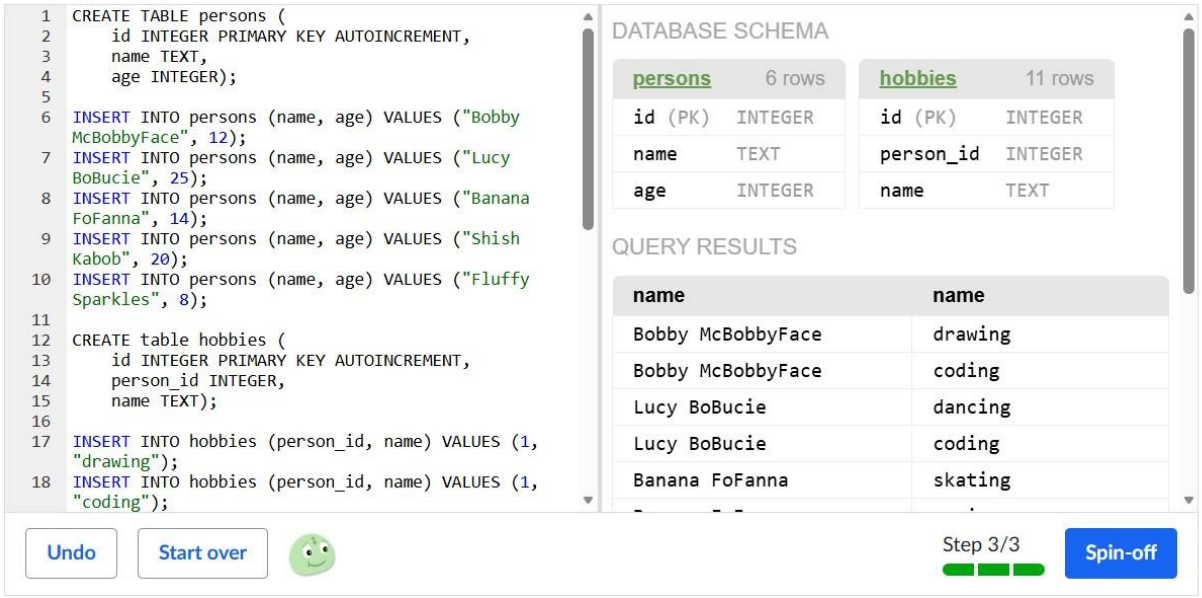

CREATE table hobbies (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   person_id INTEGER,
name TEXT);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Alex", 18);

INSERT INTO hobbies (person_id, name)
VALUES (6, "coding");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```

OUTPUT



DATABASE FOR CUSTOMERS AND THEIR ORDERS

QUESTIONS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.
(Tip: You should always GROUP BY on the column that is most likely to be unique in a row.)

ANSWERS

CREATE TABLE customers (id INTEGER

PRIMARY KEY AUTOINCREMENT,

name TEXT,

email TEXT);

INSERT INTO

customers (name,

email) VALUES

("Doctor Who",

```
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");

CREATE TABLE orders (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   customer_id INTEGER,
item TEXT,     price
REAL);

INSERT INTO orders (customer_id, item, price)
   VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
customers.email,        orders.item,
orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT
   customers.name,
customers.email,
SUM(orders.price) AS total_spent
FROM customers
```

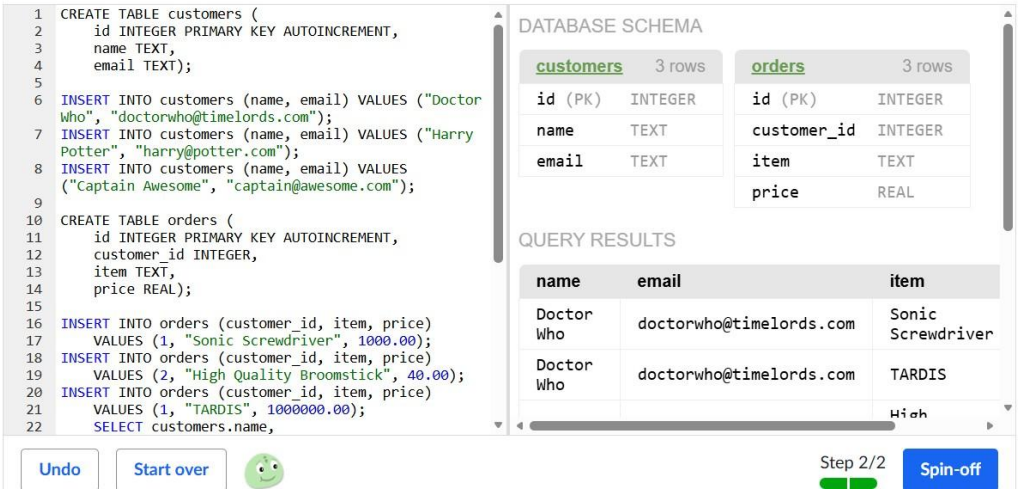
LEFT OUTER JOIN orders

ON customers.id = orders.customer_id

GROUP BY customers.id

ORDER BY total_spent DESC;

OUTPUT



DATABASE FOR ALL THE HARRY POTTER

QUESTIONS

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

ANSWERS

CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,

title TEXT, released

INTEGER, sequel_id

INTEGER); INSERT

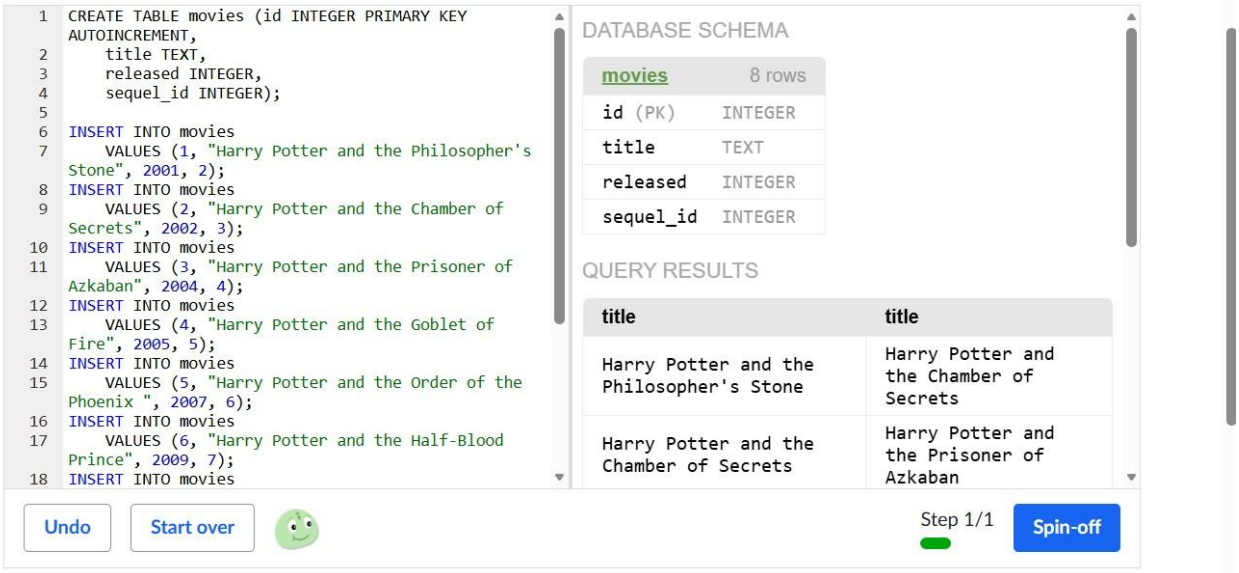
INTO movies

VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2); INSERT

INTO movies

```
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT
   m1.title,
m2.title
FROM movies m1
LEFT OUTER JOIN movies m2
ON m1.sequel_id = m2.id;
```

OUTPUT



DATABASE FOR A FRIEND NETWORKING SITE

QUESTIONS

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.

Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

ANSWERS

CREATE TABLE persons (id INTEGER

PRIMARY KEY AUTOINCREMENT,

fullname TEXT,

age INTEGER);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");

INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");

INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");

INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");

INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

```
CREATE table hobbies (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   person_id INTEGER,
name TEXT);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

CREATE table friends (     id INTEGER PRIMARY
KEY AUTOINCREMENT,
   person1_id INTEGER,
person2_id INTEGER);

INSERT INTO friends (person1_id, person2_id)
   VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);
   SELECT persons.fullname, hobbies.name
FROM persons
JOIN hobbies
```

ON persons.id = hobbies.person_id;

SELECT

p1.fullname AS person,

p2.fullname AS friend

FROM friends

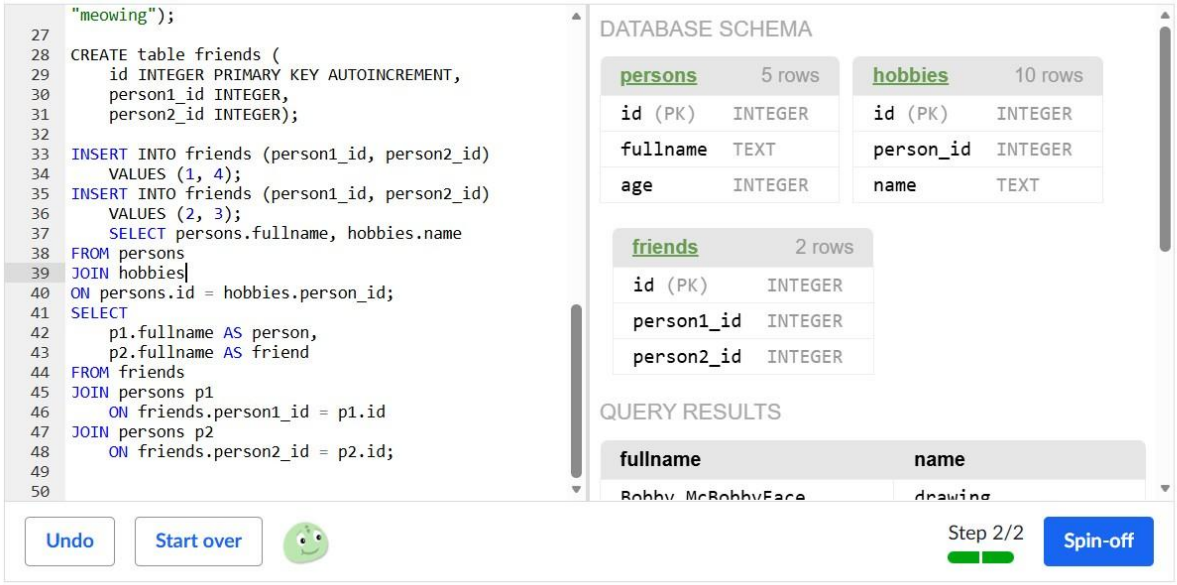
JOIN persons p1

ON friends.person1_id = p1.id

JOIN persons p2

ON friends.person2_id = p2.id;

OUTPUT



SIMPLE MOVIE STARS DATABASE

QUESTIONS

In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

- Movie stars: What movies are they in? Are they married to each other?

- Singers: What songs did they write? Where are they from?
- Authors: What books did they write?
- Fictional characters: How are they related to other characters? What books do they show up in?

ANSWERS

```
CREATE TABLE actors (     id
INTEGER PRIMARY KEY,
name TEXT,     age INTEGER,
country TEXT
);

INSERT INTO actors VALUES (1, 'Tom Hanks', 68, 'USA');
INSERT INTO actors VALUES (2, 'Emma Watson', 35, 'UK');
INSERT INTO actors VALUES (3, 'Leonardo DiCaprio', 50, 'USA');
INSERT INTO actors VALUES (4, 'Angelina Jolie', 50, 'USA');
INSERT INTO actors VALUES (5, 'Robert Downey Jr.', 60, 'USA');

CREATE TABLE movies (     id
INTEGER PRIMARY KEY,
actor_id INTEGER,
movie_name TEXT,     year
INTEGER
);

INSERT INTO movies VALUES (1, 1, 'Forrest Gump', 1994);
INSERT INTO movies VALUES (2, 1, 'Cast Away', 2000);
INSERT INTO movies VALUES (3, 2, 'Harry Potter', 2001);
INSERT INTO movies VALUES (4, 2, 'Beauty and the Beast', 2017);
```

INSERT INTO movies VALUES (5, 3, 'Titanic', 1997);

INSERT INTO movies VALUES (6, 3, 'Inception', 2010);

INSERT INTO movies VALUES (7, 4, 'Maleficent', 2014);

INSERT INTO movies VALUES (8, 4, 'Mr. & Mrs. Smith', 2005);

INSERT INTO movies VALUES (9, 5, 'Iron Man', 2008);

INSERT INTO movies VALUES (10, 5, 'Sherlock Holmes', 2009);

SELECT actors.name, movies.movie_name, movies.year

FROM actors

JOIN movies

ON actors.id = movies.actor_id;

OUTPUT

```
CREATE TABLE actors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    country TEXT
);

INSERT INTO actors VALUES (1, 'Tom Hanks', 68, 'USA');
INSERT INTO actors VALUES (2, 'Emma Watson', 35, 'UK');
INSERT INTO actors VALUES (3, 'Leonardo DiCaprio', 50, 'USA');
INSERT INTO actors VALUES (4, 'Angelina Jolie', 50, 'USA');
INSERT INTO actors VALUES (5, 'Robert Downey Jr.', 60, 'USA');

CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    actor_id INTEGER,
    movie_name TEXT,
    year INTEGER
);
```

DATABASE SCHEMA

actors	5 rows
id (PK)	INTEGER
name	TEXT
age	INTEGER
country	TEXT

movies	10 rows
id (PK)	INTEGER
actor_id	INTEGER
movie_name	TEXT
year	INTEGER

QUERY RESULTS

name	movie_name	year
Tom Hanks	Forrest Gump	1994
Tom Hanks	Cast Away	2000
Emma Watson	Harry Potter	2001
Emma Watson	Beauty and the Beast	2017

Start over | Request help | Spin-off

DATABASE FOR A DOCUMENTS APP

QUESTIONS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all

rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

ANSWERS

CREATE table documents (id INTEGER

PRIMARY KEY AUTOINCREMENT,

title TEXT,

content TEXT,

author TEXT);

INSERT INTO documents (author, title, content)

VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");

INSERT INTO documents (author, title, content)

VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");

INSERT INTO documents (author, title, content)

VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");

INSERT INTO documents (author, title, content)

VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");

INSERT INTO documents (author, title, content)

VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");

SELECT * FROM documents;

UPDATE documents

OUTPUT



DATABASE OF CLOTHES

QUESTIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now.

Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected.

Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

ANSWERS

CREATE TABLE clothes (id INTEGER PRIMARY

KEY AUTOINCREMENT,

type TEXT,

design TEXT);

INSERT INTO clothes (type, design)

VALUES ("dress", "pink polka dots");

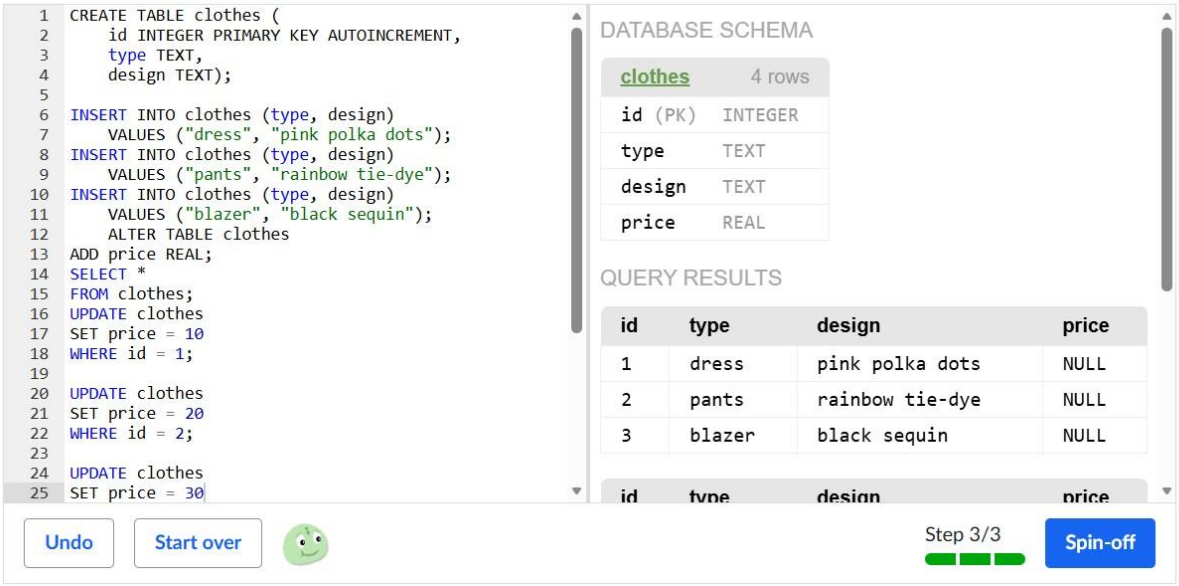
```
INSERT INTO clothes (type, design)
   VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
   VALUES ("blazer", "black sequin");
   ALTER TABLE clothes
ADD price REAL;
SELECT *
FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;

UPDATE clothes
SET price = 20
WHERE id = 2;

UPDATE clothes
SET price = 30
WHERE id = 3;

SELECT * FROM clothes;
-- Insert a new item with all attributes, including price
INSERT INTO clothes (type, design, price)
   VALUES ("jacket", "vintage denim", 45.0);

-- Final SELECT to verify the entire table
SELECT * FROM clothes;
OUTPUT
```



SHOPPING APP - STORES PRODUCTS/ORDERS

QUESTIONS

Think about your favorite apps, and pick one that stores your data- like a game that stores scores, an app that lets you post updates, etc. Now in this project, you're going to imagine that the app stores your data in a SQL database (which is pretty likely!), and write SQL statements that might look like their own SQL.

- CREATE a table to store the data.
- INSERT a few example rows in the table.
- Use an UPDATE to emulate what happens when you edit data in the app.
- Use a DELETE to emulate what happens when you delete data in the app.

ANSWERS

-- Create a table for notes

CREATE TABLE notes (id

INTEGER PRIMARY KEY,

title TEXT,

content TEXT,

created_date TEXT);

```
-- Insert example notes
INSERT INTO notes VALUES
(1, 'Shopping List', 'Buy milk, eggs, and bread', '2026-07-02');

INSERT INTO notes VALUES
(2, 'Homework', 'Complete SQL project', '2026-07-02');

INSERT INTO notes VALUES
(3, 'Workout', 'Run 5 km in the evening', '2026-07-02');

-- View all notes
SELECT * FROM notes;

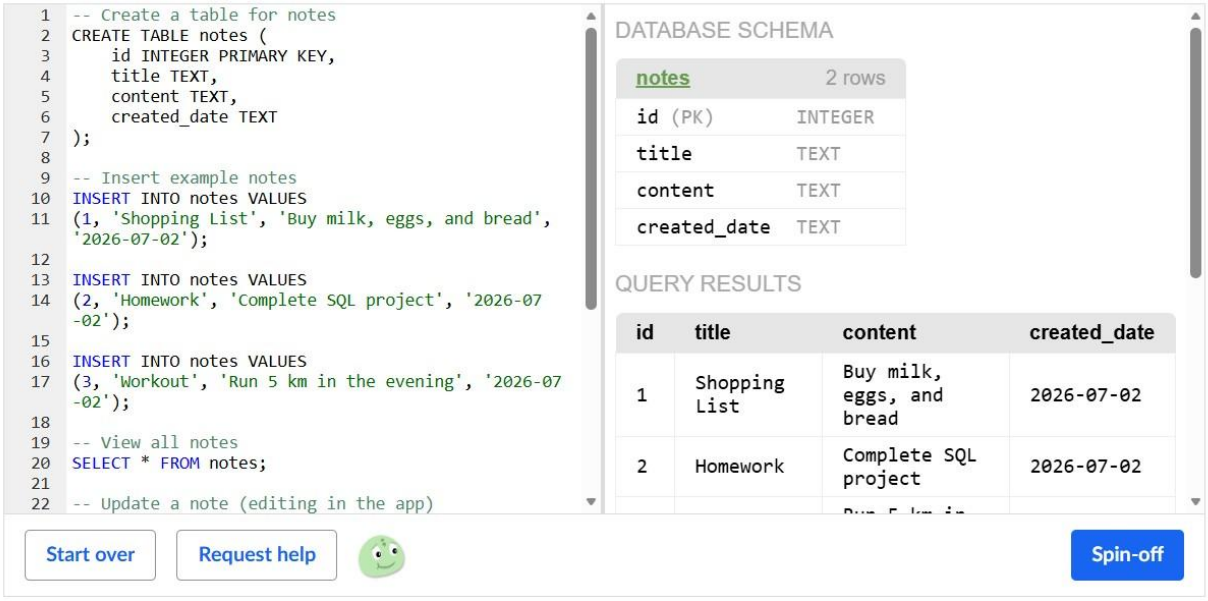
-- Update a note (editing in the app)
UPDATE notes
SET content = 'Buy milk, eggs, bread, and fruits'
WHERE id = 1;

-- View updated note
SELECT * FROM notes WHERE id = 1;

-- Delete a note (removing it from the app)
DELETE FROM notes
WHERE id = 3;

-- View remaining notes
SELECT * FROM notes;/* What does the app's SQL look like? */
```

OUTPUT



```sql
-- Create a table for notes
CREATE TABLE notes (
    id INTEGER PRIMARY KEY,
    title TEXT,
    content TEXT,
    created_date TEXT
);

-- Insert example notes
INSERT INTO notes VALUES
(1, 'Shopping List', 'Buy milk, eggs, and bread',
'2026-07-02');

INSERT INTO notes VALUES
(2, 'Homework', 'Complete SQL project', '2026-07
-02');

INSERT INTO notes VALUES
(3, 'Workout', 'Run 5 km in the evening', '2026-07
-02');

-- View all notes
SELECT * FROM notes;

-- Update a note (editing in the app)
```


HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the CITY table with populations larger than 100000. The CountryCode for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

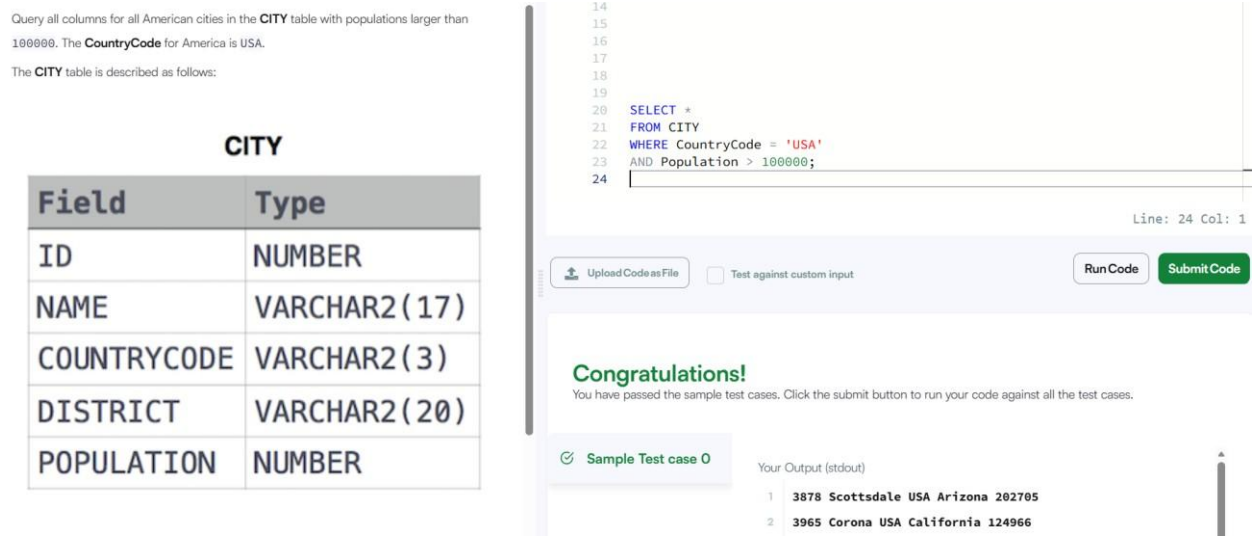
SELECT *

FROM CITY

WHERE CountryCode = 'USA'

AND Population > 100000;

OUTPUT



REVISING THE SELECT QUERY II

QUSTIONS

Query the NAME field for all American cities in the CITY table with populations larger than 120000. The *CountryCode* for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWERS

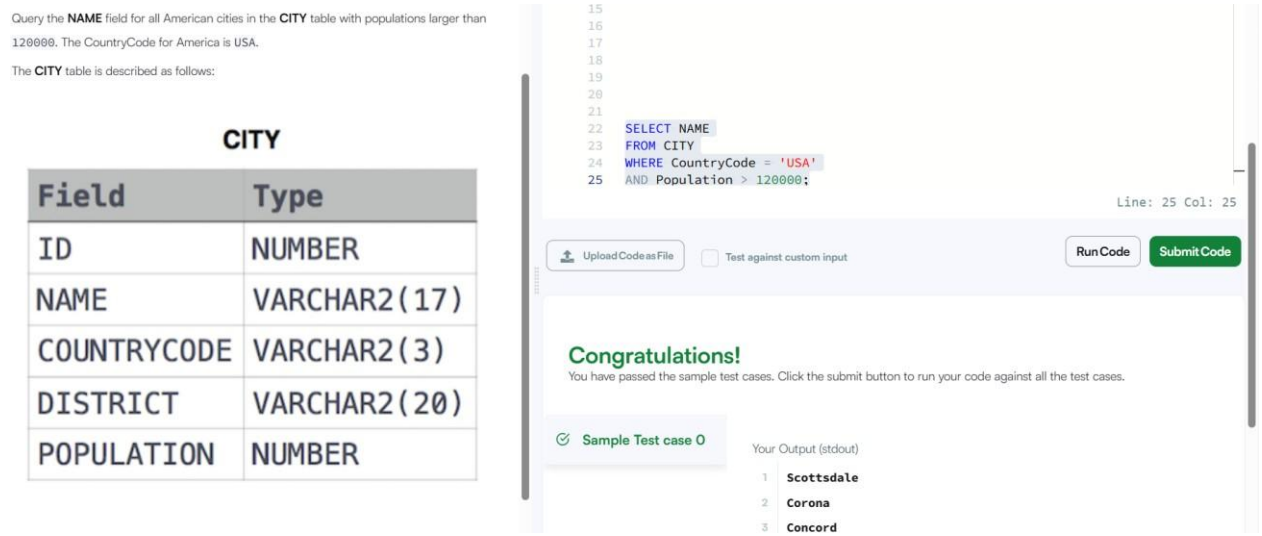
SELECT NAME

FROM CITY

WHERE CountryCode = 'USA'

AND Population > 120000;

OUTPUT



SELECT ALL

QUESTIONS

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWERS

SELECT *

FROM CITY;

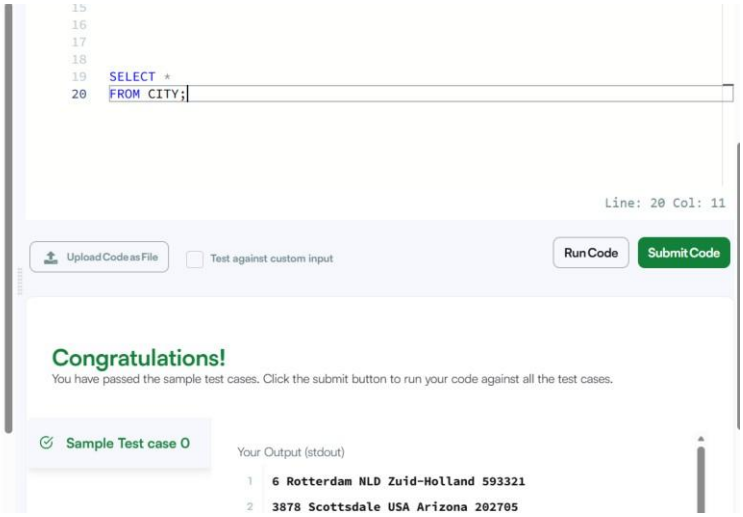
OUTPUT

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



SELECT BY ID

QUESTIONS

Query all columns for a city in **CITY** with the *ID* 1661.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWERS

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT



QUESTIONS

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWERS

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

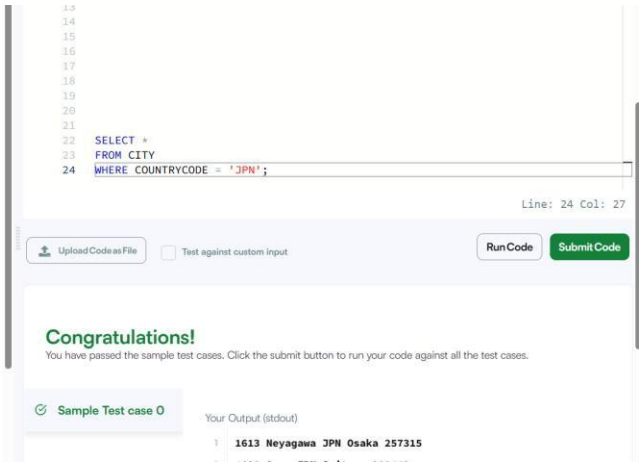
OUTPUT

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is **JPN**.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



QUESTIONS

Query a list of **CITY** and **STATE** from the **STATION** table. The **STATION** table is described as follows:

STATION

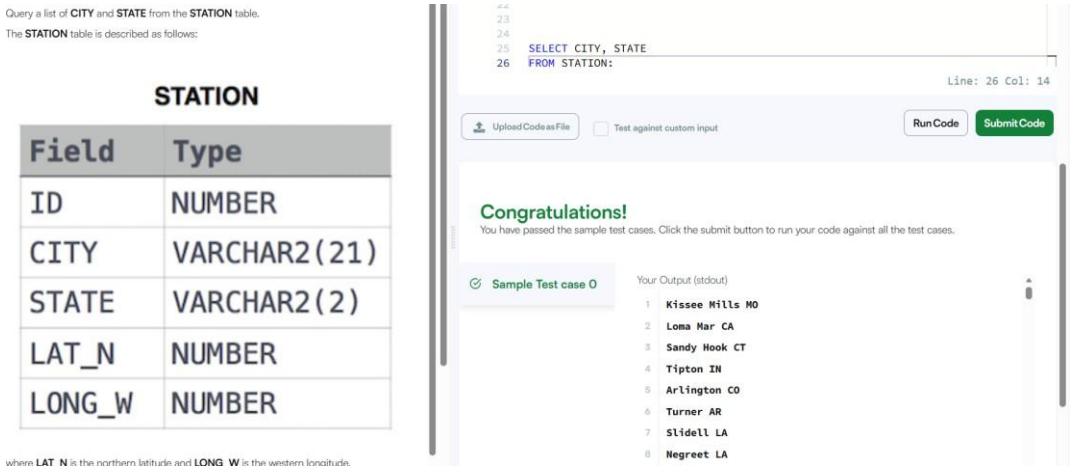
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

SELECT CITY, STATE

FROM STATION;

OUTPUT



QUESTIONS

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer. The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

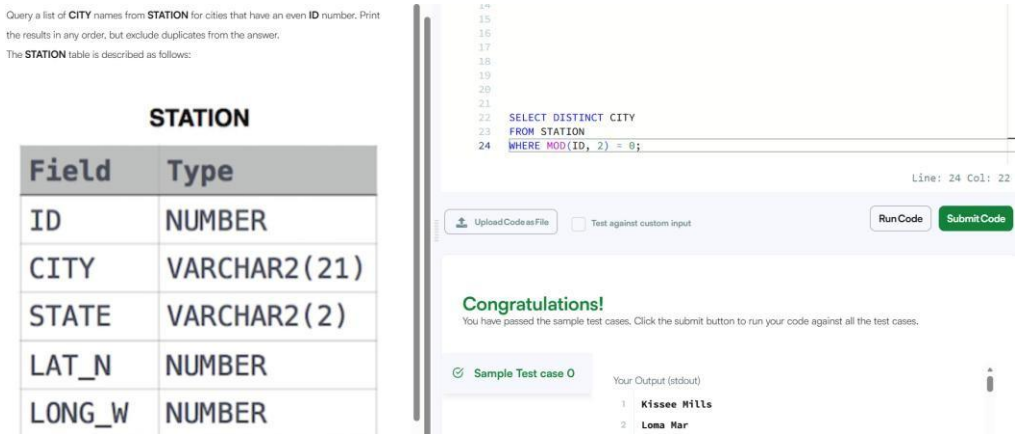
ANSWERS

SELECT DISTINCT CITY

FROM STATION

WHERE MOD(ID, 2) = 0;

OUTPUT



QUESTIONS

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:

STATION

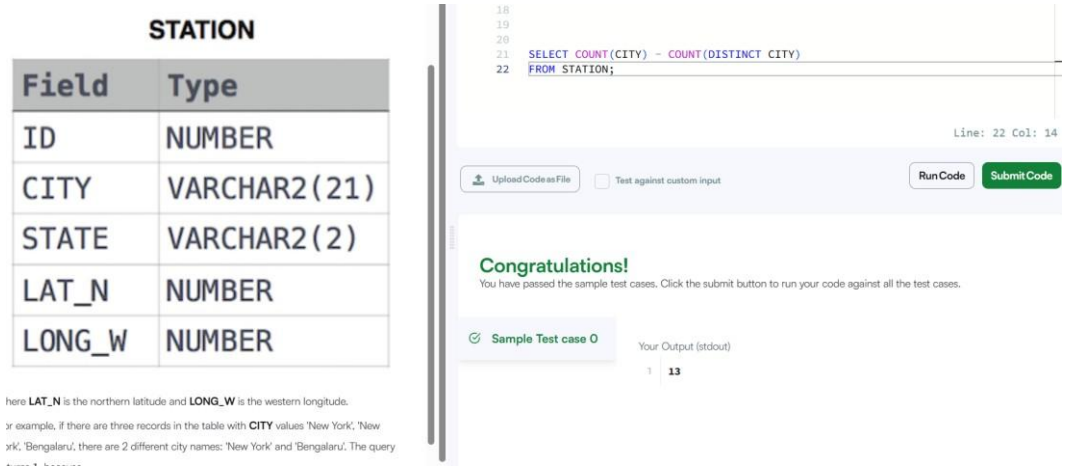
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

SELECT COUNT(CITY) - COUNT(DISTINCT CITY)

FROM STATION;

OUTPUT



STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

here **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

or example, if there are three records in the table with **CITY** values 'New York', 'New ork', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'. The query

```
18
19
20
21  SELECT COUNT(CITY) - COUNT(DISTINCT CITY)
22  FROM STATION;
```

Line: 22 Col: 14

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  13
```

QUESTIONS

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically. The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Sample Input

ANSWERS

SELECT CITY, LENGTH(CITY)

FROM STATION

ORDER BY LENGTH(CITY), CITY

LIMIT 1;

SELECT CITY, LENGTH(CITY)

FROM STATION

ORDER BY LENGTH(CITY) DESC, CITY

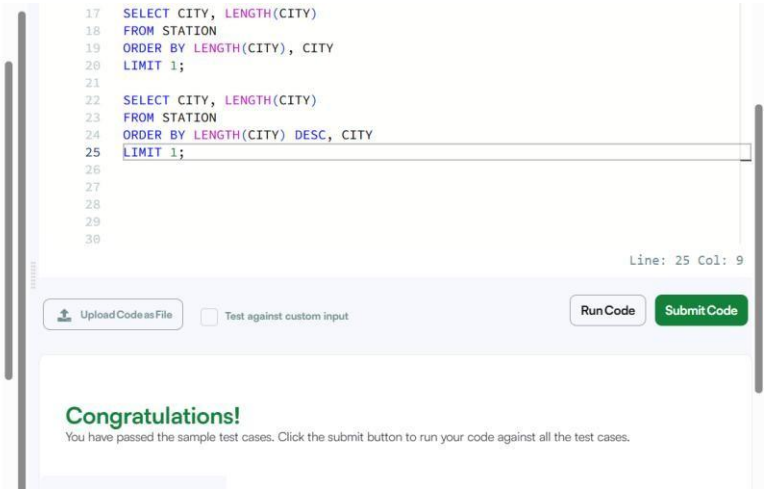
LIMIT 1;

OUTPUT

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ıere **LAT_N** is the northern latitude and **LONG_W** is the western longitude.



QUESTIONS

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

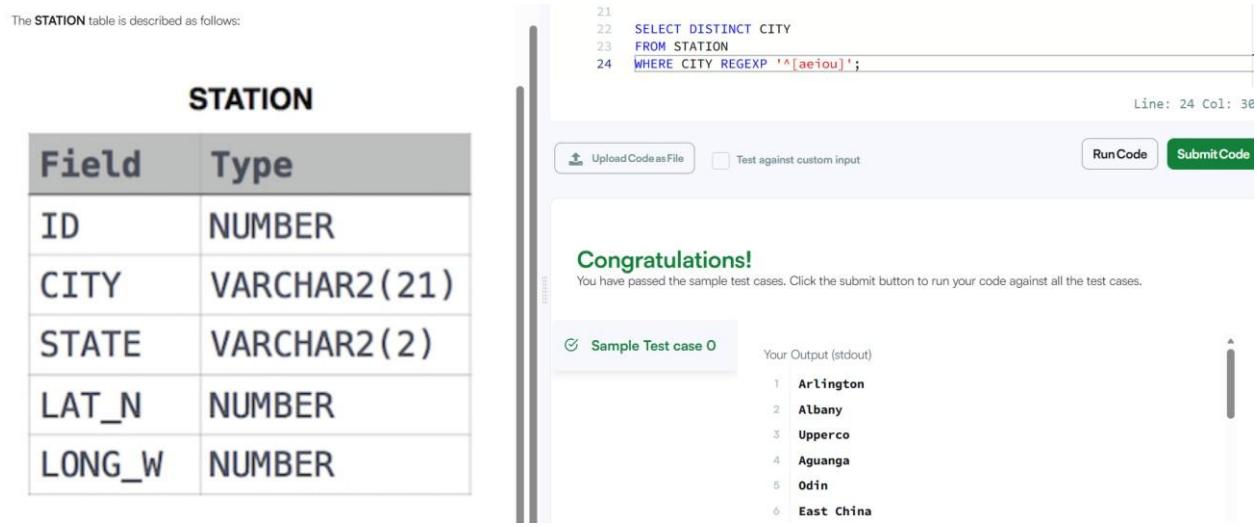
ANSWERS

SELECT DISTINCT CITY

FROM STATION

WHERE CITY REGEXP '^[aeiou]';

OUTPUT



QUESTIONS

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

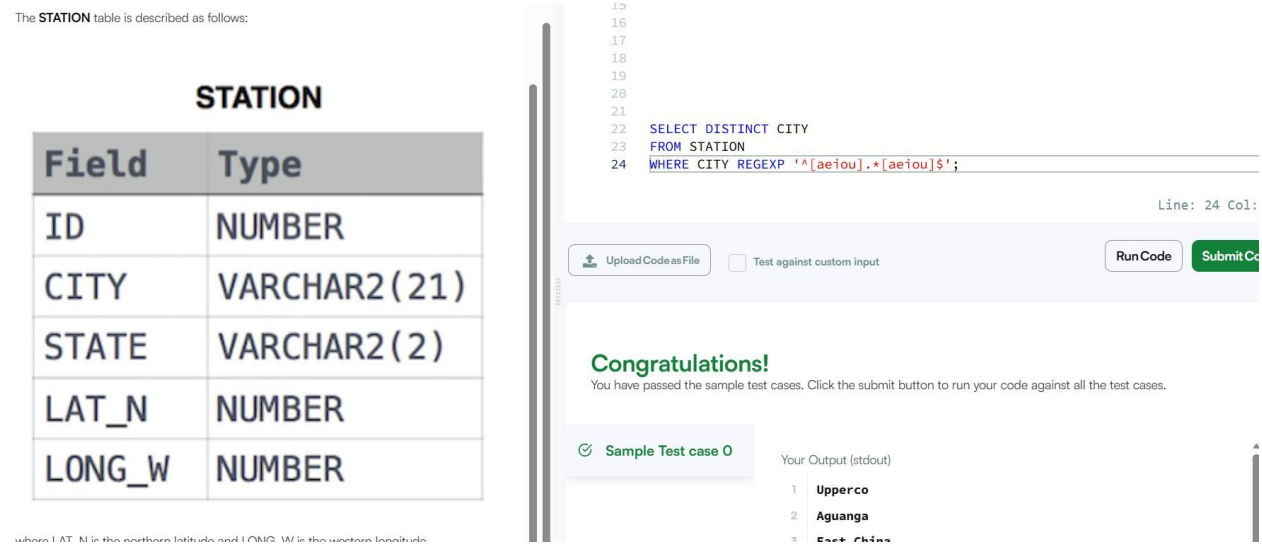
ANSWERS

SELECT DISTINCT CITY

FROM STATION

WHERE CITY REGEXP '^[aeiou].*[aeiou]$';

OUTPUT



QUESTIONS

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

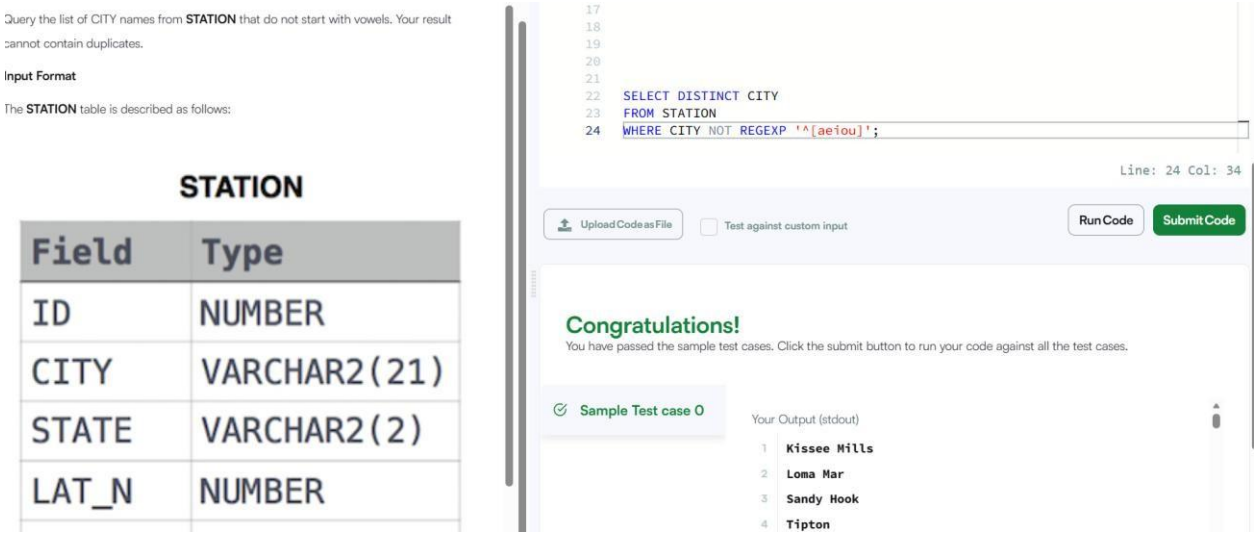
ANSWERS

SELECT DISTINCT CITY

FROM STATION

WHERE CITY NOT REGEXP '^[aeiou]';

OUTPUT



QUESTIONS

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

SELECT DISTINCT CITY

FROM STATION

WHERE CITY NOT REGEXP '[aeiou]$';

OUTPUT



QUESTIONS

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

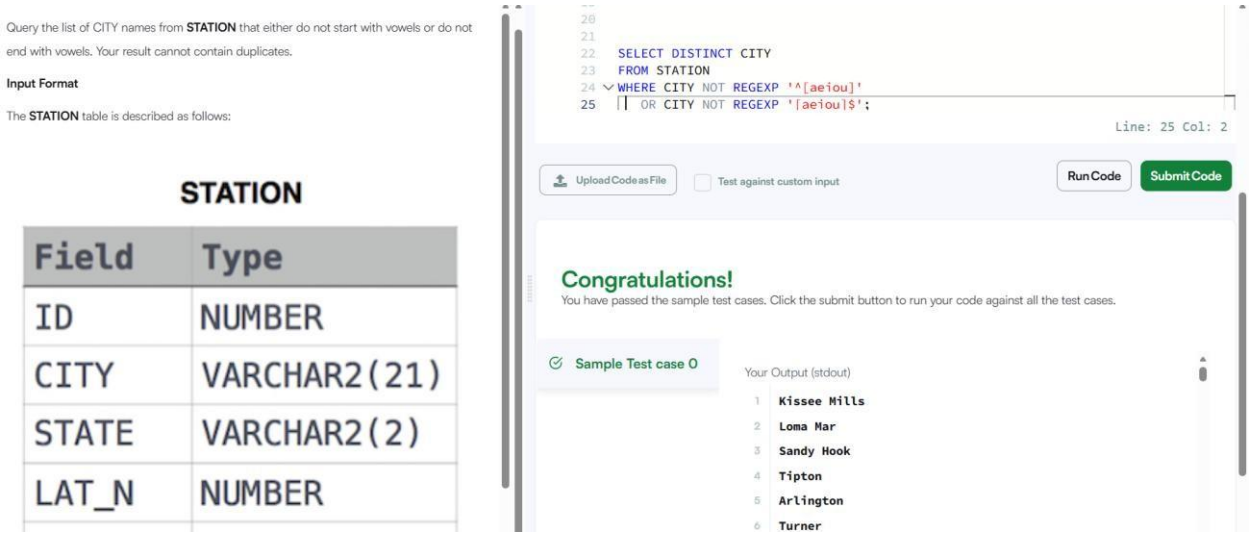
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[aeiou]'
   OR CITY NOT REGEXP '[aeiou]$';
```

OUTPUT



QUESTIONS

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

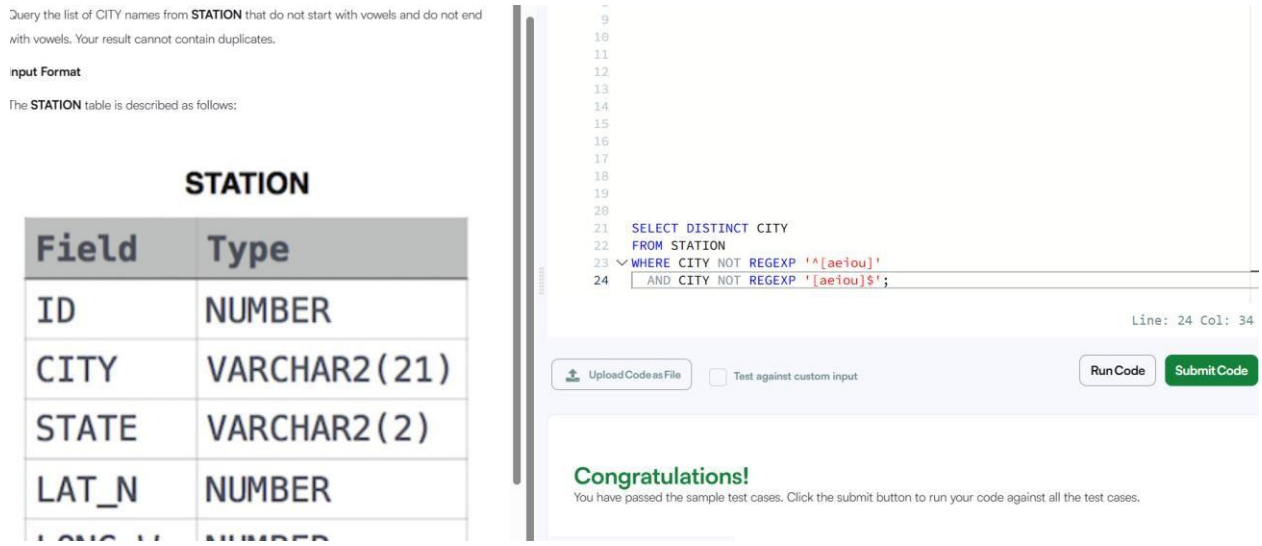
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[aeiou]'
  AND CITY NOT REGEXP '[aeiou]$';
```

OUTPUT



QUESTIONS:

Query the *Name* of any student in **STUDENTS** who scored higher than

Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

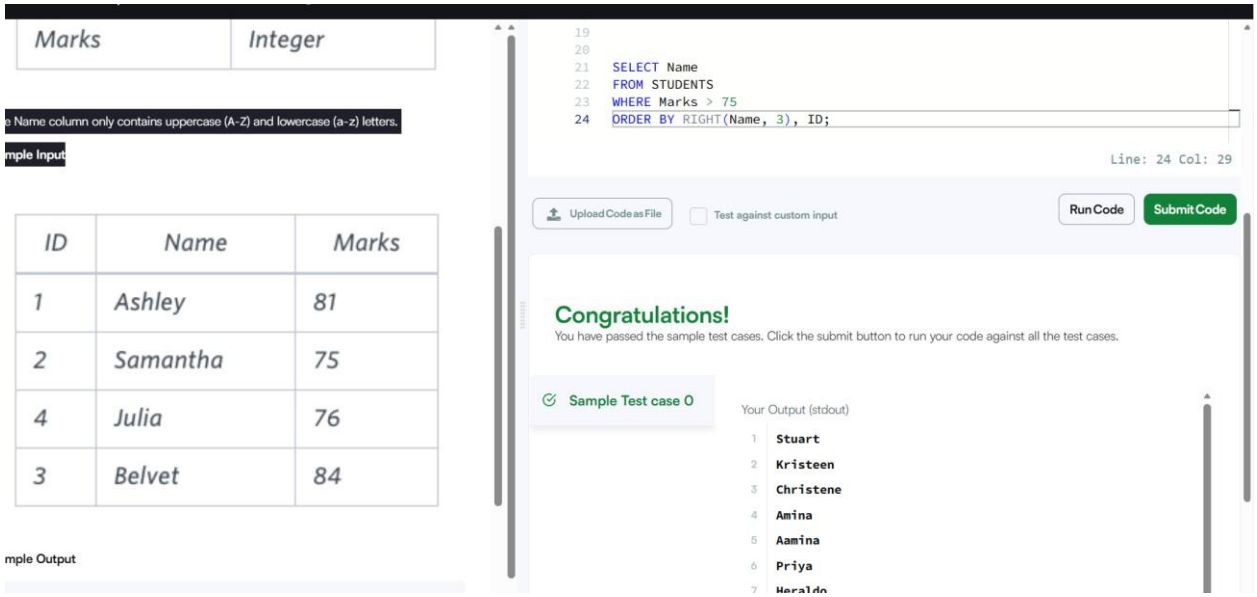
The *Name* column only contains uppercase (A-Z) and lowercase (a-z) letters.

Sample Input ANSWER:

SELECT CITY

FROM STATION;

OUTPUT



QUESTION:

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

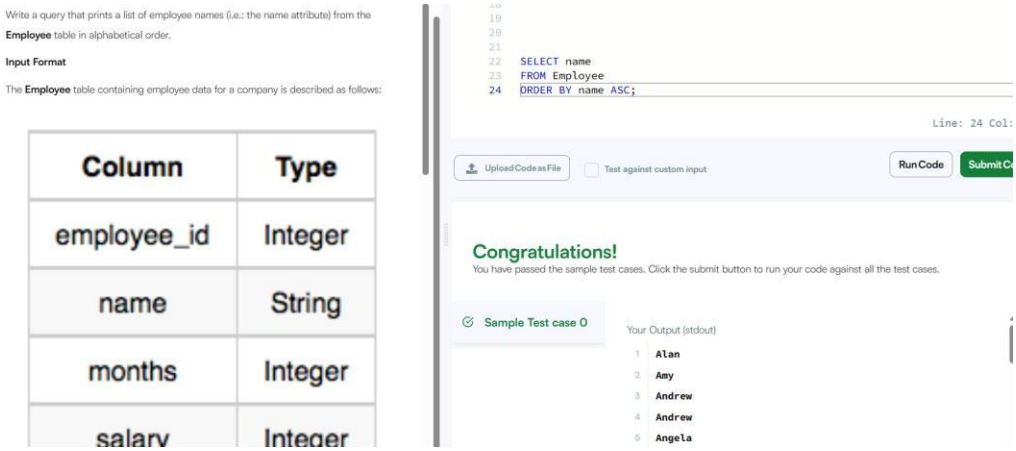
ANSWERS:

SELECT name

FROM Employee

ORDER BY name ASC;

OUTPUT



QUESTIONS

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than $2000

per month who have been employees for less than The **Employee** table containing employee data for a company is described as follows:

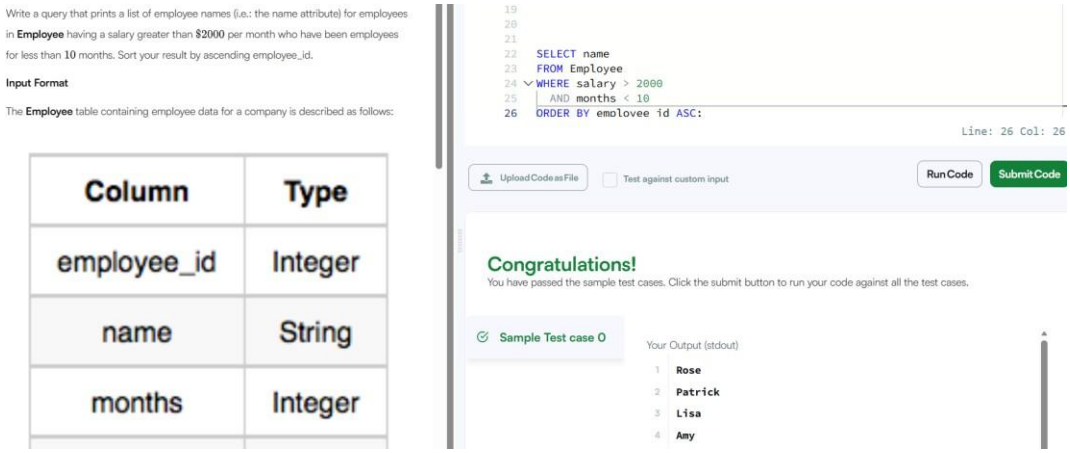
Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

ANSWERS

```
SELECT name
FROM Employee
WHERE salary > 2000
  AND months < 10
ORDER BY employee_id ASC;
```

OUTPUT



QUESTIONS

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

Input Format

The **TRIANGLES** table is described as follows:

Column	*Type*
A	*Integer*
B	*Integer*
C	*Integer*

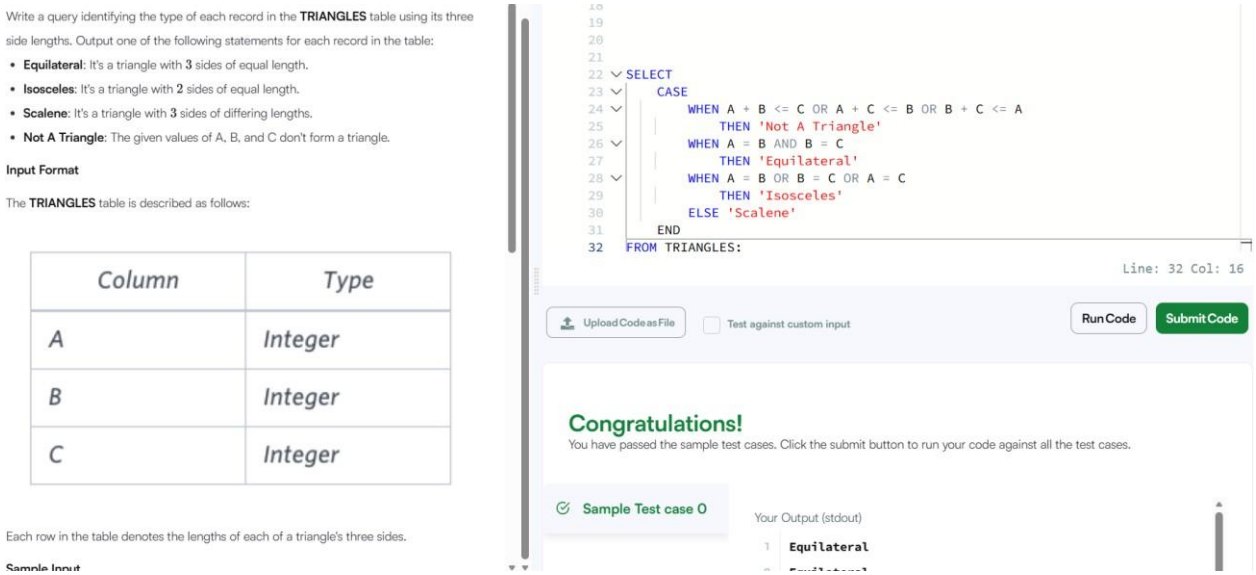
Each row in the table denotes the lengths of each of a triangle's three sides.

ANSWERS SELECT

```
  CASE
    WHEN A + B <= C OR A + C <= B OR B + C <= A
      THEN 'Not A Triangle'
    WHEN A = B AND B = C
      THEN 'Equilateral'
    WHEN A = B OR B = C OR A = C
      THEN 'Isosceles'
```

```
    ELSE 'Scalene'
  END
FROM TRIANGLES;
```

OUTPUT



Write a query identifying the type of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with **3** sides of equal length.
- **Isosceles**: It's a triangle with **2** sides of equal length.
- **Scalene**: It's a triangle with **3** sides of differing lengths.
- **Not A Triangle**: The given values of A, B, and C don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Sample Input

```
SELECT
    CASE
        WHEN A + B <= C OR A + C <= B OR B + C <= A
            THEN 'Not A Triangle'
        WHEN A = B AND B = C
            THEN 'Equilateral'
        WHEN A = B OR B = C OR A = C
            THEN 'Isosceles'
        ELSE 'Scalene'
    END
FROM TRIANGLES;
```

Line: 32 Col: 16

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Equilateral
```

QUESTIONS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).

2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:

3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

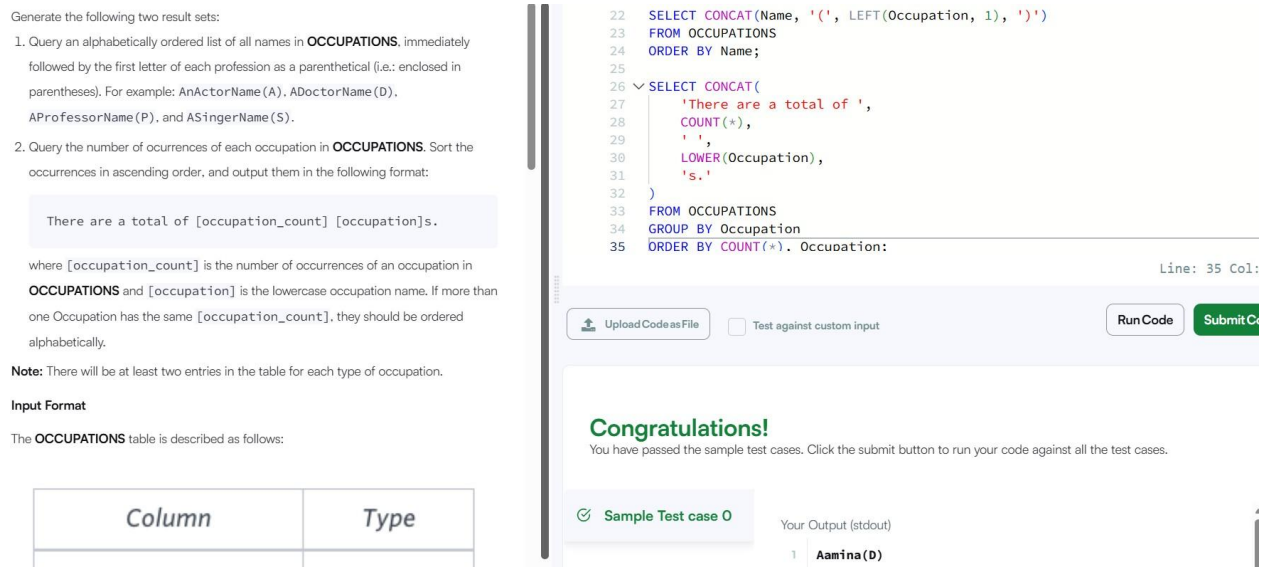
Note: There will be at least two entries in the table for each type of occupation.

ANSWER

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;

SELECT CONCAT(
    'There are a total of ',
    COUNT(*),
    ' ',
    LOWER(Occupation),
    's.'
)
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(*), Occupation;
```

OUTPUT



QUESTIONS

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four

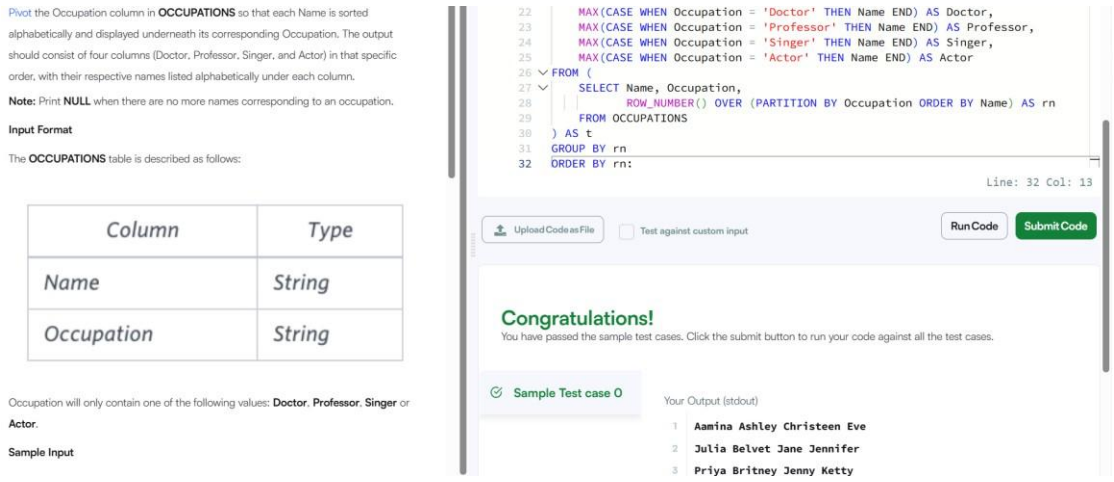
columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

ANSWERS

```
SELECT
   MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
   MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
   MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
   MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
   SELECT Name, Occupation,
      ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
   FROM OCCUPATIONS
) AS t
GROUP BY rn
ORDER BY rn;
```

OUTPUT



QUESTIONS

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

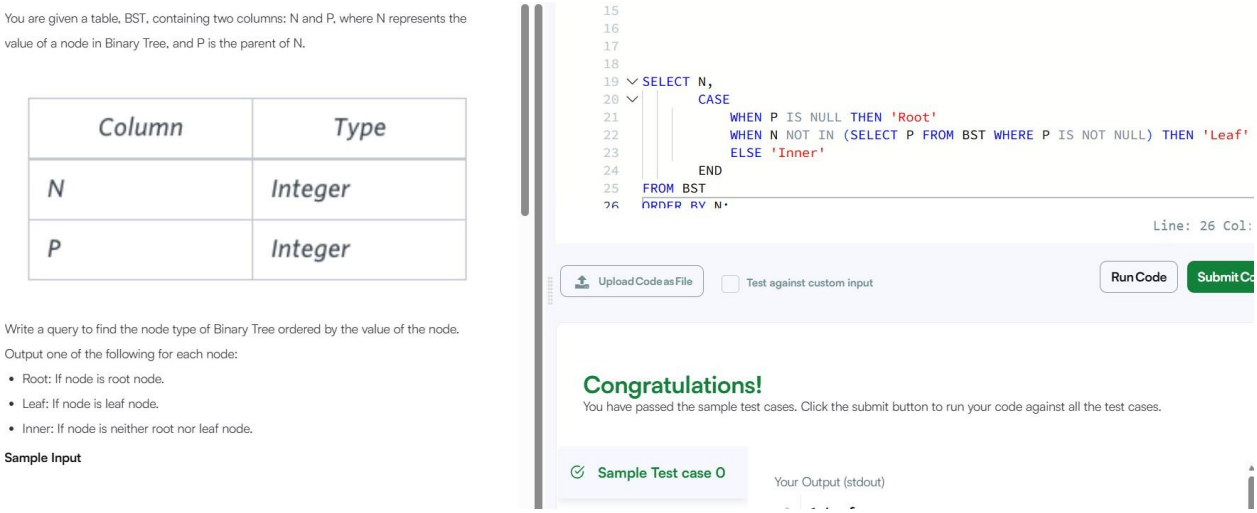
Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

ANSWER

```
SELECT N,
    CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
      ELSE 'Inner'
    END
FROM BST
ORDER BY N;
```

OUTPUT



You are given a table, BST, containing two columns: N and P, where N represents the value of a node in Binary Tree, and P is the parent of N.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of Binary Tree ordered by the value of the node. Output one of the following for each node:

- Root: If node is root node.
- Leaf: If node is leaf node.
- Inner: If node is neither root nor leaf node.

Sample Input

```
15
16
17
18
19 SELECT N,
20     CASE
21         WHEN P IS NULL THEN 'Root'
22         WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
23         ELSE 'Inner'
24     END
25 FROM BST
26 ORDER BY N;
```

Line: 26 Col:

Upload Code as File | Test against custom input | Run Code | Submit Co

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0 | Your Output (stdout)

QUESTIONS

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:

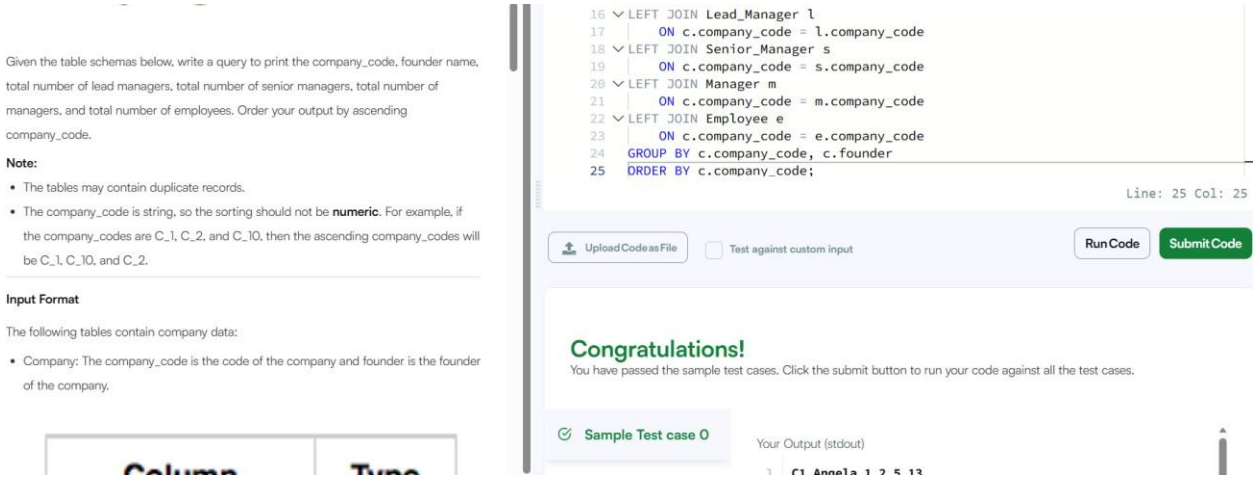
Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

ANSWER

```
SELECT c.company_code,
    c.founder,
    COUNT(DISTINCT l.lead_manager_code),
    COUNT(DISTINCT s.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager l
   ON c.company_code = l.company_code
LEFT JOIN Senior_Manager s
   ON c.company_code = s.company_code
LEFT JOIN Manager m
   ON c.company_code = m.company_code
```

```
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code;
```

OUTPUT



QUESTIONS

Query a *count* of the number of cities in **CITY** having a *Population* larger than 10,000

Input Format

The **CITY** table is described as follows:

CITY

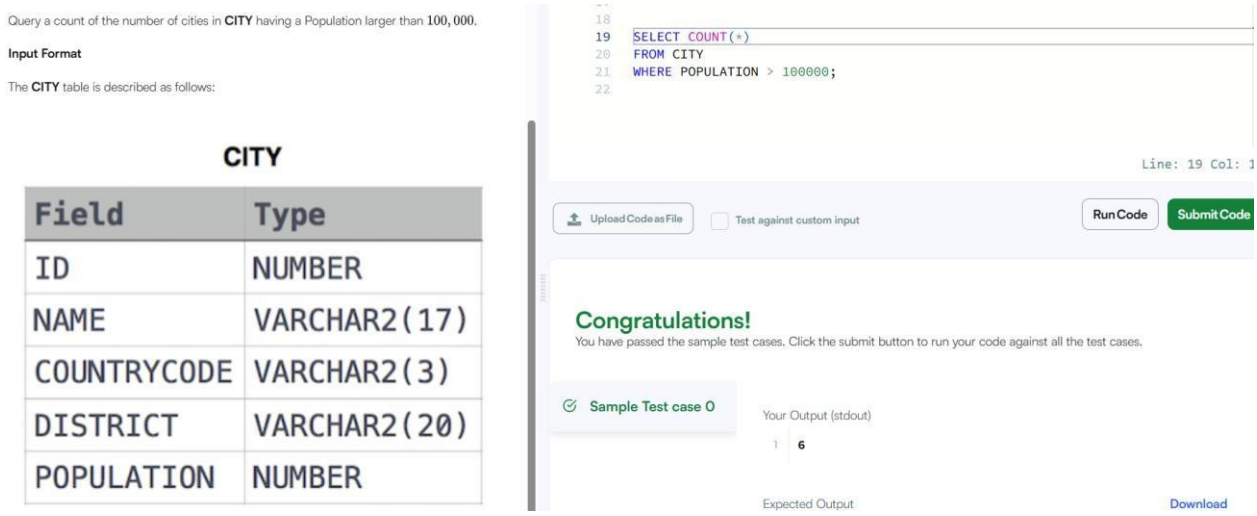
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER

```
SELECT COUNT(*)
FROM CITY
```

WHERE POPULATION > 100000;

OUTPUT



QUESTIONS

Query the total population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

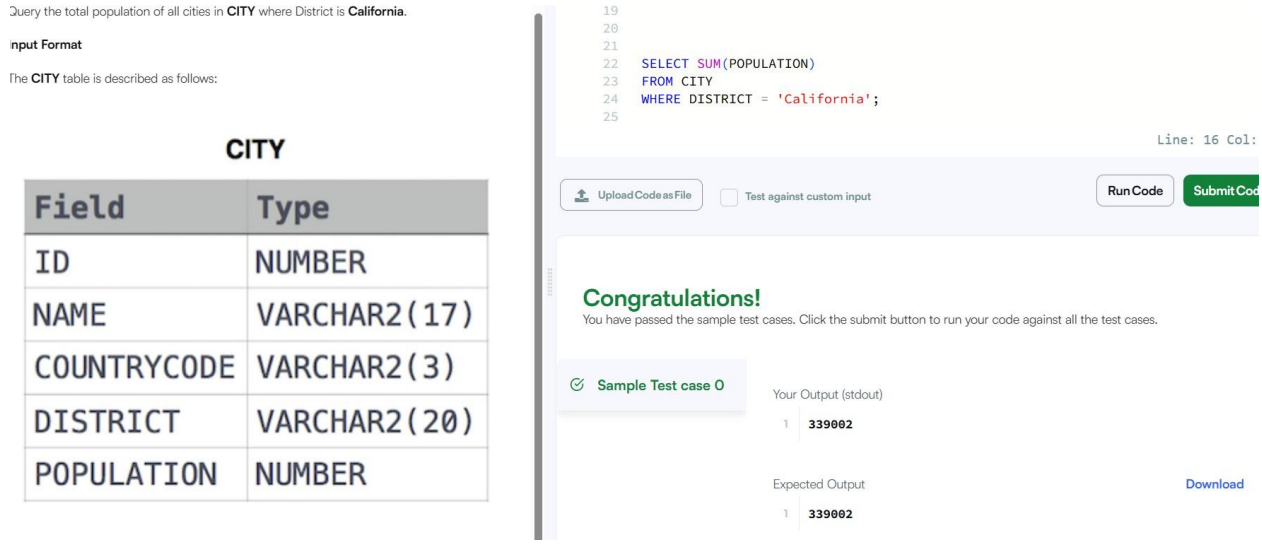
ANSWER

SELECT SUM(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';

OUTPUT



QUESTION

Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

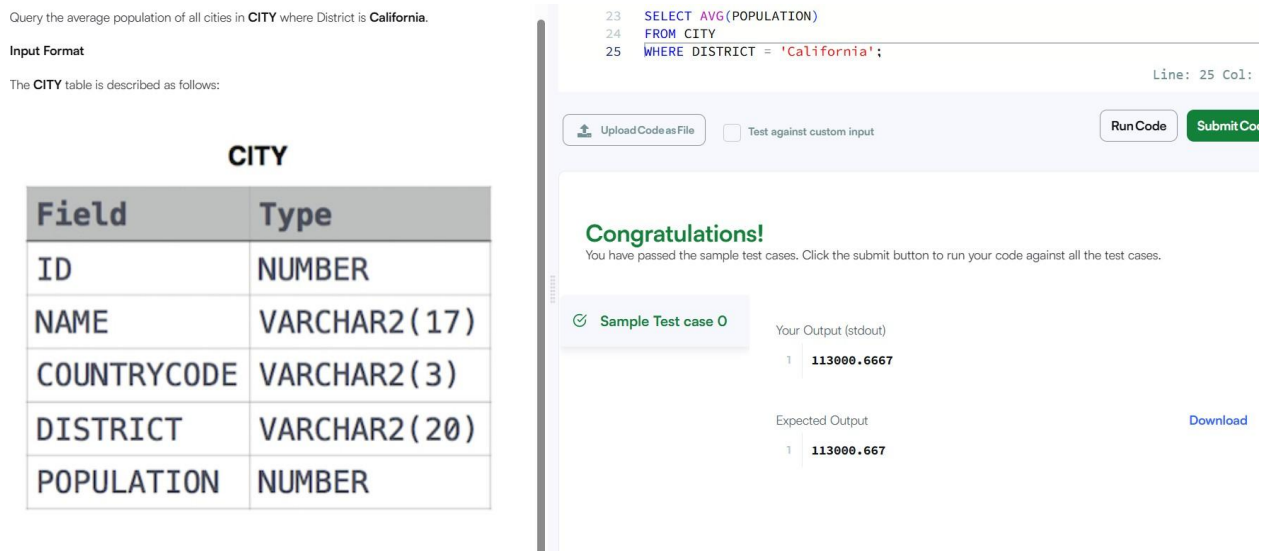
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



QUESTION

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

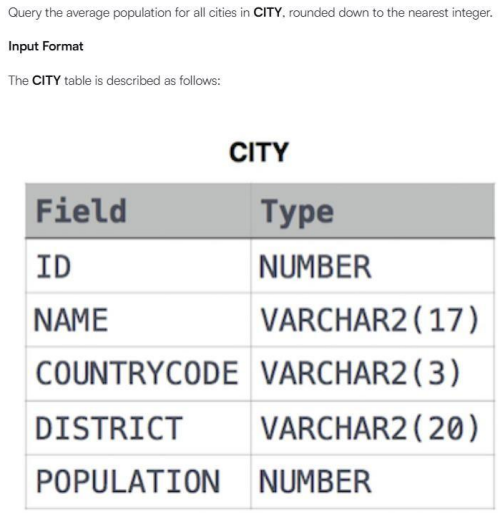
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT

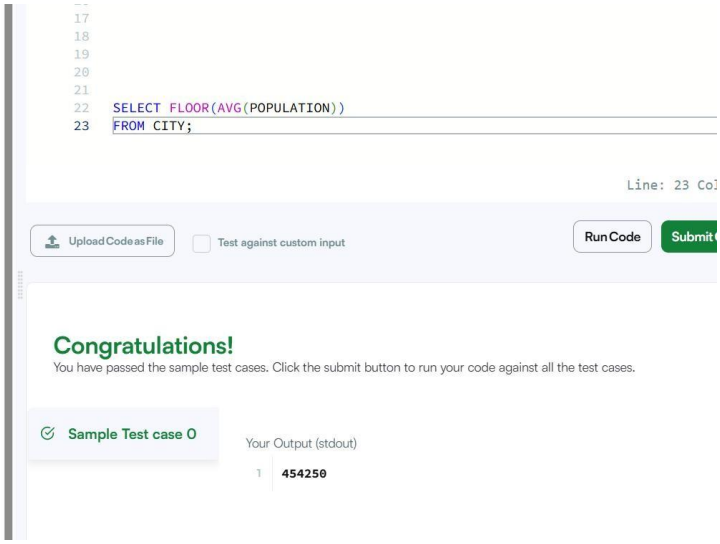

Query the average population for all cities in **CITY**, rounded down to the nearest integer.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



QUESTIONS

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows:

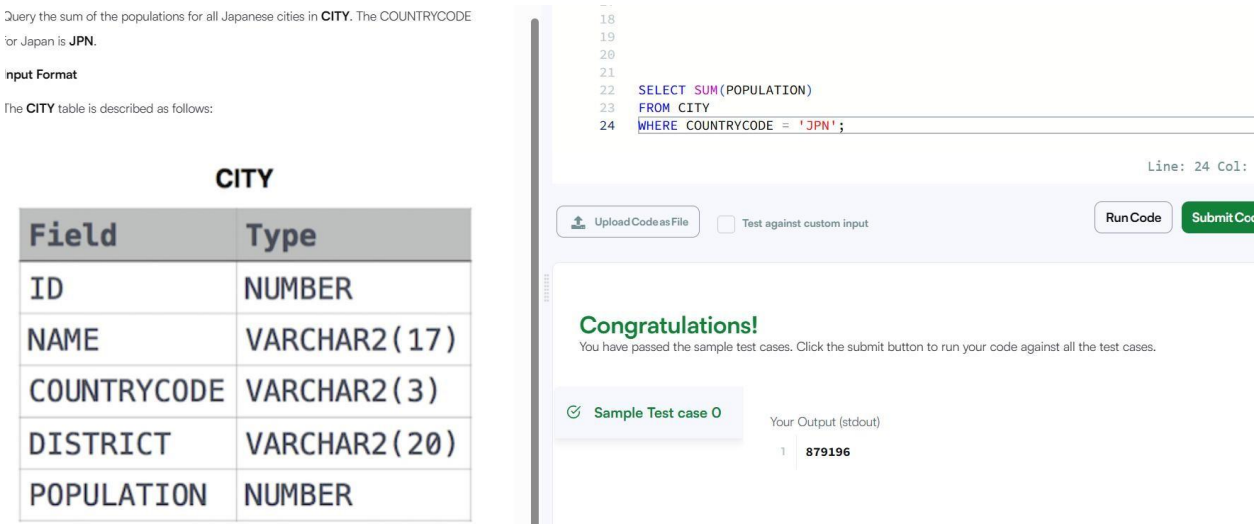
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



QUESTIONS

Query the difference between the maximum and minimum populations in **CITY**.

Input Format

The **CITY** table is described as follows:

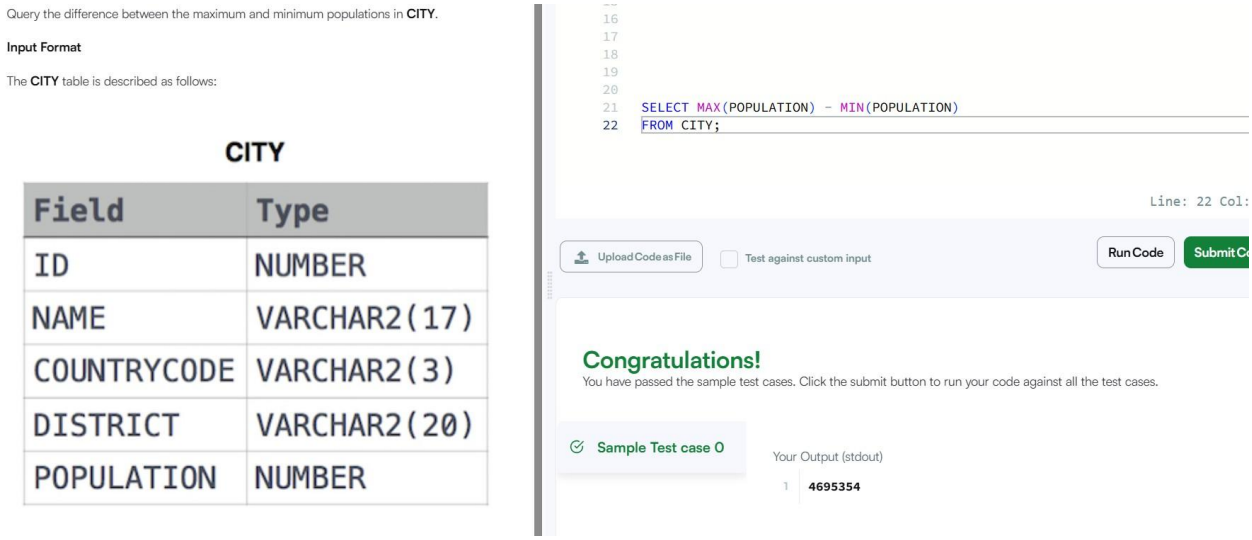
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWERS

```
SELECT MAX(POPULATION) - MIN(POPULATION)
FROM CITY;
```

OUTPUT



Query the difference between the maximum and minimum populations in **CITY**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
16
17
18
19
20
21  SELECT MAX(POPULATION) - MIN(POPULATION)
22  FROM CITY;
```

Line: 22 Col:

Upload Code as File | Test against custom input | Run Code | Submit C

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  4695354
```

QUESTIONS

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's 0 key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error

Input Format

The **EMPLOYEES** table is described as follows:

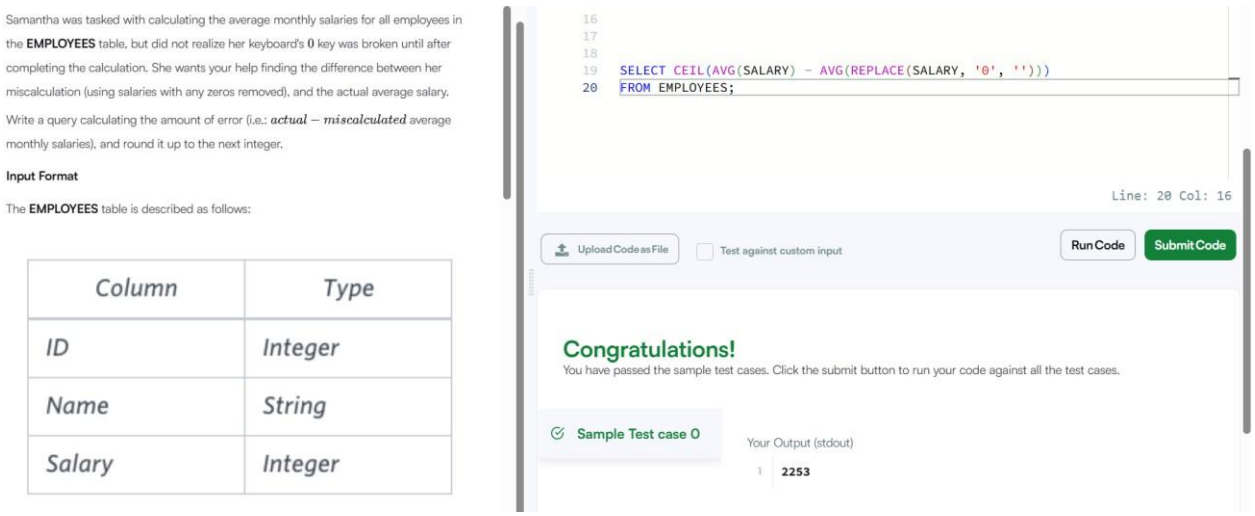
Column	*Type*
ID	*Integer*
Name	*String*
Salary	*Integer*

ANSWER

SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', '')))

FROM EMPLOYEES;

OUTPUT



Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's **0** key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: $actual - miscalculated$ average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

```
16
17
18
19  SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', '')))
20  FROM EMPLOYEES;
```

Line: 20 Col: 16

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  2253
```

QUESTIONS

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as 2 space-separated integers.

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

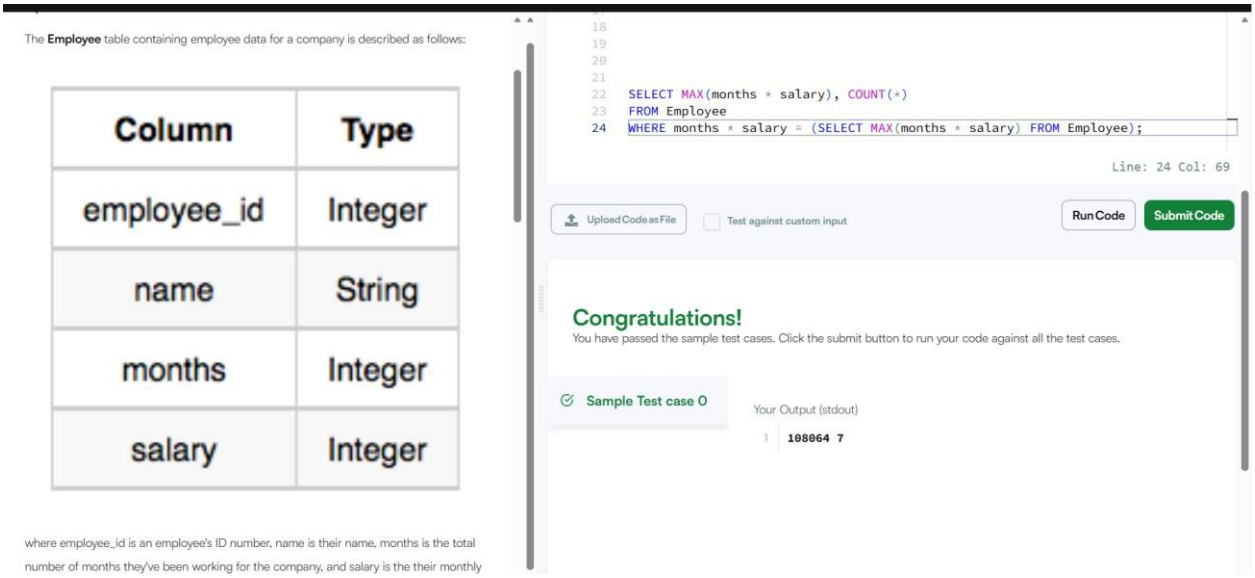
ANSWERS

SELECT MAX(months * salary), COUNT(*)

FROM Employee

WHERE months * salary = (SELECT MAX(months * salary) FROM Employee);

OUTPUT



QUESTION

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of 2 decimal places
2. The sum of all values in *LONG_W* rounded to a scale of 2 decimal places

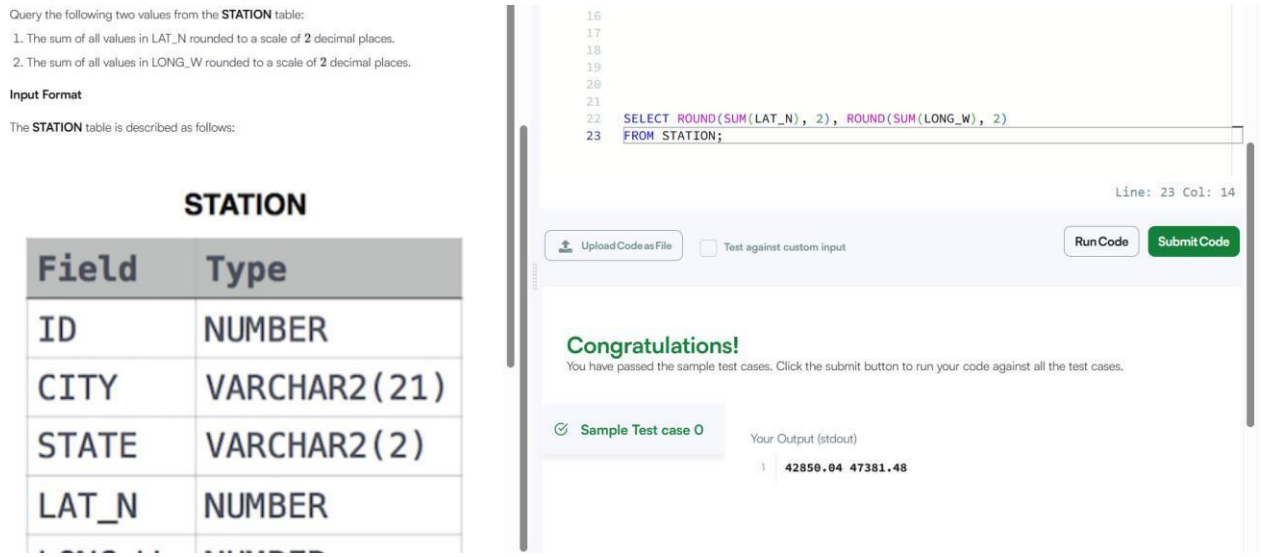
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWER

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT



QUESTION

Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than 38.7880 and less than 137.2345

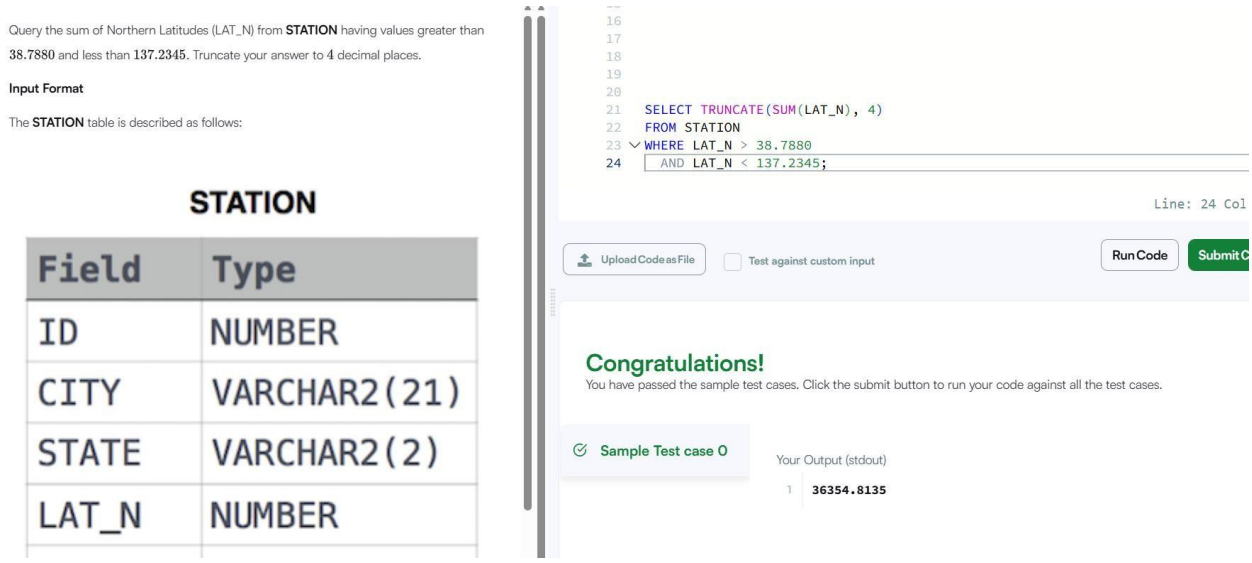
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWER

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT



QUESTIONS

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than 137.2345

Truncate your answer to 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWER

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT



QUESTION

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than 137.2345

Round your answer to 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

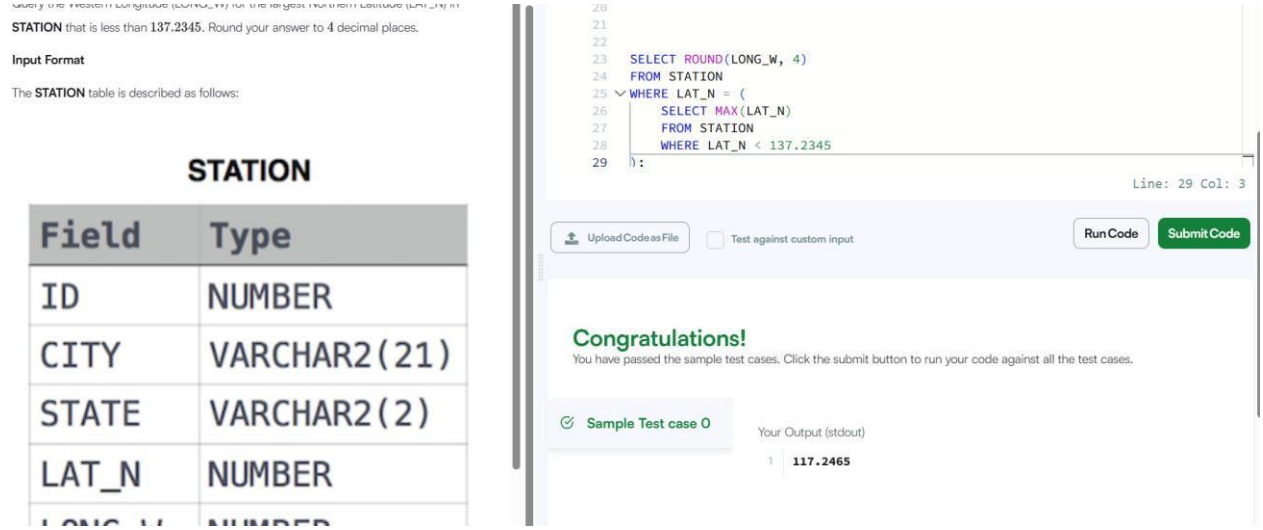
ANSWERS

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
```

WHERE LAT_N < 137.2345

);

OUTPUT



QUESTIONS

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than 137.2345

Round your answer to 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

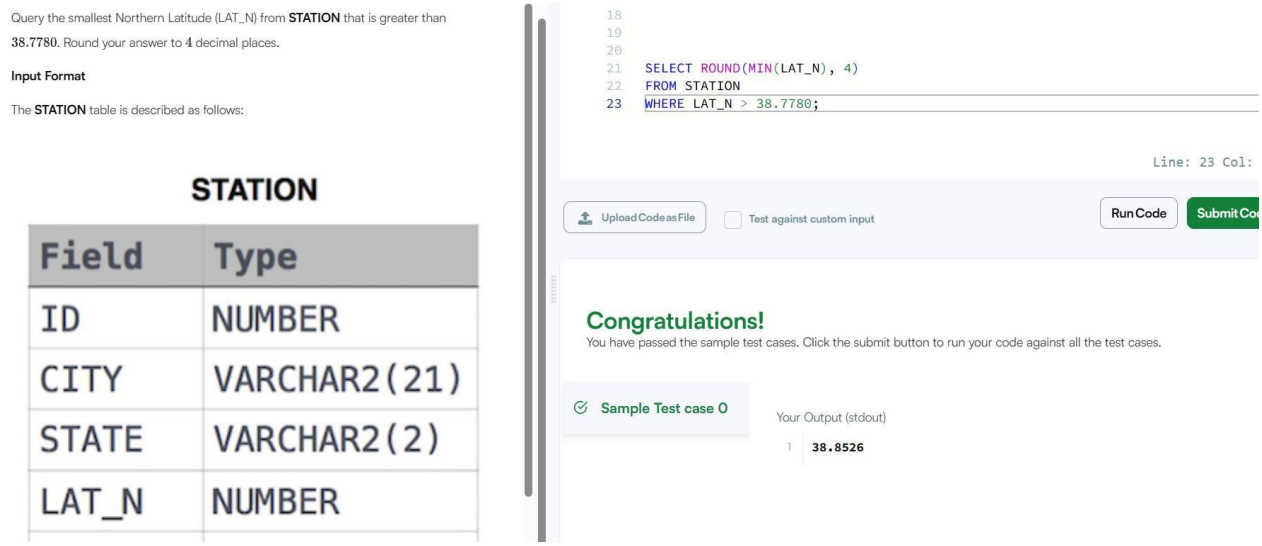
ANSWERS

SELECT ROUND(MIN(LAT_N), 4)

FROM STATION

WHERE LAT_N > 38.7780;

OUTPUT



QUESTIONS

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than 38.7880

Round your answer to 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

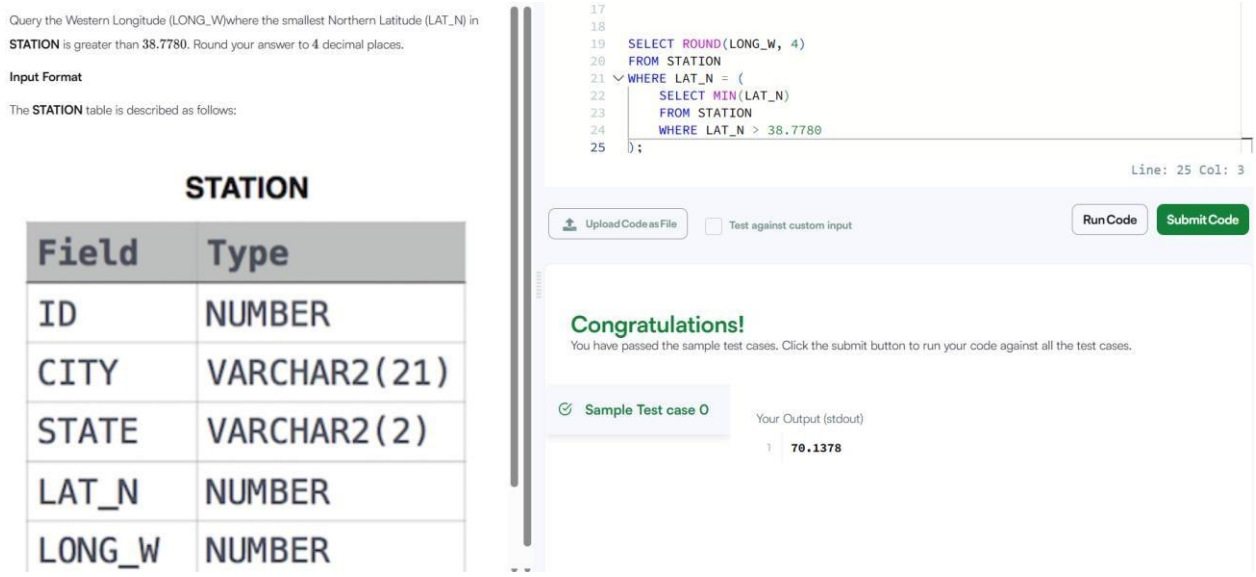
ANSWER

SELECT ROUND(LONG_W, 4)

FROM STATION

```
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT



QUESTIONS

Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane.

- *a* happens to equal the minimum value in Northern Latitude (LAT_N in STATION).
- *b* happens to equal the minimum value in Western Longitude (LONG_W in STATION).
- *c* happens to equal the maximum value in Northern Latitude (LAT_N in STATION).
- *d* happens to equal the maximum value in Western Longitude (LONG_W in STATION).

Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.

Input Format

The **STATION** table is described as follows:

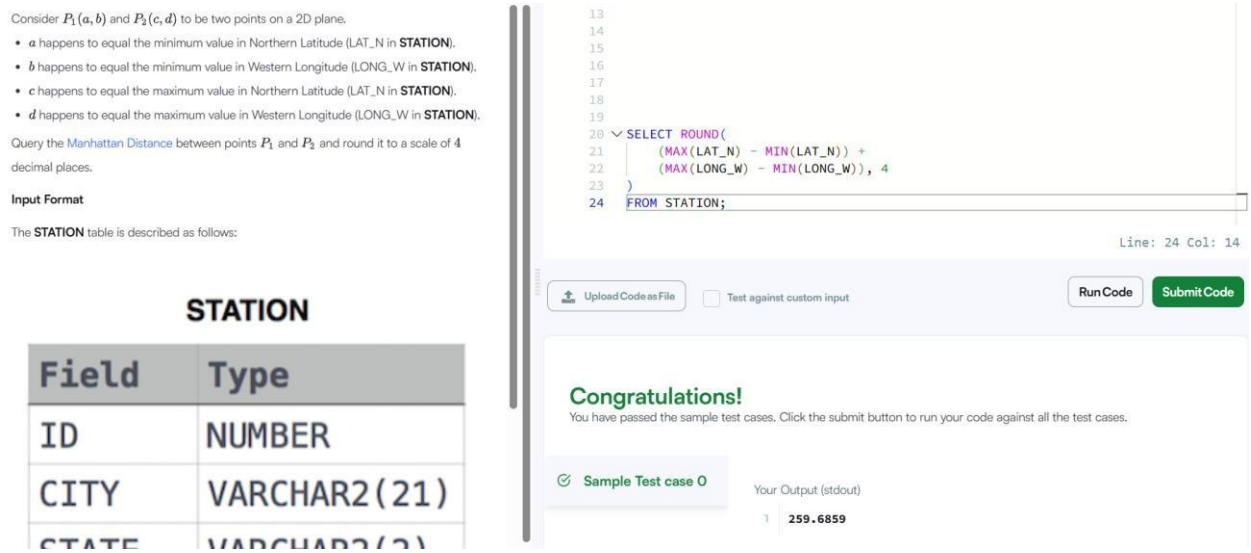
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

```
SELECT ROUND(
    (MAX(LAT_N) - MIN(LAT_N)) +
    (MAX(LONG_W) - MIN(LONG_W)), 4
)
FROM STATION;
```

OUTPUT



QUESTIONS

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to 4 decimal places.

Input Format

The **STATION** table is described as follows:

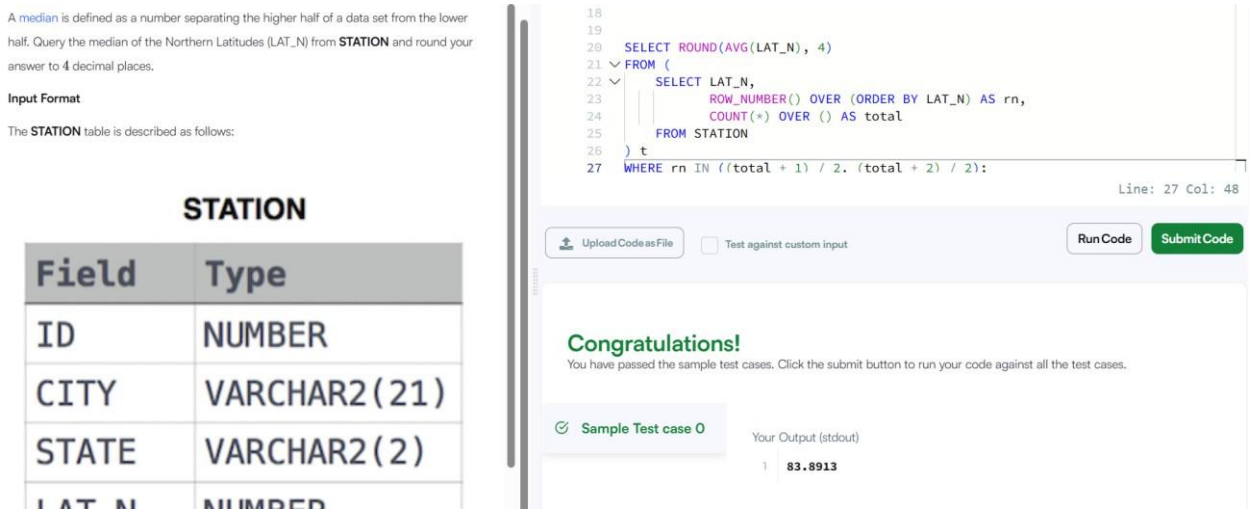
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWERS

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
   SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn,
        COUNT(*) OVER () AS total
   FROM STATION
) t
WHERE rn IN ((total + 1) / 2, (total + 2) / 2);
```

OUTPUT



QUESTIONS

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

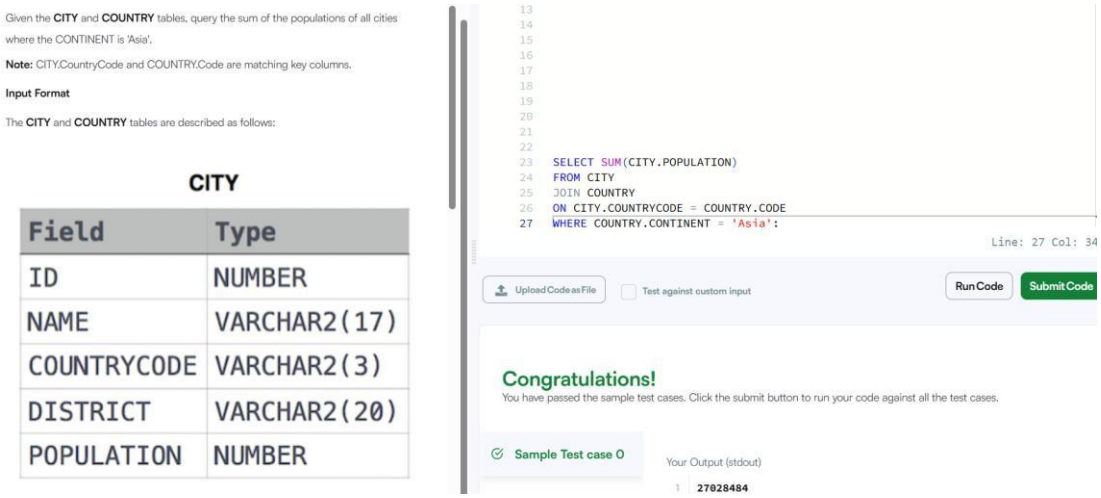
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)

ANSWER

```
SELECT SUM(CITY.POPULATION)
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

OUTPUT



QUESTIONS

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

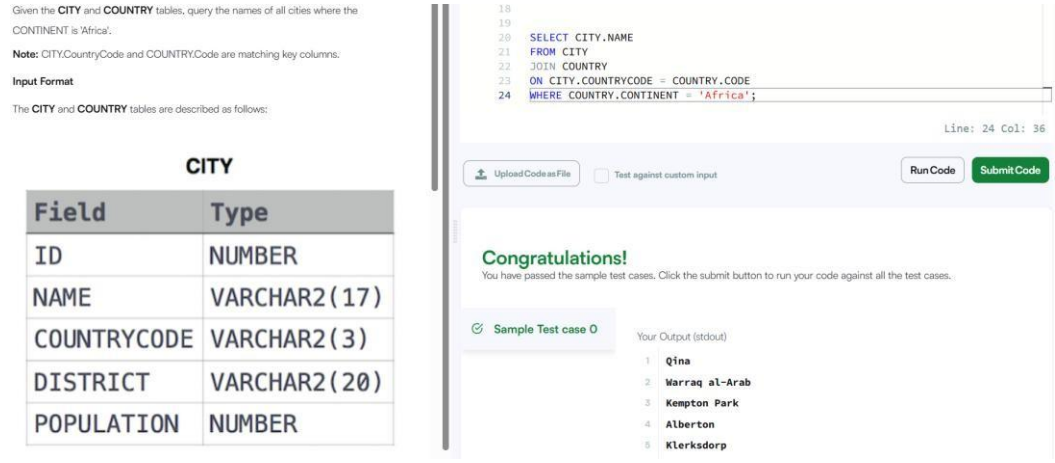
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)

ANSWER

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Africa';
```

OUTPUT



Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
18
19
20  SELECT CITY.NAME
21  FROM CITY
22  JOIN COUNTRY
23  ON CITY.COUNTRYCODE = COUNTRY.CODE
24  WHERE COUNTRY.CONTINENT = 'Africa';
```

Line: 24 Col: 36

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Qina
Warraq al-Arab
Kempton Park
Alberton
Klerksdorp
```

QUESTIONS

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

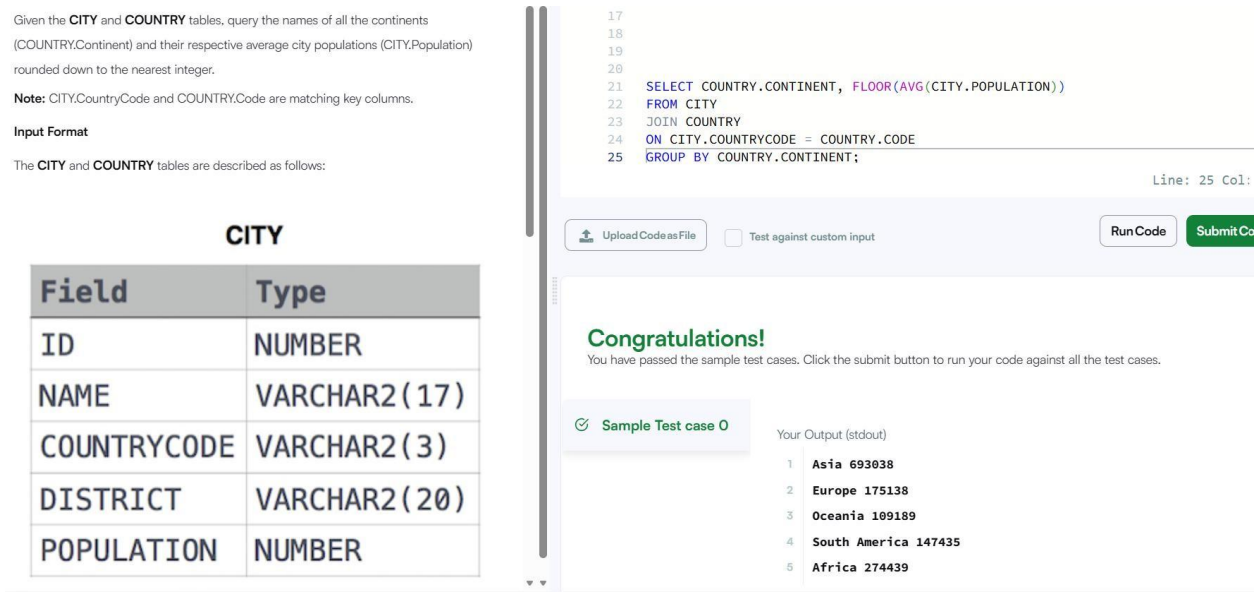
Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)

ANSWER

SELECT COUNTRY.CONTINENT, FLOOR(AVG(CITY.POPULATION))

```
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
GROUP BY COUNTRY.CONTINENT;
```

OUTPUT



Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
17
18
19
20
21  SELECT COUNTRY.CONTINENT, FLOOR(AVG(CITY.POPULATION))
22  FROM CITY
23  JOIN COUNTRY
24  ON CITY.COUNTRYCODE = COUNTRY.CODE
25  GROUP BY COUNTRY.CONTINENT;
```

Line: 25 Col:

Upload Code as File | Test against custom input | Run Code | Submit Co

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  Asia 693038
2  Europe 175138
3  Oceania 109189
4  South America 147435
5  Africa 274439
```

QUESTIONS

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

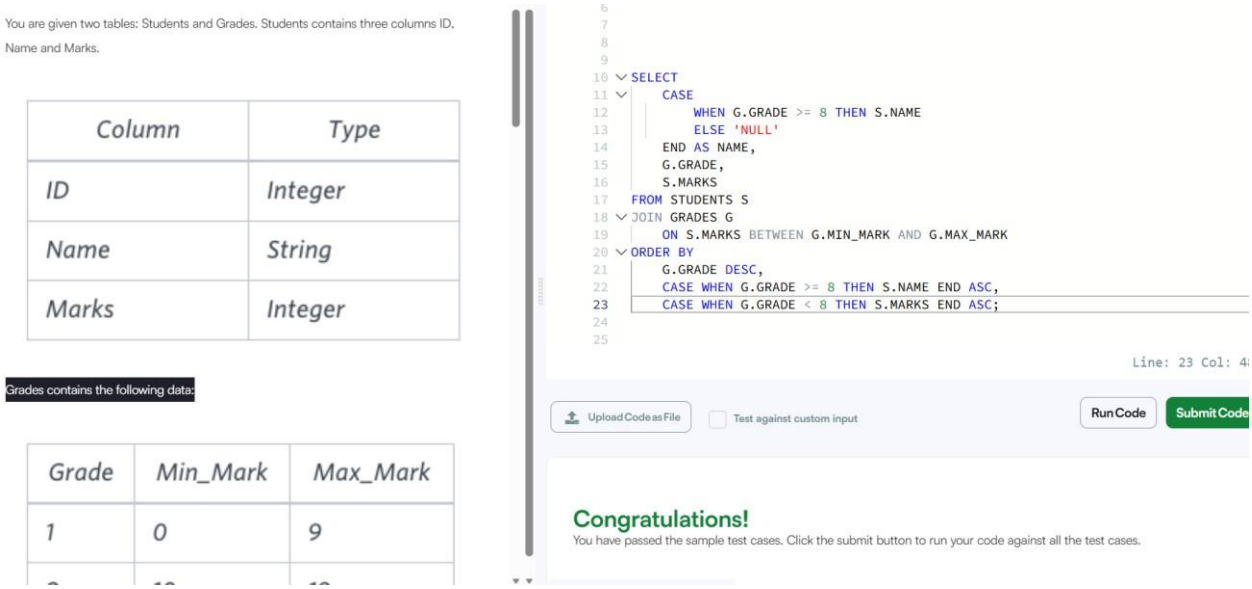
Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89

ANSWER

```
SELECT
   CASE
      WHEN G.GRADE >= 8 THEN S.NAME
      ELSE 'NULL'
   END AS NAME,
   G.GRADE,
   S.MARKS
FROM STUDENTS S
JOIN GRADES G
   ON S.MARKS BETWEEN G.MIN_MARK AND G.MAX_MARK
ORDER BY
   G.GRADE DESC,
   CASE WHEN G.GRADE >= 8 THEN S.NAME END ASC,
   CASE WHEN G.GRADE < 8 THEN S.MARKS END ASC;
```

OUTPUT



QUESTIONS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*. **Input Format**

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

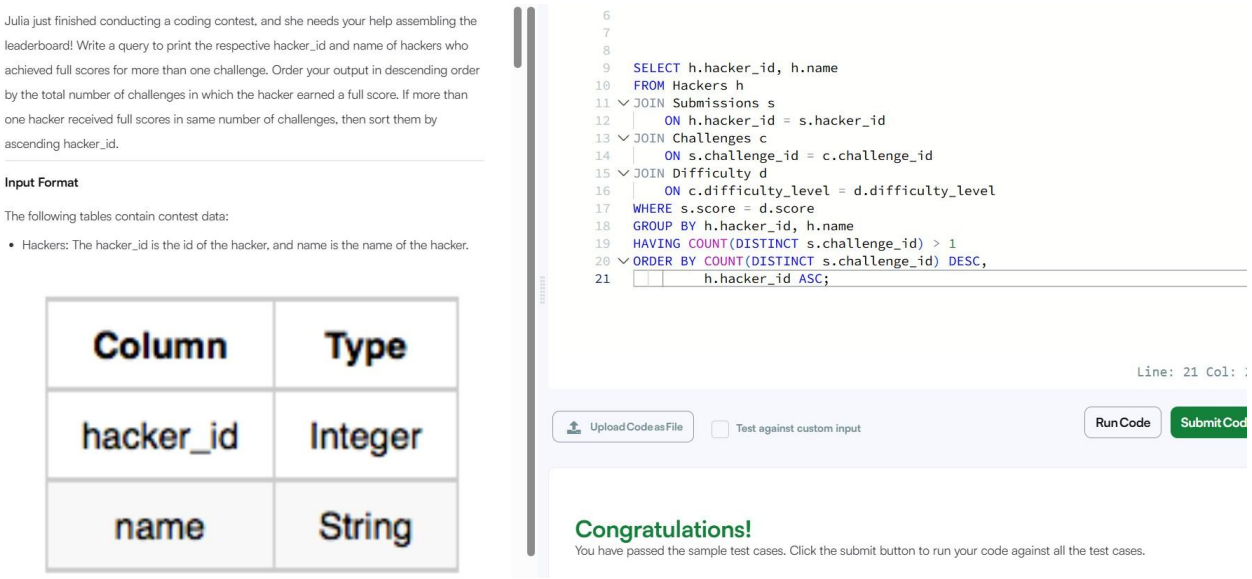
ANSWER

SELECT h.hacker_id, h.name

FROM Hackers h

```
JOIN Submissions s
    ON h.hacker_id = s.hacker_id
JOIN Challenges c
    ON s.challenge_id = c.challenge_id
JOIN Difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1 ORDER
BY COUNT(DISTINCT s.challenge_id) DESC,
        h.hacker_id ASC;
```

OUTPUT



Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.

Input Format

The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name
FROM Hackers h
JOIN Submissions s
    ON h.hacker_id = s.hacker_id
JOIN Challenges c
    ON s.challenge_id = c.challenge_id
JOIN Difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1
ORDER BY COUNT(DISTINCT s.challenge_id) DESC,
        h.hacker_id ASC;
```

Line: 21 Col:

Upload Code as File | Test against custom input | Run Code | Submit Cod

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

QUESTION

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order

of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

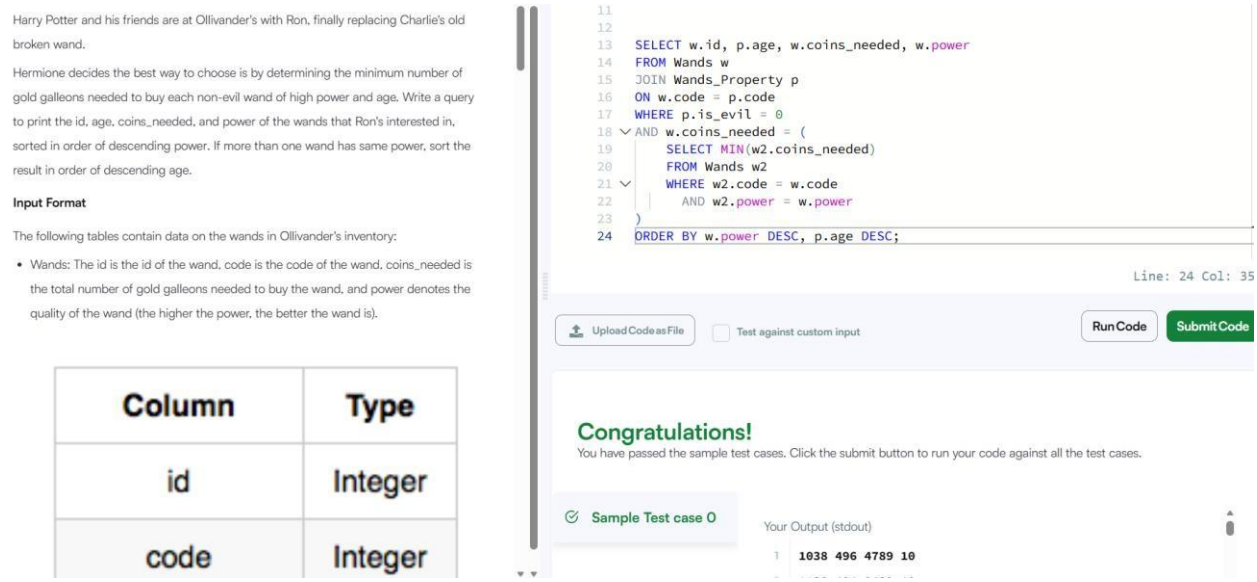
Wands_Property: The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs.

ANSWER

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands w
JOIN Wands_Property p
ON w.code = p.code
WHERE p.is_evil = 0
AND w.coins_needed = (
   SELECT MIN(w2.coins_needed)
   FROM Wands w2
```

```
    WHERE w2.code = w.code
      AND w2.power = w.power
)
ORDER BY w.power DESC, p.age DESC;
```

OUTPUT



Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each non-evil wand of high power and age. Write a query to print the id, age, coins_needed, and power of the wands that Ron's interested in, sorted in order of descending power. If more than one wand has same power, sort the result in order of descending age.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- Wands: The id is the id of the wand, code is the code of the wand, coins_needed is the total number of gold galleons needed to buy the wand, and power denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands w
JOIN Wands_Property p
ON w.code = p.code
WHERE p.is_evil = 0
AND w.coins_needed = (
    SELECT MIN(w2.coins_needed)
    FROM Wands w2
    WHERE w2.code = w.code
      AND w2.power = w.power
)
ORDER BY w.power DESC, p.age DESC;
```

Line: 24 Col: 35

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1038 496 4789 10
```

QUESTION

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

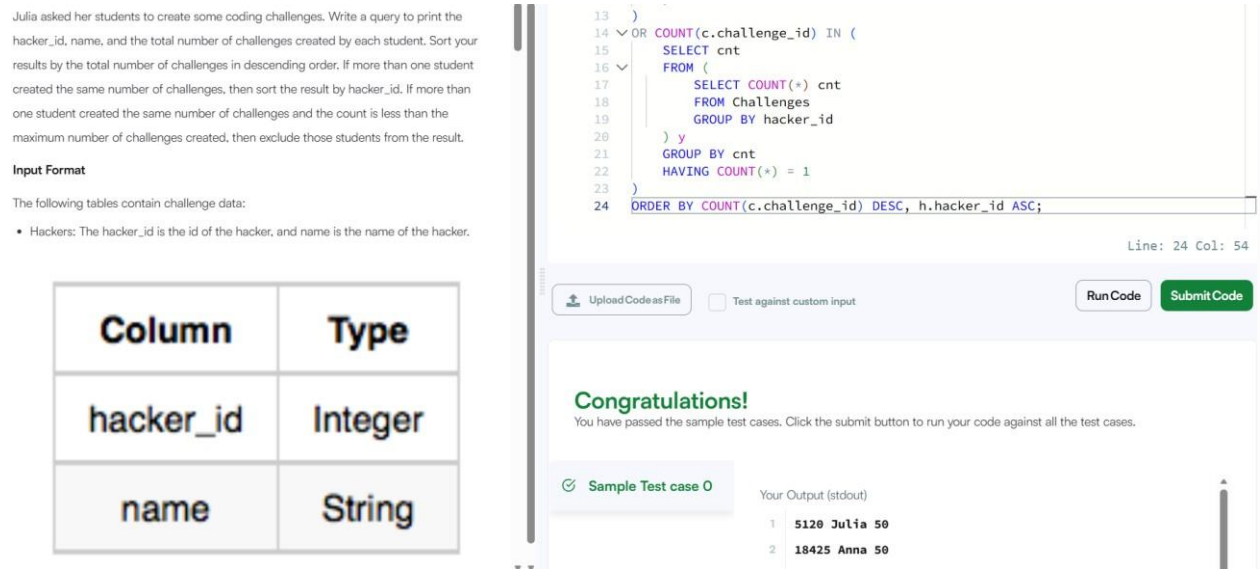
Column	Type
hacker_id	Integer
name	String

ANSWER

```
SELECT h.hacker_id, h.name, COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING COUNT(c.challenge_id) = (
    SELECT MAX(cnt)
    FROM (
        SELECT hacker_id, COUNT(*) cnt
        FROM Challenges
        GROUP BY hacker_id
    ) x
)
OR COUNT(c.challenge_id) IN (
    SELECT cnt
    FROM (
        SELECT COUNT(*) cnt
        FROM Challenges
        GROUP BY hacker_id
    ) y
```

```
    GROUP BY cnt
    HAVING COUNT(*) = 1
)
ORDER BY COUNT(c.challenge_id) DESC, h.hacker_id ASC;
```

OUTPUT



Julia asked her students to create some coding challenges. Write a query to print the hacker_id, name, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by hacker_id. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
)
OR COUNT(c.challenge_id) IN (
    SELECT cnt
    FROM (
        SELECT COUNT(*) cnt
        FROM Challenges
        GROUP BY hacker_id
    ) y
    GROUP BY cnt
    HAVING COUNT(*) = 1
)
ORDER BY COUNT(c.challenge_id) DESC, h.hacker_id ASC;
```

Line: 24 Col: 54

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
5120 Julia 50
18425 Anna 50
```

QUESTIONS

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of

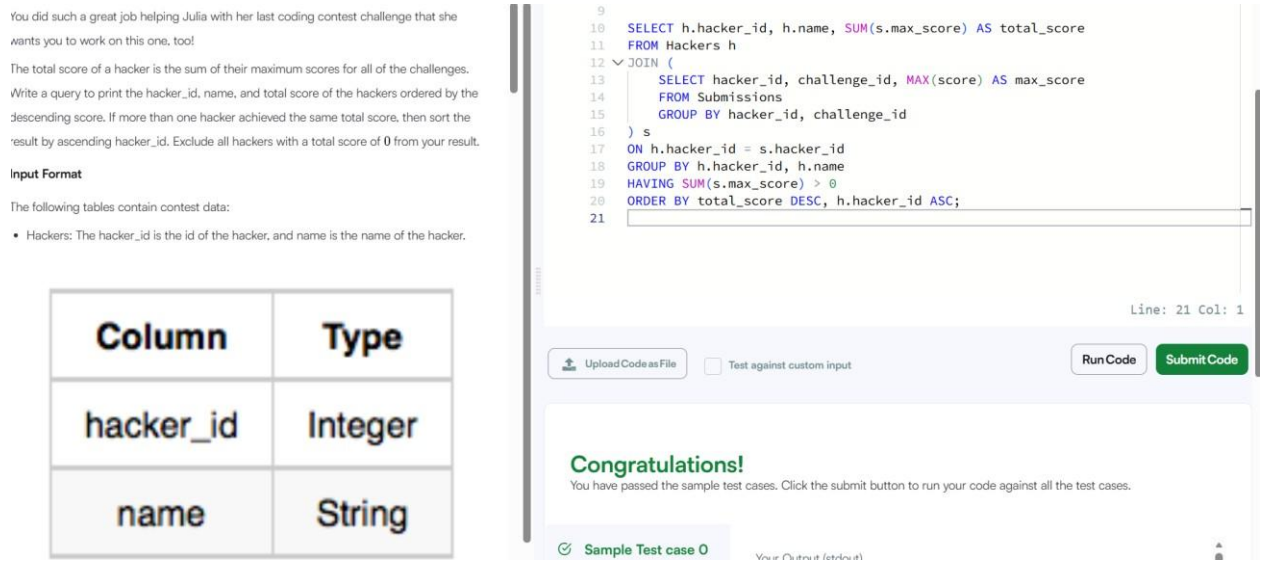
Column	Type
hacker_id	Integer
name	String

Submissions: The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to, and *score* is the score of the submission.

ANSWER

```
SELECT h.hacker_id, h.name, SUM(s.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) s
ON h.hacker_id = s.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(s.max_score) > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```

OUTPUT



QUESTION

You are given a table, *Projects*, containing three columns: *Task_ID, Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

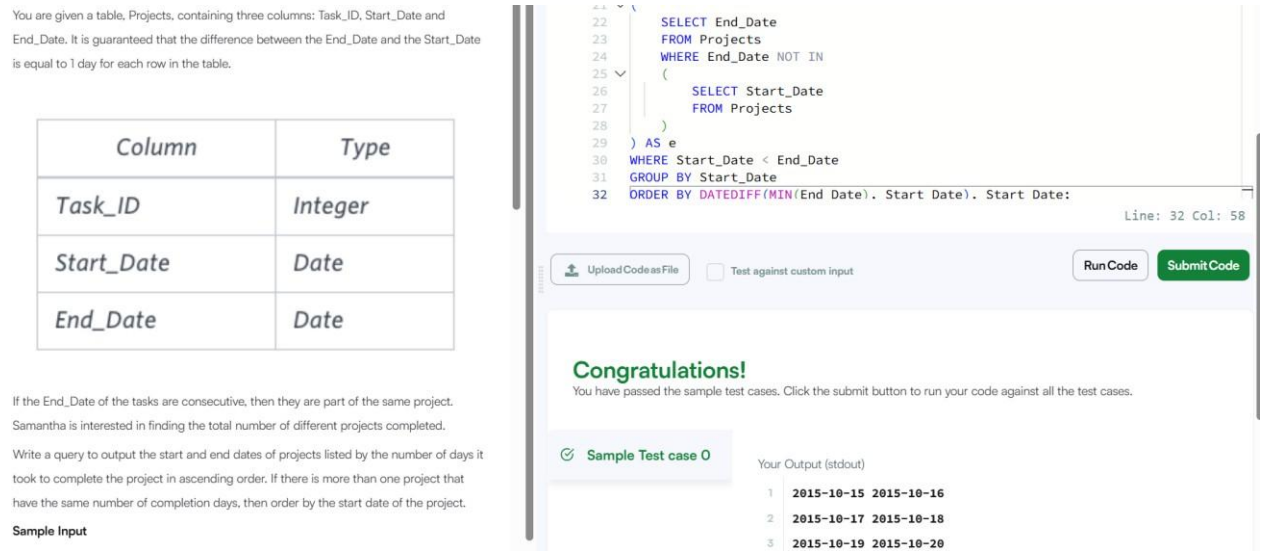
Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

ANSWER

```
SELECT Start_Date, MIN(End_Date)
FROM
(
   SELECT Start_Date
   FROM Projects
   WHERE Start_Date NOT IN
   (
      SELECT End_Date
      FROM Projects
   )
) AS s,
(
   SELECT End_Date
   FROM Projects
   WHERE End_Date NOT IN
   (
      SELECT Start_Date
```

```
        FROM Projects
    )
) AS e
WHERE Start_Date < End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End_Date), Start_Date), Start_Date;
```

OUTPUT



You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

```
    SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN
    (
        SELECT Start_Date
        FROM Projects
    )
) AS e
WHERE Start_Date < End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End Date). Start Date). Start Date:
```

Line: 32 Col: 58

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2015-10-15 2015-10-16
2015-10-17 2015-10-18
2015-10-19 2015-10-20
```

QUESTION

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

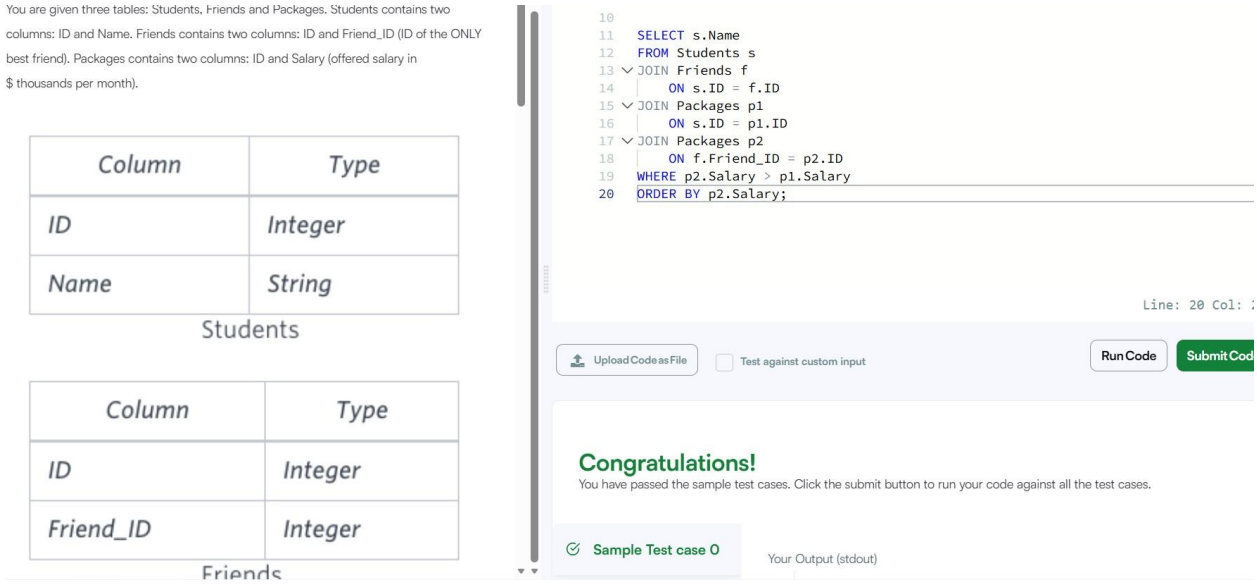
ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

ANSWER

```
SELECT s.Name
```

```
FROM Students s
JOIN Friends f
    ON s.ID = f.ID
JOIN Packages p1
    ON s.ID = p1.ID
JOIN Packages p2
    ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```

OUTPUT



QUESTION

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

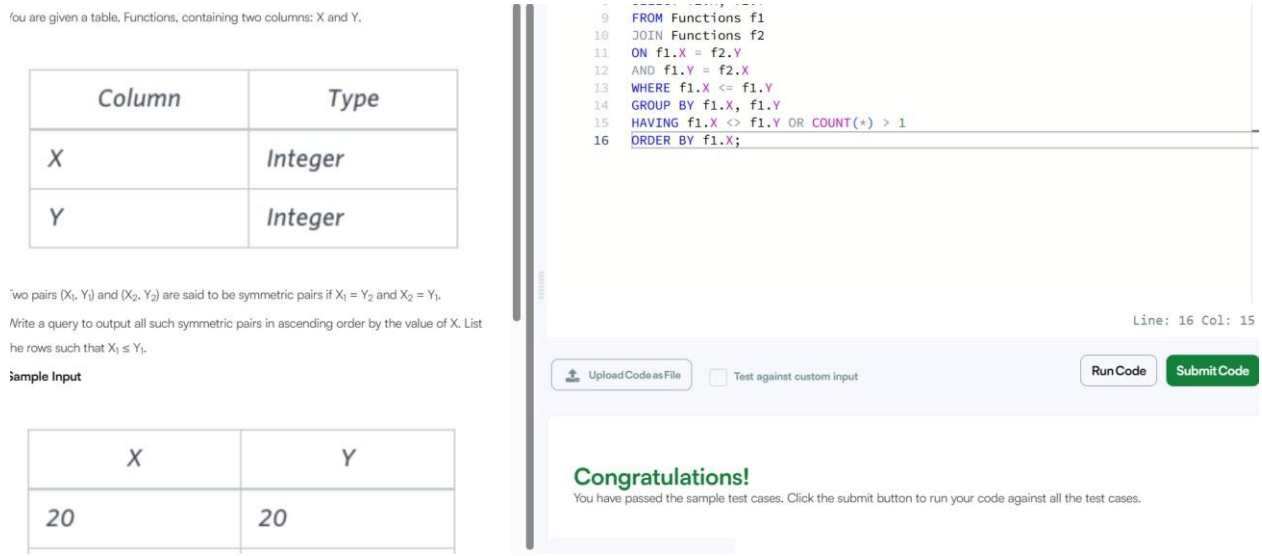
Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

ANSWER

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
ON f1.X = f2.Y
AND f1.Y = f2.X
WHERE f1.X <= f1.Y
GROUP BY f1.X, f1.Y
HAVING f1.X <> f1.Y OR COUNT(*) > 1
ORDER BY f1.X;
```

OUTPUT



QUESTIONS

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*: *

* * * *

* * * *

* * *

* *

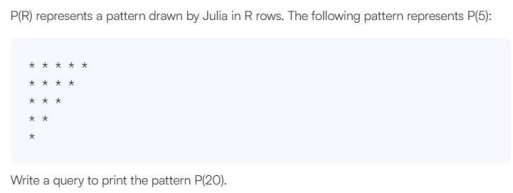
*

Write a query to print the pattern *P(20)*.

ANSWER

```
WITH RECURSIVE numbers AS (
   SELECT 20 AS n
   UNION ALL
   SELECT n - 1
   FROM numbers
   WHERE n > 1
)
SELECT REPEAT('* ', n) AS pattern
FROM numbers;
```

OUTPUT





QUESTION

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

*

* *

* * *

* * * *

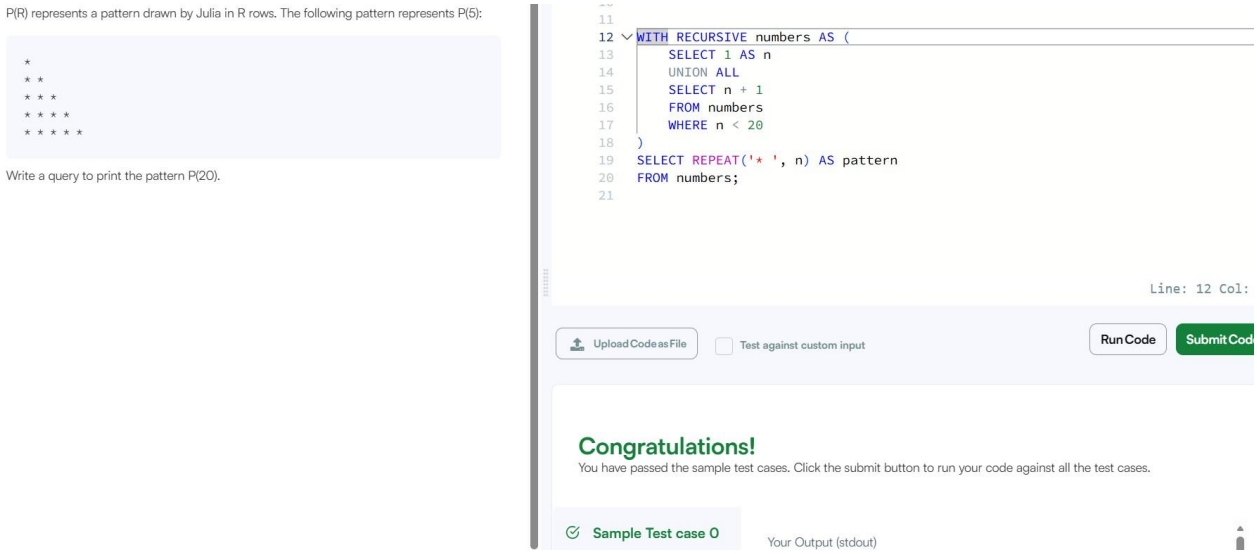
* * * * *

Write a query to print the pattern *P(20)*.

ANSWER

```
WITH RECURSIVE numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1
    FROM numbers
    WHERE n < 20
)
SELECT REPEAT('* ', n) AS pattern
FROM numbers;
```

OUTPUT



QUESTION

Write a query to print all *prime numbers* less than or equal to 1000 Print your result on a single line, and use the ampersand (&) character as your separator (instead of a space).

For example, the output for all prime numbers.

2&3&5&7

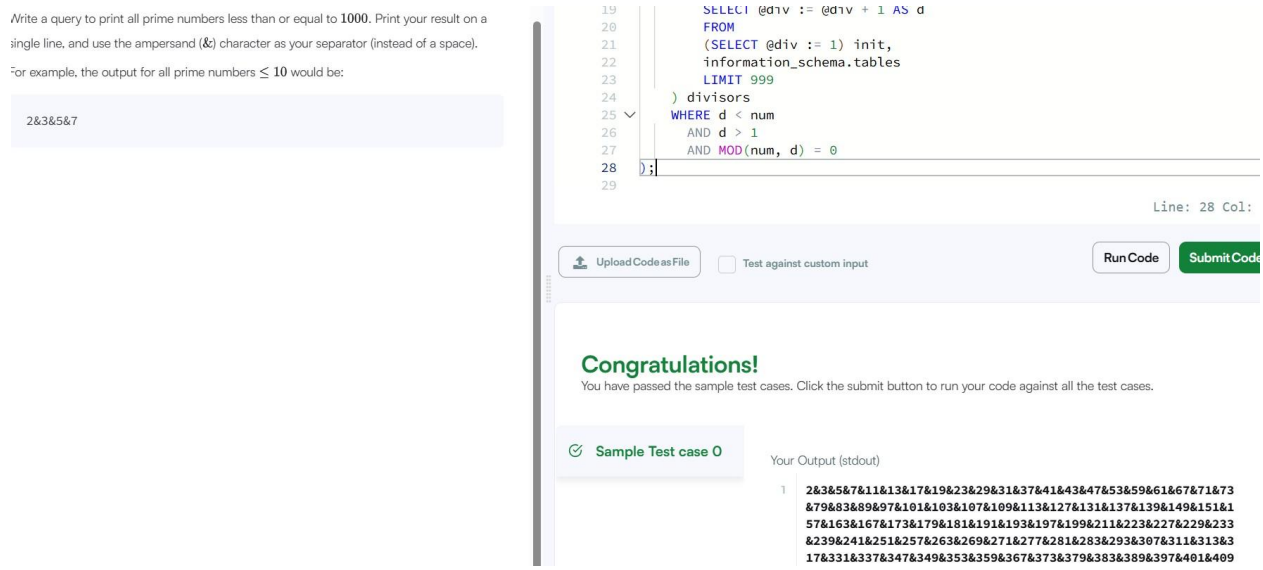
ANSWER

```
SET @num = 1;


SELECT GROUP_CONCAT(num SEPARATOR '&')
FROM
(
   SELECT @num := @num + 1 AS num
FROM information_schema.tables t1,
information_schema.tables t2
   LIMIT 999
) numbers
WHERE NOT EXISTS
(
   SELECT 1
   FROM
   (
      SELECT @div := @div + 1 AS d
      FROM
      (SELECT @div := 1) init,
information_schema.tables
      LIMIT 999
   ) divisors
   WHERE d < num
     AND d > 1
     AND MOD(num, d) = 0
```

);

OUTPUT



CODE CHEF

TASK

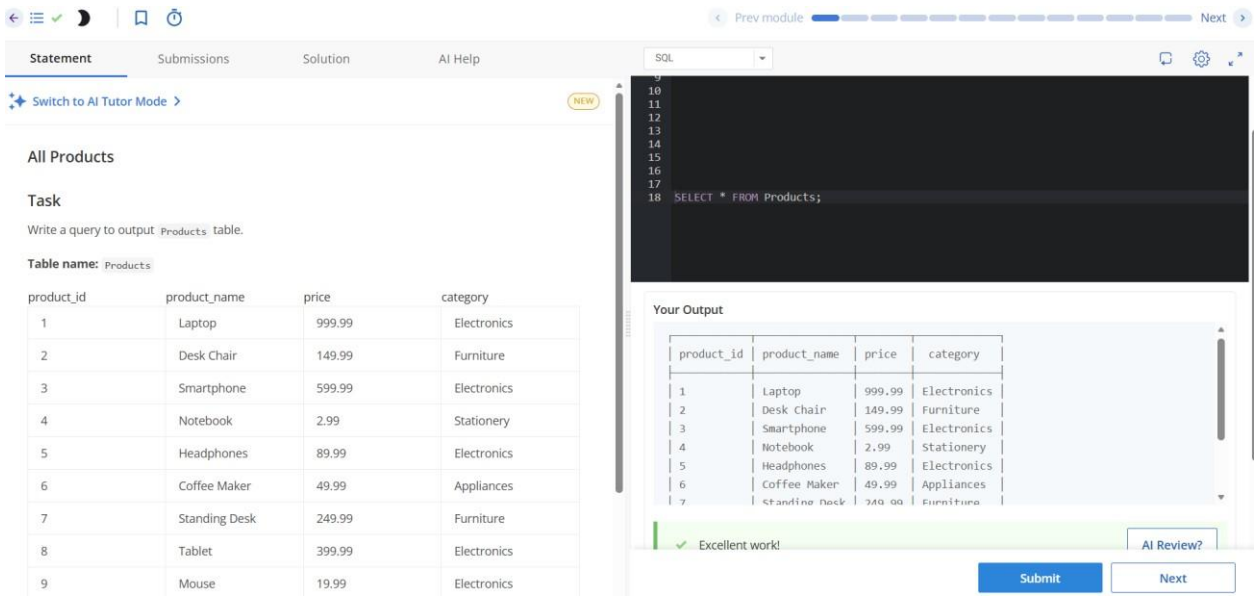
Write a query to output Products table.

Table name: Products

SOLUTION

SELECT * FROM Products;

OUTPUT



TASK

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

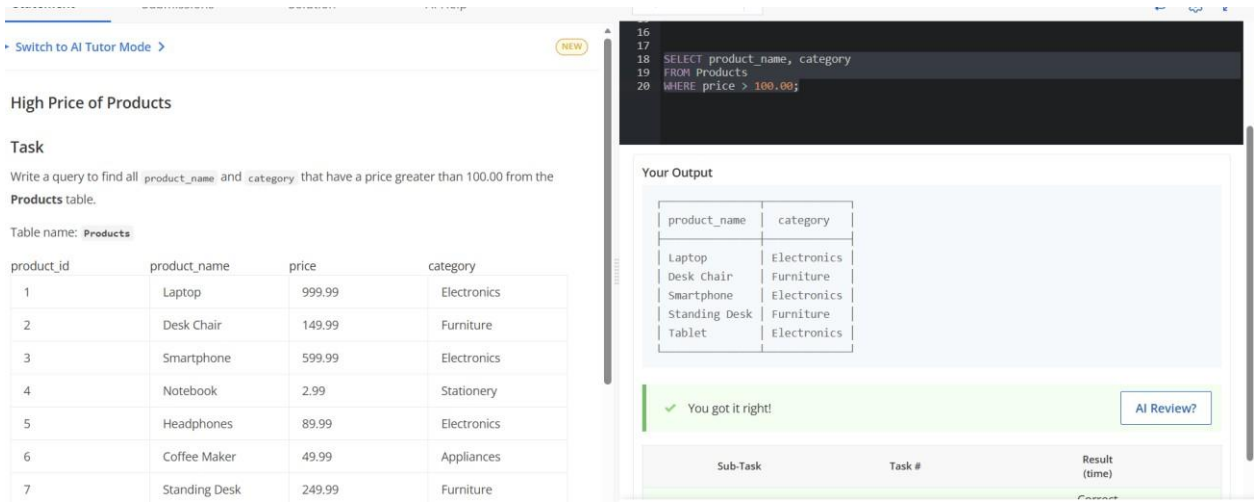
SOLUTION

SELECT product_name, category

FROM Products

WHERE price > 100.00;

OUTPUT



TASK

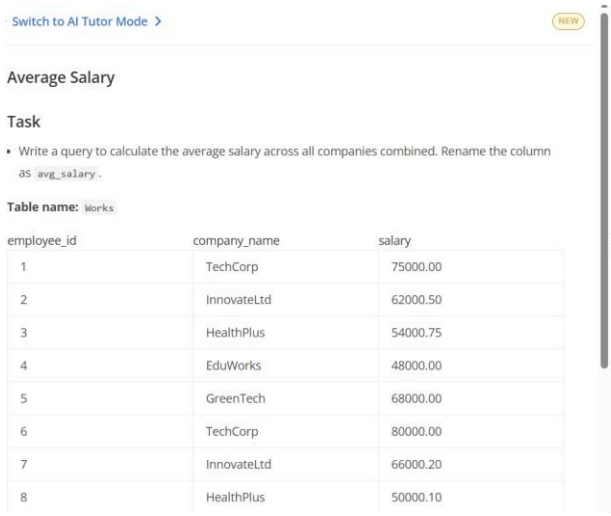
- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

SOLUTION

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT


Switch to AI Tutor Mode >

NEW

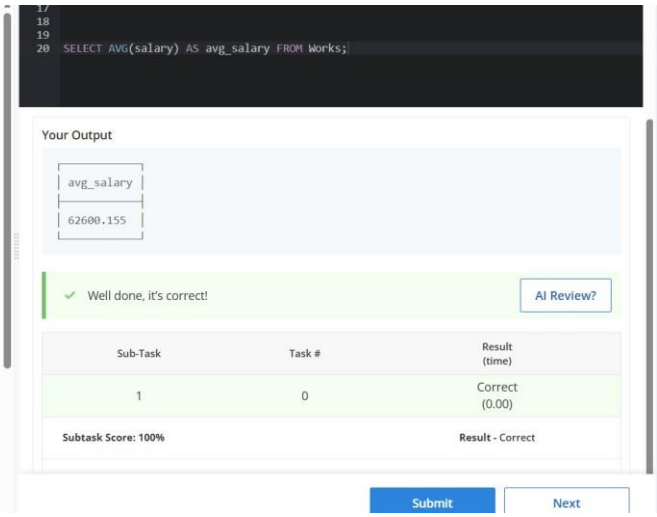
Average Salary

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as `avg_salary`.

Table name: `Works`

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10


```
17
18
19
20  SELECT AVG(salary) AS avg_salary FROM Works;
```

Your Output

avg_salary
62600.155

Well done, it's correct!

AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Subtask Score: 100%

Result - Correct

Submit

Next

LOCATE PEOPLE

TASK

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

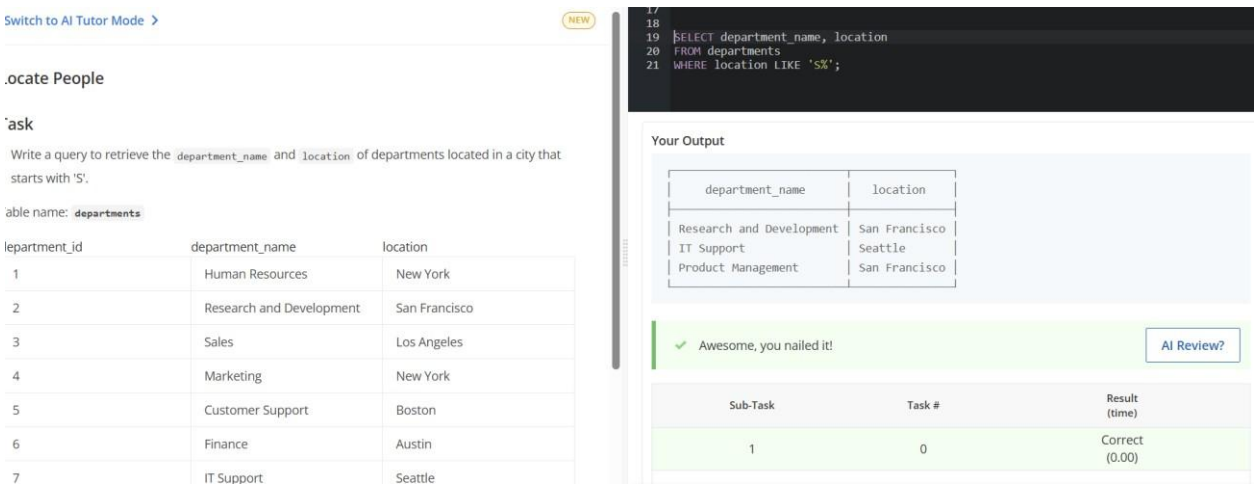
Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

SOLUTION

```
SELECT department_name, location
FROM departments
WHERE location LIKE 'S%';
```

OUTPUT



DISTINCT COMPANIES

TASK

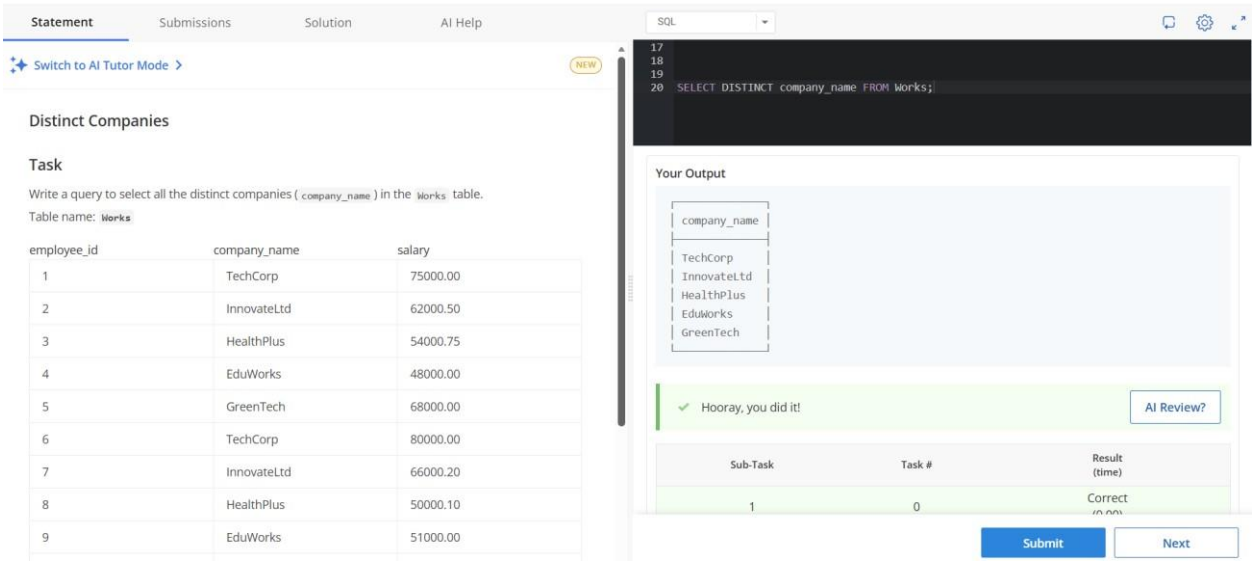
Write a query to select all the distinct companies (company_name) in the Works table. Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

SOLUTION

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT



FICTION COLLECTION SIZE

TASK

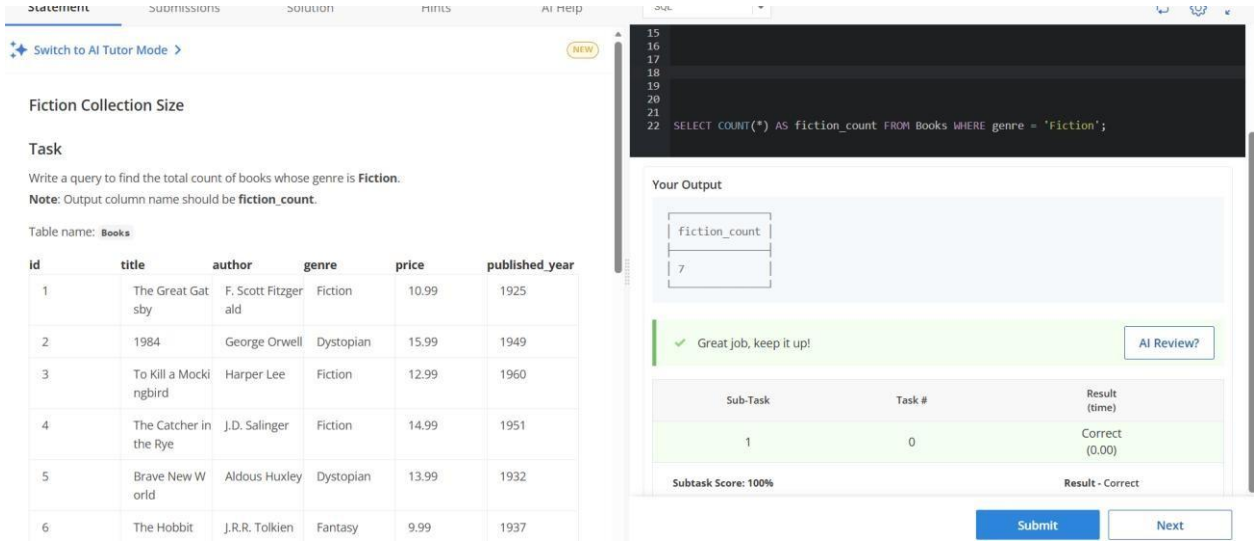
Write a query to find the total count of books whose genre is **Fiction**. **Note**: Output column name should be **fiction_count**.

Table name: **Books**

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 45	Ray Bradbury	Dystopian	12.50	1953

SOLUTION

SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction'; OUTPUT



LIST OF MOVIES WITH RATINGS

TASK

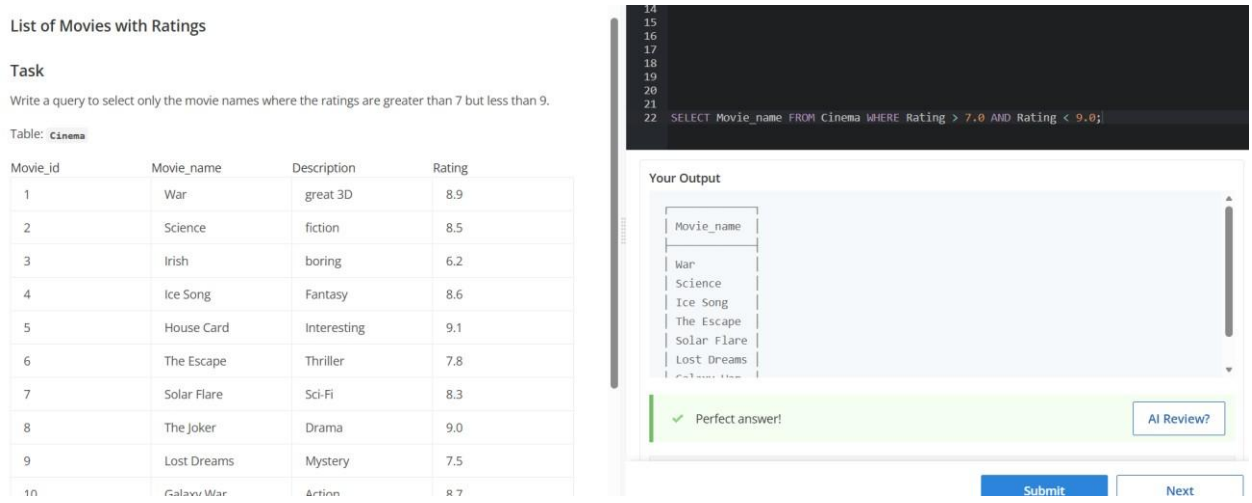
Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

SOLUTION

SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0; OUTPUT



HANDLING NULL VALUES

TASK

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

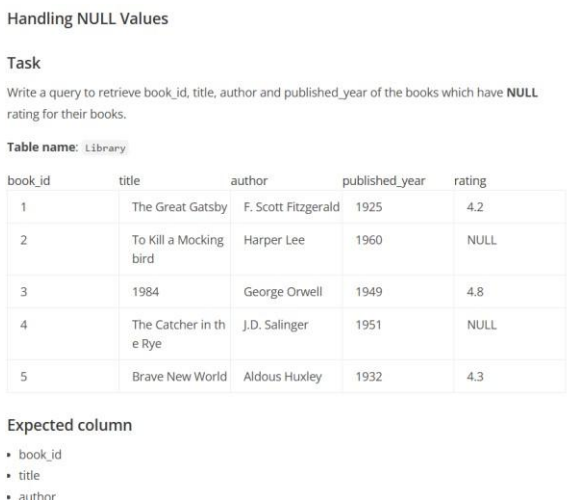
Table name: Library

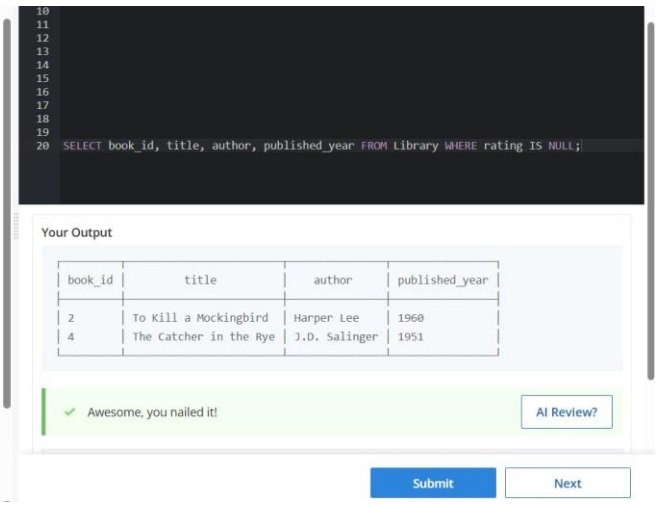
book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mocking bird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in th e Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

SOLUTION

SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;

OUTPUT





SALARY OF EMPLOYEES

TASK

Create a query to retrieve the employee_name, company, and salary for employees in the fulltime category, ordered by salary in **descending** order

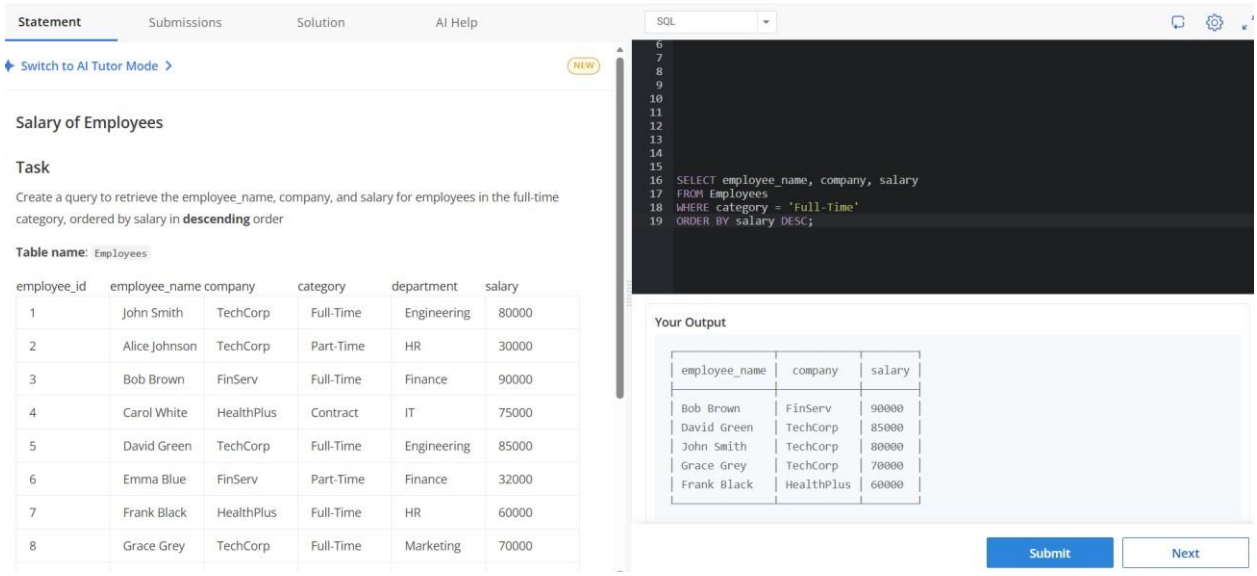
Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

SOLUTION

```
SELECT employee_name, company, salary
FROM Employees
WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

OUTPUT



DEPARTMENT OF EACH EMPLOYEE

TASK

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

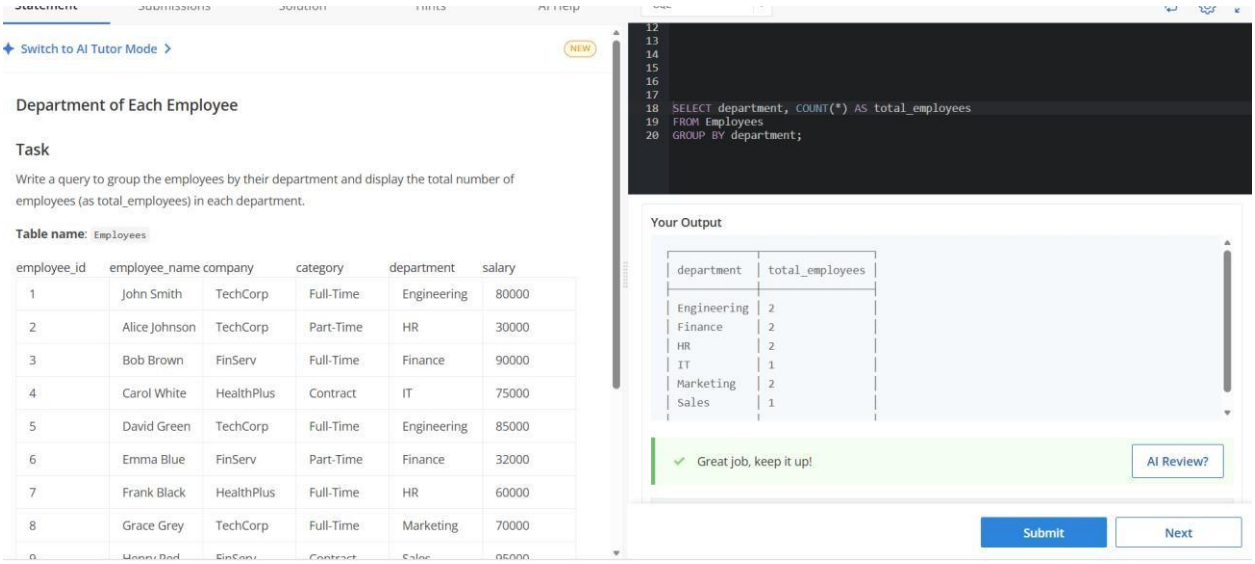
SOLUTION

SELECT department, COUNT(*) AS total_employees

FROM Employees

GROUP BY department;

OUTPUT



ARTICLE VIEWS

TASK

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

SOLUTION SELECT

DISTINCT

author_id,

author_name,

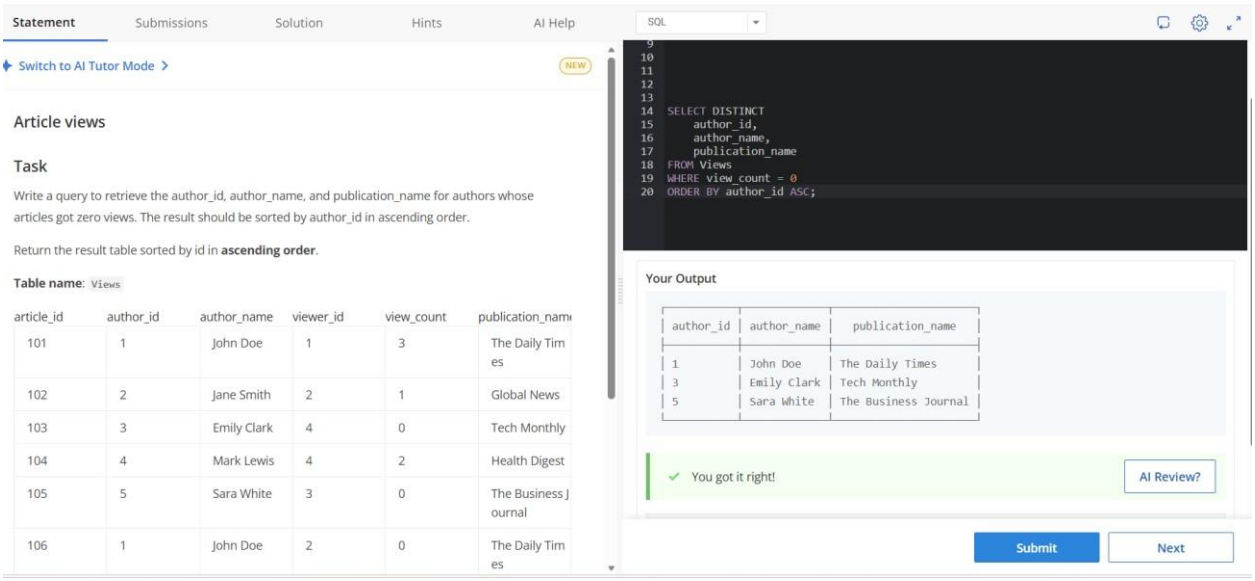
publication_name

FROM Views

WHERE view_count = 0

ORDER BY author_id ASC;

OUTPUT



PLAYER PERFORMANCE INSIGHTS

TASK

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

SOLUTION SELECT

DISTINCT

```
    p.player_name,
    p.score
FROM Players p
JOIN Matches m
ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

OUTPUT

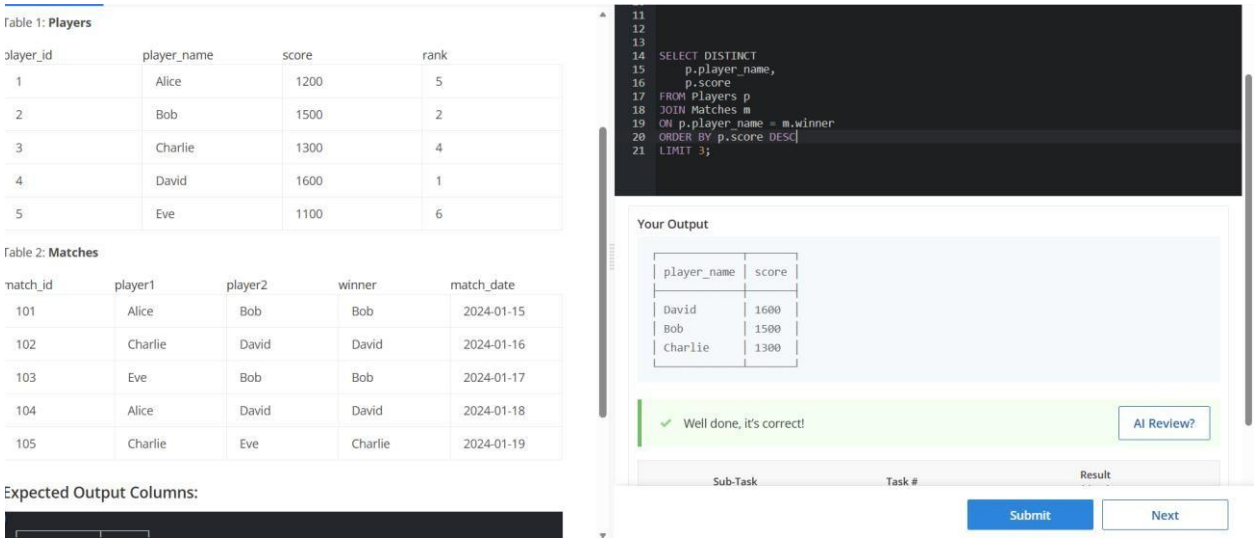


Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

Expected Output Columns:

```
11
12
13
14 SELECT DISTINCT
15     p.player_name,
16     p.score
17 FROM Players p
18 JOIN Matches m
19 ON p.player_name = m.winner
20 ORDER BY p.score DESC
21 LIMIT 3;
```

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

Well done, it's correct!

AI Review?

Sub-Task | Task # | Result

Submit | Next

PLAYER DETAILS

TASK

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

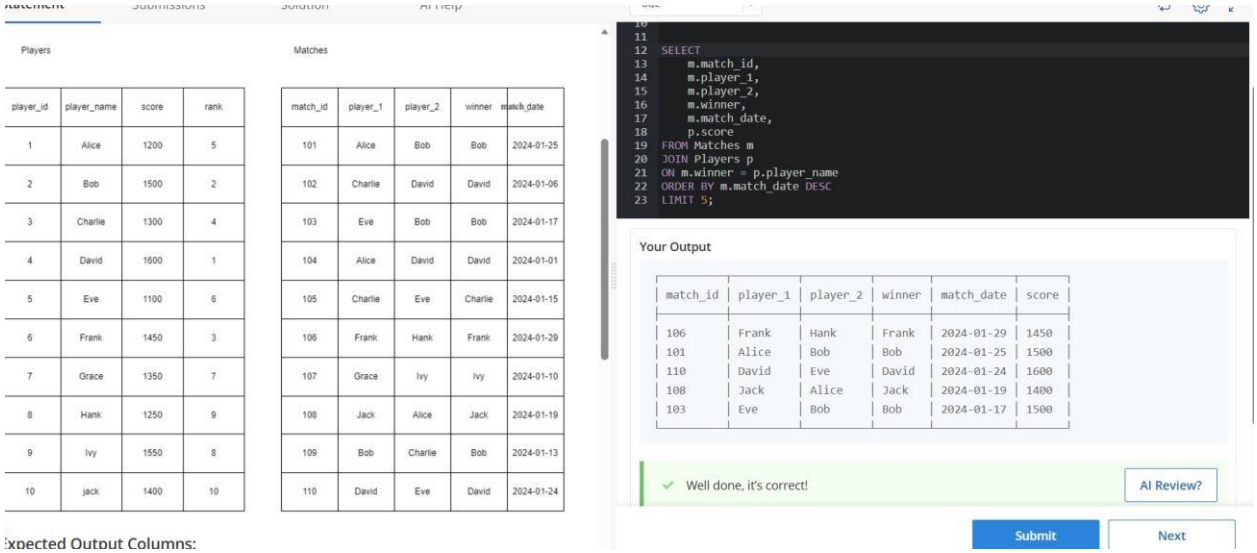
Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

SOLUTION

```
SELECT
    m.match_id,
    m.player_1,
    m.player_2,
    m.winner,
    m.match_date,
    p.score
FROM Matches m
JOIN Players p
ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT



Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

```
SELECT
    m.match_id,
    m.player_1,
    m.player_2,
    m.winner,
    m.match_date,
    p.score
FROM Matches m
JOIN Players p
ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

Well done, it's correct! AI Review?

Submit Next

Expected Output Columns:

FIND THE ERROR IN QUERY

Spot the Error in following query Select

count(),user_name from Users;

- You can refer to the dataset here.

SQL PRACTICE

Question 1: CABHUB and CUSTOMER

Create CABHUB Table

```
CREATE TABLE CABHUB (
   Vcode NUMBER(3),
   VehicleName VARCHAR2(20),
   Make VARCHAR2(20),
   Color VARCHAR2(10),
   Capacity NUMBER(2),
   Charges NUMBER(5)
);
```

Insert Records into CABHUB

```
INSERT INTO CABHUB VALUES (100, 'Innova', 'Toyota', 'WHITE', 7, 15);

INSERT INTO CABHUB VALUES (102, 'SX4', 'Suzuki', 'BLUE', 4, 14);

INSERT INTO CABHUB VALUES (104, 'C-Class', 'Mercedes', 'RED', 4, 35);

INSERT INTO CABHUB VALUES (105, 'A-Star', 'Suzuki', 'WHITE', 3, 14);

INSERT INTO CABHUB VALUES (108, 'Indigo', 'Tata', 'SILVER', 3, 12);
```

Create CUSTOMER Table

```
CREATE TABLE CUSTOMER (
   Code NUMBER(2),
   CName VARCHAR2(30),
   VCode NUMBER(3)
);
```

Insert Records into CUSTOMER

```
INSERT INTO CUSTOMER VALUES (1, 'Hemant Sahu', 101);

INSERT INTO CUSTOMER VALUES (2, 'Raj Lal', 108);
```

INSERT INTO CUSTOMER VALUES (3, 'Feroza Shah', 105);

INSERT INTO CUSTOMER VALUES (4, 'Ketan Dhal', 104);

SQL Queries 1

SELECT VehicleName

FROM CABHUB

WHERE Color = 'WHITE';

OUTPUT

Results Explain Describe Saved SQL History

VEHICLENAME
Innova
A-Star

2 rows returned in 0.03 seconds CSV Export

SQL query 2

SELECT VehicleName, Make, Capacity

FROM CABHUB

ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
Indigo	Tata	3
A-Star	Suzuki	3
SX4	Suzuki	4
C-Class	Mercedes	4
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

SQL query 3

SELECT MAX(Charges)

FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.01 seconds CSV Export

SQL query 4

```
SELECT C.CName, B.VehicleName
FROM CUSTOMER C, CABHUB B
WHERE C.VCode = B.VCode;
```

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.01 seconds CSV Export

SQL query 5

```
SELECT COUNT(DISTINCT Make)
FROM CABHUB;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.02 seconds CSV Export

SQL query 6

```
SELECT MAX(Charges), MIN(Charges)
FROM CABHUB;
```

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.01 seconds CSV Export

SQL query 7

```
SELECT COUNT(*), Make
```

FROM CABHUB

GROUP BY Make;



Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.03 seconds CSV Export

SQL query 8

SELECT VehicleName

FROM CABHUB

WHERE Capacity = 4;



Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.01 seconds CSV Export

Question 2: CO2_EMISSION

Create Table

CREATE TABLE CO2_EMISSION (

VehicleID NUMBER(3),

Company VARCHAR2(20),

Model VARCHAR2(20),

FuelType VARCHAR2(10),

EngineSize NUMBER(3,1),

CO2Emission NUMBER(3)

);

Insert Records

```
INSERT INTO CO2_EMISSION VALUES (101, 'Toyota', 'Corolla', 'Petrol', 1.8, 165);

INSERT INTO CO2_EMISSION VALUES (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 145);

INSERT INTO CO2_EMISSION VALUES (103, 'Honda', 'City', 'Petrol', 1.5, 152);

INSERT INTO CO2_EMISSION VALUES (104, 'Tata', 'Nexon', 'Diesel', 1.5, 138);

INSERT INTO CO2_EMISSION VALUES (105, 'Maruti', 'Swift', 'Petrol', 1.2, 125);

INSERT INTO CO2_EMISSION VALUES (106, 'Kia', 'Seltos', 'Diesel', 1.5, 150);

INSERT INTO CO2_EMISSION VALUES (107, 'Mahindra', 'XUV700', 'Diesel', 2.2, 185);
```

Query 1

```
SELECT *
FROM CO2_EMISSION
WHERE FuelType = 'Petrol';
```

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125
105	Maruti	Swift	Petrol	1.2	125

4 rows returned in 0.02 seconds CSV Export

Query 2

```
SELECT Company, Model FROM
CO2_EMISSION
WHERE EngineSize > 1.5;
```

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.01 seconds CSV Export

Query 3

SELECT Company, Model, CO2Emission

FROM CO2_EMISSION

ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
Toyota	Corolla	165
Mahindra	XUV700	185

8 rows returned in 0.00 seconds CSV Export

Query 4

SELECT *

FROM CO2_EMISSION

WHERE CO2Emission > 150;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185

3 rows returned in 0.01 seconds CSV Export

Query 5

SELECT COUNT(*)

FROM CO2_EMISSION

WHERE FuelType = 'Diesel';



Query 6

INSERT INTO CO2_EMISSION

VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);

Query 7

SELECT MAX(CO2Emission)

FROM CO2_EMISSION;



Query 8

SELECT MIN(EngineSize)

FROM CO2_EMISSION;



Query 9

SELECT AVG(CO2Emission)

FROM CO2_EMISSION

WHERE FuelType = 'Petrol';



Query 10

```
SELECT COUNT(*)
FROM CO2_EMISSION
WHERE EngineSize >= 1.5;
```



Question 3: APPLICANTS AND COURSES

Create Table – APPLICANTS

```
CREATE TABLE APPLICANTS (
   NO NUMBER(4),
   NAME VARCHAR2(20),
   FEE NUMBER(6),
   GENDER CHAR(1),
   C_ID VARCHAR2(3),
   JOINYEAR NUMBER(4)
);
```

Insert Records – APPLICANTS

```
INSERT INTO APPLICANTS VALUES (1012, 'Amandeep', 30000, 'M', 'A01', 2012);
```

INSERT INTO APPLICANTS VALUES (1102, 'Avisha', 25000, 'F', 'A02', 2009); INSERT INTO APPLICANTS VALUES (1103, 'Ekant', 30000, 'M', 'A02', 2011);

INSERT INTO APPLICANTS VALUES (1049, 'Arun', 30000, 'M', 'A03', 2009);

INSERT INTO APPLICANTS VALUES (1025, 'Amber', 40000, 'M', 'A02', 2011);

INSERT INTO APPLICANTS VALUES (1106, 'Ela', 40000, 'F', 'A05', 2010);

INSERT INTO APPLICANTS VALUES (1017, 'Nikita', 35000, 'F', 'A03', 2012);

INSERT INTO APPLICANTS VALUES (1108, 'Arleena', 30000, 'F', 'A03', 2012);

INSERT INTO APPLICANTS VALUES (2109, 'Shakti', 35000, 'M', 'A04', 2011);

INSERT INTO APPLICANTS VALUES (1101, 'Kirat', 25000, 'M', 'A01', 2012);

Create Table – COURSES

```
CREATE TABLE COURSES (
   C_ID VARCHAR2(3),
   COURSE VARCHAR2(30)
);
```

Insert Records – COURSES

```
INSERT INTO COURSES VALUES ('A01', 'Fashion Design');
INSERT INTO COURSES VALUES ('A02', 'Networking');
INSERT INTO COURSES VALUES ('A03', 'Hotel Management');
 INSERT INTO COURSES VALUES ('A04', 'Event Management');
INSERT INTO COURSES VALUES ('A05', 'Office Management');
 COMMIT;
```

QUERY 1

```
SELECT NAME, FEE, GENDER, JOINYEAR
FROM APPLICANTS
WHERE JOINYEAR < 2010;
```

Results | Explain | Describe | Saved SQL | History

NAME	FEE	GENDER	JOINYEAR
Avisha	25000	F	2009
Arun	30000	M	2009

2 rows returned in 0.01 seconds CSV Export

QUERY 2

```
SELECT NAME
FROM APPLICANTS
WHERE FEE > 30000;
```

Results | Explain | Describe | Saved SQL | History

NAME
Ela
Amber
Ela
Nikita
Shakti

5 rows returned in 0.01 seconds CSV Export

QUERY 3

```
SELECT NAME
FROM APPLICANTS
ORDER BY JOINYEAR ASC;
```

NAME
Arun
Avisha
Ela
Ela
Ekant
Shakti
Amber
Kirat
Nikita
Arleena
More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT JOINYEAR, COUNT(*) AS TOTAL
FROM APPLICANTS
GROUP BY JOINYEAR;
```

Results Explain Describe Saved SQL History

JOINYEAR	TOTAL
2010	2
2009	2
2011	3
2012	5

4 rows returned in 0.01 seconds CSV Export

QUERY 5

```
SELECT C_ID, COUNT(*) AS TOTAL
FROM APPLICANTS
GROUP BY C_ID;
```

Results Explain Describe Saved SQL History

C_ID	TOTAL
A03	3
A04	1
A02	3
A05	2
A01	3

5 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT A.NAME, C.COURSE
FROM APPLICANTS A
JOIN COURSES C
ON A.C_ID = C.C_ID;
```

Results Explain Describe Saved SQL History

NAME	COURSE
Amandeep	Fashion Design
Ela	Office Management
Amandeep	Fashion Design
Avisha	Networking
Ekant	Networking
Arun	Hotel Management
Amber	Networking
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
More than 10 rows available. Increase rows selector to view more rows.	

10 rows returned in 0.01 seconds CSV Export

QUERY 7

```
SELECT NAME, JOINYEAR
FROM APPLICANTS
WHERE GENDER = 'F' AND C_ID = 'A02';
```

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Avisha	2009

1 rows returned in 0.02 seconds CSV Export

QUERY 8

```
SELECT MIN(JOINYEAR)
FROM APPLICANTS
WHERE GENDER = 'M';
```



QUERY 9

```
SELECT AVG(FEE)
FROM APPLICANTS
WHERE C_ID = 'A01' OR C_ID = 'A05';
```



QUERY 10

```
SELECT SUM(FEE), C_ID
FROM APPLICANTS
GROUP BY C_ID
HAVING COUNT(*) = 2;
```



PRODUCTS AND SUPPLIERS

CREATE TABLE

```
CREATE TABLE PRODUCTS (
```

```
    PID NUMBER,
    PNAME VARCHAR2(30),
    QTY NUMBER,
    PRICE NUMBER,
    COMPANY VARCHAR2(20),
    SUPCODE VARCHAR2(5)
);

CREATE TABLE SUPPLIERS (
    SUPCODE VARCHAR2(5),
    SNAME VARCHAR2(30),
    CITY VARCHAR2(20)
);
```

INSERT RECORDS

```
INSERT INTO PRODUCTS VALUES (101, 'DIGITAL CAMERA14X', 120, 12000, 'RENIX', 'S01');

INSERT INTO PRODUCTS VALUES (102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP', 'S02');

INSERT INTO PRODUCTS VALUES (104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', 'S01');

INSERT INTO PRODUCTS VALUES (106, 'LED SCREEN', 70, 28000, 'DISEXPERTS', 'S02');

INSERT INTO PRODUCTS VALUES (105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON', 'S03');


INSERT INTO SUPPLIERS VALUES ('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS VALUES ('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS VALUES ('S02', 'DIGI BUSY GROUP', 'CHENNAI');
```

COMMIT; **QUERY**

1

SELECT *

FROM PRODUCTS

ORDER BY PNAME ASC;

Results Explain Describe Saved SQL History

PID	PNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

5 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT PNAME, PRICE

FROM PRODUCTS

WHERE PRICE BETWEEN 10000 AND 15000;

Results Explain Describe Saved SQL History

PNAME	PRICE
DIGITAL CAMERA14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.02 seconds CSV Export

QUERY 3

SELECT SUPCODE, COUNT(*)

FROM PRODUCTS

GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.04 seconds CSV Export

QUERY 4

```
SELECT PNAME, QTY, PRICE
FROM PRODUCTS
WHERE QTY > 100;
```

Results | Explain | Describe | Saved SQL | History

PNAME	QTY	PRICE
DIGITAL CAMERA14X	120	12000
PEN DRIVE 16 GB	500	1100

2 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT SNAME
FROM SUPPLIERS
WHERE CITY = 'DELHI' OR CITY = 'CHENNAI';
```

Results | Explain | Describe | Saved SQL | History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT COMPANY, PNAME
FROM PRODUCTS
ORDER BY COMPANY DESC;
```

Results | Explain | Describe | Saved SQL | History

COMPANY	PNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN
DIGI POP	DIGITAL PAD 11i

5 rows returned in 0.02 seconds CSV Export

QUERY 7

```
SELECT DISTINCT SUPCODE
```

FROM PRODUCTS;

Results Explain Describe Saved SQL Histor

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MAX(PRICE), MIN(PRICE)

FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT PRICE * QTY AS AMOUNT

FROM PRODUCTS

WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT P.PNAME, S.SNAME

FROM PRODUCTS P, SUPPLIERS S

WHERE P.SUPCODE = S.SUPCODE

AND P.QTY > 100;

Results Explain Describe Saved SQL History

PNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA14X	GET ALL INC

2 rows returned in 0.02 seconds CSV Export

SHOPPE and ACCESSORIES

CREATE TABLE

```
CREATE TABLE SHOPPE (
   ID VARCHAR2(5),
   SNAME VARCHAR2(30),
   AREA VARCHAR2(20)
);

CREATE TABLE ACCESSORIES (
   NO VARCHAR2(5),
   NAME VARCHAR2(30),
   PRICE NUMBER,
   ID VARCHAR2(5)
);
```

INSERT RECORDS

```
INSERT INTO SHOPPE VALUES ('S0001', 'ABC Computeronics', 'CP');
INSERT INTO SHOPPE VALUES ('S0002', 'All Infotech Media', 'GK II');
INSERT INTO SHOPPE VALUES ('S0003', 'Tech Shoppe', 'CP');
INSERT INTO SHOPPE VALUES ('S0004', 'Greeks Techno Soft', 'Nehru Place');
INSERT INTO SHOPPE VALUES ('S0005', 'Hitech Tech Store', 'Nehru Place');

INSERT INTO ACCESSORIES VALUES ('A01', 'Mother Board', 12000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A02', 'Hard Disk', 5000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A03', 'Keyboard', 500, 'S02');
INSERT INTO ACCESSORIES VALUES ('A04', 'Mouse', 300, 'S01');
INSERT INTO ACCESSORIES VALUES ('A05', 'Mother Board', 13000, 'S02');
INSERT INTO ACCESSORIES VALUES ('A06', 'Keyboard', 4000, 'S03');
```

INSERT INTO ACCESSORIES VALUES ('A07', 'LCD', 6000, 'S04');

INSERT INTO ACCESSORIES VALUES ('A08', 'LCD', 5000, 'S05');

INSERT INTO ACCESSORIES VALUES ('A09', 'Mouse', 350, 'S05');

INSERT INTO ACCESSORIES VALUES ('A10', 'Hard Disk', 4500, 'S03');

COMMIT; **QUERY**

1

SELECT NAME, PRICE

FROM ACCESSORIES

ORDER BY PRICE ASC;

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	500
Keyboard	4000
Hard Disk	4500
LCD	5000
Hard Disk	5000
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.03 seconds CSV Export

QUERY 2

SELECT S.ID, S.SNAME

FROM SHOPPE S

WHERE S.AREA = 'Nehru Place';

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.02 seconds CSV Export

QUERY 3

```
SELECT NAME, MIN(PRICE), MAX(PRICE)
FROM ACCESSORIES
GROUP BY NAME;
```

Results | Explain | Describe | Saved SQL | History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5000	6000
Mouse	300	350
Keyboard	500	4000
Hard Disk	4500	5000

5 rows returned in 0.01 seconds CSV Export

QUERY 4

```
SELECT A.NAME, A.PRICE, S.SNAME
FROM ACCESSORIES A, SHOPPE S
WHERE TO_NUMBER(SUBSTR(A.ID,2)) = TO_NUMBER(SUBSTR(S.ID,2));
```

Results | Explain | Describe | Saved SQL | History

NAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All Infotech Media
Keyboard	500	All Infotech Media
Mouse	300	ABC Computeronics
Mouse	300	ABC Computeronics
Mother Board	13000	All Infotech Media
Mother Board	13000	All Infotech Media
More than 10 rows available. Increase rows selector to view more rows.		

10 rows returned in 0.02 seconds CSV Export

QUERY 5

```
SELECT DISTINCT NAME
FROM ACCESSORIES
WHERE PRICE >= 500;
```

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.02 seconds CSV Export

QUERY 6

```
SELECT AREA, COUNT(*)
FROM SHOPPE
GROUP BY AREA;
```

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	2
CP	4
Nehru Place	4

3 rows returned in 0.02 seconds CSV Export

QUERY 7

```
SELECT COUNT(DISTINCT AREA)
FROM SHOPPE;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT NAME, PRICE * 0.05 AS DISCOUNT
FROM ACCESSORIES
WHERE ID IN ('S02', 'S03');
```



Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	200
Hard Disk	225
Keyboard	25
Mother Board	650
Keyboard	200
Hard Disk	225

8 rows returned in 0.00 seconds CSV Export

HOSPITAL

CREATE TABLE

```
CREATE TABLE HOSPITAL (
   NO NUMBER,
   NAME VARCHAR2(20),
   AGE NUMBER,
   DEPARTMENT VARCHAR2(30),
   DATEOFADMISSION DATE,
   CHARGES NUMBER,
   GENDER CHAR(1)
);
```

INSERT RECORDS

```
INSERT INTO HOSPITAL VALUES (1, 'Arpit', 62, 'Surgery', '21/01/98', 300, 'M');
INSERT INTO HOSPITAL VALUES (2, 'Zarina', 22, 'ENT', '12/12/97', 250, 'F');
INSERT INTO HOSPITAL VALUES (3, 'Kareem', 32, 'Orthopedic', '19/02/98', 200, 'M');
INSERT INTO HOSPITAL VALUES (4, 'Arun', 12, 'Surgery', '11/01/98', 300, 'M');
INSERT INTO HOSPITAL VALUES (5, 'Zubin', 30, 'ENT', '24/02/98', 250, 'M');
INSERT INTO HOSPITAL VALUES (6, 'Ketaki', 16, 'ENT', '12/01/98', 250, 'M');
INSERT INTO HOSPITAL VALUES (7, 'Ankit', 29, 'Cardiology', '20/02/98', 800, 'F');
```

INSERT INTO HOSPITAL VALUES (8, 'Zareen', 45, 'Gynecology', '22/02/98', 300, 'F');

INSERT INTO HOSPITAL VALUES (9, 'Kush', 19, 'Cardiology', '13/01/98', 800, 'M');

INSERT INTO HOSPITAL VALUES (10, 'Shilpa', 23, 'Nuclear Medicine', '21/02/98', 400, 'F');

COMMIT; **QUERY**

1

SELECT *

FROM HOSPITAL

WHERE DEPARTMENT = 'Cardiology';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADMISSION	CHARGES	GENDER
7	Ankit	29	Cardiology	20-FEB-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT NAME

FROM HOSPITAL

WHERE GENDER = 'F' AND DEPARTMENT = 'ENT';

Results Explain Describe Saved SQL History

NAME
Zarina

1 rows returned in 0.01 seconds CSV Export

QUERY 3

SELECT NAME, DATEOFADMISSION

FROM HOSPITAL

ORDER BY DATEOFADMISSION ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFADMISSION
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankit	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.01 seconds CSV Export

QUERY 4

```
SELECT NAME, CHARGES, AGE
FROM HOSPITAL
WHERE GENDER = 'F';
```

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankit	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.02 seconds CSV Export

QUERY 5

```
SELECT COUNT(*)
FROM HOSPITAL
WHERE AGE < 30;
```

Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
INSERT INTO HOSPITAL
```

VALUES (11, 'Aftab', 24, 'Surgery', '25/02/98', 300, 'M');

COMMIT;

NO	NAME	AGE	DEPARTMENT	DATEOFADMISSION	CHARGES	GENDER
1	Arpit	62	Surgery	21-JAN-98	300	M
2	Zarina	22	ENT	12-DEC-97	250	F
3	Kareem	32	Orthopedic	19-FEB-98	200	M
4	Arun	12	Surgery	11-JAN-98	300	M
5	Zubin	30	ENT	24-FEB-98	250	M
6	Ketaki	16	ENT	12-JAN-98	250	M
7	Ankit	29	Cardiology	20-FEB-98	800	F
8	Zareen	45	Gynecology	22-FEB-98	300	F
9	Kush	19	Cardiology	13-JAN-98	800	M
10	Shilpa	23	Nuclear Medicine	21-FEB-98	400	F

More than 10 rows available. Increase rows selector to view more rows.

QUERY 7

SELECT COUNT(DISTINCT CHARGES)

FROM HOSPITAL;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCHARGES)
5

1 rows returned in 0.02 seconds CSV Export

QUERY 8

SELECT MIN(AGE)

FROM HOSPITAL

WHERE GENDER = 'F';

Results Explain Describe Saved SQL History

MIN(AGE)
22

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT SUM(CHARGES)

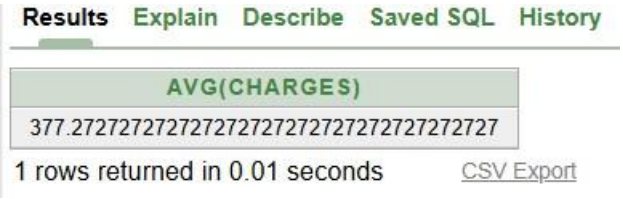
FROM HOSPITAL

WHERE DEPARTMENT = 'ENT';



QUERY 10

```
SELECT AVG(CHARGES)
FROM HOSPITAL
WHERE DATEOFADMISSION < '12/08/98';
```



STORE and **ITEM**

CREATE TABLE

```
CREATE TABLE STORE (
   SNO VARCHAR2(5),
   SNAME VARCHAR2(30),
   AREA VARCHAR2(20)
);

CREATE TABLE ITEM (
   INO VARCHAR2(5),
   INAME VARCHAR2(30),
   PRICE NUMBER,
   SNO VARCHAR2(5)
);
```

INSERT RECORDS

```
INSERT INTO STORE VALUES ('S01', 'ABC Computronics', 'GK II');
```

```
INSERT INTO STORE VALUES ('S02', 'All InfoTech Media', 'CP');
INSERT INTO STORE VALUES ('S03', 'Tech Shoppe', 'Nehru Place');
INSERT INTO STORE VALUES ('S04', 'Greeks Techno Soft', 'Nehru Place');
INSERT INTO STORE VALUES ('S05', 'Hitech Techno Store', 'CP');

INSERT INTO ITEM VALUES ('T01', 'Mother Board', 12000, 'S01');
INSERT INTO ITEM VALUES ('T02', 'Hard Disk', 5000, 'S01');
INSERT INTO ITEM VALUES ('T03', 'Keyboard', 500, 'S02');
INSERT INTO ITEM VALUES ('T04', 'Mouse', 300, 'S01');
INSERT INTO ITEM VALUES ('T05', 'Mother Board', 13000, 'S02');
INSERT INTO ITEM VALUES ('T06', 'Keyboard', 400, 'S03');
INSERT INTO ITEM VALUES ('T07', 'LCD', 6000, 'S04');
INSERT INTO ITEM VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ITEM VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ITEM VALUES ('T10', 'Hard Disk', 4500, 'S03');

COMMIT; QUERY
1
SELECT INAME, PRICE
FROM ITEM
ORDER BY PRICE ASC;
```

Results Explain Describe Saved SQL History

INAME	PRICE
Mother Board	12000
Hard Disk	5000
Keyboard	500
Mouse	300
Mother Board	13000
Keyboard	400
LCD	6000
LCD	5500
Mouse	350
Hard Disk	4500

10 rows returned in 0.02 seconds CSV Export

QUERY 2

```
SELECT SNO, SNAME
FROM STORE
WHERE AREA = 'CP';
```

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Techno Store

2 rows returned in 0.01 seconds CSV Export

QUERY 3

```
SELECT INAME, MIN(PRICE), MAX(PRICE)
FROM ITEM
GROUP BY INAME;
```

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT I.INAME, I.PRICE, S.SNAME
FROM ITEM I, STORE S
WHERE I.SNO = S.SNO;
```

Results | Explain | Describe | Saved SQL | History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computronics
Hard Disk	5000	ABC Computronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Techno Store
Mouse	350	Hitech Techno Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.02 seconds CSV Export

QUERY 5

```
SELECT DISTINCT INAME
FROM ITEM
WHERE PRICE >= 5000;
```

Results | Explain | Describe | Saved SQL | History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.02 seconds CSV Export

QUERY 6

```
SELECT AREA, COUNT(*)
FROM STORE
GROUP BY AREA;
```

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.02 seconds CSV Export

QUERY 7

```
SELECT COUNT(DISTINCT AREA)
FROM STORE;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT INAME, PRICE * 0.05 AS DISCOUNT
FROM ITEM
WHERE SNO IN ('S02', 'S03');
```

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.02 seconds CSV Export

TEACHER OF HISTORY DEPARTMENT

CREATE TABLE TEACHER

```
(
   No NUMBER,
   Name VARCHAR2(20),
   Age NUMBER,
   Department VARCHAR2(20),
```

```
    Dateofjoin DATE,
    Salary NUMBER,
    Gender CHAR(1)
);
```

INSERT RECORDS

```
INSERT INTO TEACHER VALUES (1, 'Jugal', 34, 'Computer',
TO_DATE('10/01/97','DD/MM/YY'), 12000, 'M');

INSERT INTO TEACHER VALUES (2, 'Sharmila', 31, 'History',
TO_DATE('24/03/98','DD/MM/YY'), 20000, 'F');

INSERT INTO TEACHER VALUES (3, 'Sandeep', 32, 'Maths',
TO_DATE('12/12/96','DD/MM/YY'), 30000, 'M');

INSERT INTO TEACHER VALUES (4, 'Sangeeta', 35, 'History',
TO_DATE('01/07/99','DD/MM/YY'), 40000, 'F');

INSERT INTO TEACHER VALUES (5, 'Rakesh', 42, 'Maths',
TO_DATE('05/09/97','DD/MM/YY'), 25000, 'M');

INSERT INTO TEACHER VALUES (6, 'Shyam', 50, 'History',
TO_DATE('27/06/98','DD/MM/YY'), 30000, 'M');

INSERT INTO TEACHER VALUES (7, 'Shiv Om', 44, 'Computer',
TO_DATE('25/02/97','DD/MM/YY'), 21000, 'M');

INSERT INTO TEACHER VALUES (8, 'Shalakha', 33, 'Maths',
TO_DATE('31/07/97','DD/MM/YY'), 20000, 'F');

COMMIT;
```

QUERY 1

```
SELECT *
FROM TEACHER
WHERE Department = 'History';
```

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.02 seconds CSV Export

Query 2

```
SELECT Name
FROM TEACHER
WHERE Gender = 'F'
AND Department = 'History';
```

Results | Explain | Describe | Saved SQL | History

NAME
Sharmila
Sangeeta

2 rows returned in 0.01 seconds CSV Export

Query 3

```
SELECT Name, Dateofjoin
FROM TEACHER
ORDER BY Dateofjoin ASC;
```

Results | Explain | Describe | Saved SQL

NAME	DATEOFJOIN
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98
Sangeeta	01-JUL-99

8 rows returned in 0.02 seconds CS

Query 4

```
SELECT Name, Salary, Age
FROM TEACHER
WHERE Gender = 'M';
```

Results Explain Describe Saved SQL Hist

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

5 rows returned in 0.00 seconds CSV Expo

Query 5

```
SELECT COUNT(*)
FROM TEACHER
WHERE Age > 23;
```

Results Explain Describe Saved SQL His

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Exp

Query 6

```
INSERT INTO TEACHER
VALUES (9, 'Raja', 26, 'Computer',
TO_DATE('13/09/95','DD/MM/YY'), 2300, 'M');
```

Results Explain Desc

```
1 row(s) inserted.
```

0.02 seconds

Query 7 (I)

```
SELECT COUNT(DISTINCT Department)
FROM TEACHER;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.02 seconds CSV Export

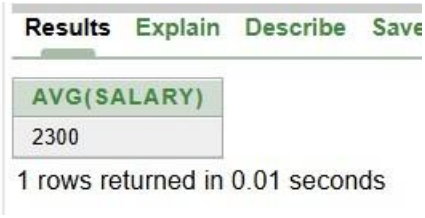
Query 7 (II)

```
SELECT MAX(Age)
FROM TEACHER
WHERE Gender = 'F';
```



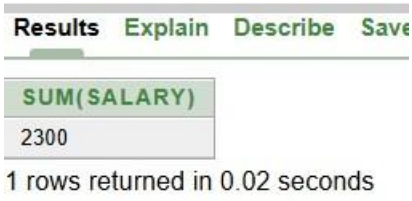
Query 7 (III)

```
SELECT AVG(Salary)
FROM TEACHER
WHERE Dateofjoin < TO_DATE('12/07/96','DD/MM/YY');
```



Query 7 (iv)

```
SELECT SUM(Salary)
FROM TEACHER
WHERE Dateofjoin < TO_DATE('12/07/96','DD/MM/YY');
```



LIBRARY:

CREATE TABLE

```
CREATE TABLE BOOKS
(
   Book_Id VARCHAR2(10),
```

```
   Book_Name VARCHAR2(30),
   Author_Name VARCHAR2(30),
   Publishers VARCHAR2(20),
   Price NUMBER,
   Type VARCHAR2(20),
   Qty NUMBER
);

CREATE TABLE ISSUED
(
   Book_Id VARCHAR2(10),
   Quantity_Issued NUMBER
);
```

INSERT RECORDS

```
INSERT INTO BOOKS VALUES ('F0001', 'The Tears', 'William Hopkins', 'First Publ.', 750, 'Fiction', 10);
INSERT INTO BOOKS VALUES ('F0002', 'Thunderbolts', 'Anna Roberts', 'First Publ.', 700, 'Fiction', 5);
INSERT INTO BOOKS VALUES ('T0001', 'My First C++', 'Brian & Brooke', 'EPB', 250, 'Text', 10);
INSERT INTO BOOKS VALUES ('T0002', 'C++ Brainworks', 'A. W. Rossaine', 'TDH', 325, 'Text', 5);
INSERT INTO BOOKS VALUES ('C0001', 'Fast Cook', 'Latkapoor', 'EPB', 350, 'Cookery', 8);
INSERT INTO ISSUED VALUES ('F0001', 3);
INSERT INTO ISSUED VALUES ('T0001', 1);
INSERT INTO ISSUED VALUES ('C0001', 5);
COMMIT;
```

QUERY 1

```
SELECT Book_Name, Author_Name, Price
```

FROM BOOKS

WHERE Publishers = 'EPB';

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
My First C++	Brian & Brooke	250
Fast Cook	Latkapoor	350

2 rows returned in 0.01 seconds CSV Export

QUERY 2

SELECT Book_Name

FROM BOOKS

WHERE Type = 'Fiction';

Results Explain Describe Saved SQL

BOOK_NAME
The Tears
Thunderbolts

2 rows returned in 0.02 seconds CSV

QUERY 3

SELECT Book_Name, Price

FROM BOOKS

ORDER BY Price DESC;

Results Explain Describe Saved

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.02 seconds

QUERY 4

UPDATE BOOKS

SET Price = Price + 50

WHERE Publishers = 'First Publ.';

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	AUTHOR_NAME	PUBLISHERS	PRICE	TYPE	QTY
F0001	The Tears	William Hopkins	First Publ.	800	Fiction	10
F0002	Thunderbolts	Anna Roberts	First Publ.	750	Fiction	5
T0001	My First C++	Brian & Brooke	EPB	250	Text	10
T0002	C++ Brainworks	A. W. Rossaine	TDH	325	Text	5
C0001	Fast Cook	Latkapoor	EPB	350	Cookery	8

5 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT BOOKS.Book_Id, Book_Name, Quantity_Issued
FROM BOOKS, ISSUED
WHERE BOOKS.Book_Id = ISSUED.Book_Id;
```

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
F0001	The Tears	3
T0001	My First C++	1
C0001	Fast Cook	5

3 rows returned in 0.02 seconds CSV Export

QUERY 6

```
INSERT INTO ISSUED VALUES ('F0002', 4);
COMMIT;
```

QUERY 7 (I)

```
SELECT COUNT(DISTINCT Publishers)
FROM BOOKS;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 7 (II)

```
SELECT SUM(Price)
FROM BOOKS
WHERE Qty > 5;
```

Results Explain Describe Save

SUM(PRICE)
1400

1 rows returned in 0.02 seconds

QUERY 7 (III)

```
SELECT BOOK_NAME, AUTHOR_NAME
FROM BOOKS
WHERE Price < 500;
```

Results Explain Describe Saved SQL H

BOOK_NAME	AUTHOR_NAME
My First C++	Brian & Brooke
C++ Brainworks	A. W. Rossaine
Fast Cook	Latkapoor

3 rows returned in 0.01 seconds CSV E

QUERY 7 (IV)

```
SELECT COUNT(*)
FROM BOOKS;
```

Results Explain Describe Saved

COUNT(*)
5

1 rows returned in 0.02 seconds

CARDEN and CUSTOMER

CREATE TABLE CARDEN

```
CREATE TABLE CARDEN (
    Ccode INT PRIMARY KEY,
    CarName VARCHAR(20),
```

```
    Make VARCHAR(20),
    Color VARCHAR(20),
    Capacity INT,
    Charges INT
);
```

INSERT INTO CARDEN

```
INSERT INTO CARDEN VALUES (501, 'A-star', 'Suzuki', 'RED', 3, 14);
INSERT INTO CARDEN VALUES (503, 'Indigo', 'Tata', 'SILVER', 3, 12);
INSERT INTO CARDEN VALUES (502, 'Innova', 'Toyota', 'WHITE', 7, 15);
INSERT INTO CARDEN VALUES (509, 'SX4', 'Suzuki', 'SILVER', 4, 14);
INSERT INTO CARDEN VALUES (510, 'C-Class', 'Mercedes', 'RED', 4, 35);
```

CREATE TABLE CUSTOMER1

```
CREATE TABLE CUSTOMER (
    CCode INT PRIMARY KEY,
    Cname VARCHAR(30),
    Ccode_Car INT
);
```

INSERT INTO CUSTOMER1

```
INSERT INTO CUSTOMER1 VALUES (1001, 'HamantSahu', 501);
INSERT INTO CUSTOMER1 VALUES (1002, 'Raj Lal', 509);
INSERT INTO CUSTOMER1 VALUES (1003, 'Feroja Shah', 503);
INSERT INTO CUSTOMER1 VALUES (1004, 'Ketan Dhal', 502);
```

QUERY 1

```
SELECT CarName
FROM CARDEN
WHERE Color = 'SILVER';
```

Results Explain Describe Saved

CARNAME
Indigo
SX4

2 rows returned in 0.02 seconds

QUERY 2

```
SELECT CarName, Make, Capacity
FROM CARDEN
ORDER BY Capacity DESC;
```

Results Explain Describe Saved SQL Hist

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Expo

QUERY 3

```
SELECT MAX(Charges)
FROM CARDEN;
```

Results Explain Describe Saved SQ

MAX(CHARGES)
35

1 rows returned in 0.01 seconds

QUERY 4

```
SELECT CUSTOMER1.Cname, CARDEN.CarName
FROM CUSTOMER1
JOIN CARDEN
ON CUSTOMER1.Ccode_Car = CARDEN.Ccode;
```

Results Explain Describe Saved SQL Histor

CNAME	CARNAME
HamantSahu	A-star
Feroja Shah	Indigo
Ketan Dhal	Innova
Raj Lal	SX4

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(DISTINCT Make)

FROM CARDEN;

Results Explain Describe Save

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds

QUERY 6

SELECT MAX(Charges), MIN(Charges)

FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(*), Make

FROM CARDEN

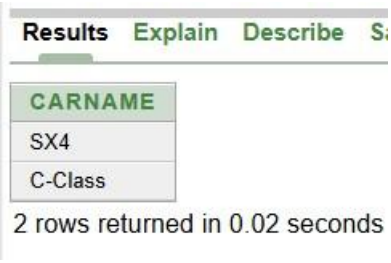
GROUP BY Make;

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds

QUERY 8

```
SELECT CarName
FROM CARDEN
WHERE Capacity = 4;
```



EMPLOYEE and SALGRADE

CREATE TABLE EMPLOYEE1

```
CREATE TABLE EMPLOYEE1 (
   ECODE INT PRIMARY KEY,
   NAME VARCHAR(30),
   DESIG VARCHAR(30),
   SGRADE VARCHAR(10),
   DOJ DATE,
   DOB DATE
);
```

INSERT INTO EMPLOYEE1

```
INSERT INTO EMPLOYEE1 VALUES (101, 'Abdul Ahmad', 'EXECUTIVE', 'S03', '23-MAR2003', '13-JAN-1980');
INSERT INTO EMPLOYEE1 VALUES (102, 'Ravi Chander', 'HEAD-IT', 'S02', '12-FEB-2010', '22-JUL-1987');
INSERT INTO EMPLOYEE1 VALUES (103, 'John Ken', 'Receptionist', 'S03', '24-JUN-2009', '24-FEB-1983');
```

```
INSERT INTO EMPLOYEE1 VALUES (105, 'NazamAreen', 'GM', 'S02', '11-AUG-2006',
'03MAR-1984');

INSERT INTO EMPLOYEE1 VALUES (108, 'PriyamSen', 'CEO', 'S01', '29-DEC-2004',
'19JAN-1982');
```

CREATE TABLE SALGRADE

```
CREATE TABLE SALGRADE (
   SGRADE VARCHAR(10),
   SALARY INT,
   HRA INT
);
```

INSERT INTO SALGRADE

```
INSERT INTO SALGRADE VALUES ('S01', 56000, 18000);

INSERT INTO SALGRADE VALUES ('S02', 32000, 12000);

INSERT INTO SALGRADE VALUES ('S03', 24000, 8000);
```

QUERY 1

```
SELECT *
FROM EMPLOYEE1
ORDER BY DOJ DESC;
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazamAreen	GM	S02	11-AUG-06	03-MAR-84
108	PriyamSen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.00 seconds CSV Export

QUERY 2

```
SELECT NAME, DESIG
FROM EMPLOYEE1
WHERE SGRADE IN ('S02', 'S03');
```

Results Explain Describe Sa

NAME	DESIG
Abdul Ahmad	EXECUTIVE
Ravi Chander	HEAD-IT
John Ken	Receptionist
NazamAreen	GM

4 rows returned in 0.00 seconds

QUERY 3

```
SELECT *
FROM EMPLOYEE1
WHERE DOJ BETWEEN '09-FEB-2006' AND '08-AUG-2009';
```

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazamAreen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.01 seconds CSV Export

QUERY 4

```
INSERT INTO EMPLOYEE1
VALUES (109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983');
```

QUERY 5

```
SELECT COUNT(SGRADE), SGRADE
FROM EMPLOYEE1
GROUP BY SGRADE;
```

COUNT(SGRADE)	SGRADE
2	S03
3	S02
1	S01

3 rows returned in 0.00 seconds

QUERY 6

```
SELECT MIN(DOB), MAX(DOJ)
FROM EMPLOYEE1;
```

Results Explain Describe Save

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.01 seconds

QUERY 7

```
SELECT NAME, SALARY
FROM EMPLOYEE1 E, SALGRADE S
WHERE E.SGRADE = S.SGRADE
AND E.ECODE < 103;
```

Results Explain Describe Sav

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.01 seconds

QUERY 8

```
SELECT SGRADE, SALARY + HRA
FROM SALGRADE
WHERE SGRADE = 'S02';
```

Results Explain Describe Saved

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.01 seconds

WORKER and PAYLAVEL

CREATE TABLE WORKER

```
CREATE TABLE WORKER (
   ECODE INT PRIMARY KEY,
   NAME VARCHAR(30),
   DESIGN VARCHAR(30),
   PLEVEL VARCHAR(10),
```

```
   DOJ DATE,
   DOB DATE
);
```

INSERT INTO WORKER

```
INSERT INTO WORKER VALUES (11, 'RadheShyam', 'Supervisor', 'P001', '13-SEP-2004',
'23-AUG-1981');
INSERT INTO WORKER VALUES (12, 'ChanderNath', 'Operator', 'P003', '22-FEB-2010',
'12JUL-1987');
INSERT INTO WORKER VALUES (13, 'Fizza', 'Operator', 'P003', '14-JUN-2009', '14-
OCT1983');
INSERT INTO WORKER VALUES (15, 'Ahmeen Ahmad', 'Mechanic', 'P002', '21-AUG-2006',
'13-MAR-1984');
INSERT INTO WORKER VALUES (18, 'Sanya', 'Clerk', 'P002', '19-DEC-2005', '09-JUN1983');
```

CREATE TABLE PAYLEVEL CREATE

```
TABLE PAYLEVEL (
   PLEVEL VARCHAR(10),
   PAY INT,
   ALLOWANCE INT
);
```

INSERT INTO PAYLEVEL

```
INSERT INTO PAYLEVEL VALUES ('P001', 26000, 12000);
INSERT INTO PAYLEVEL VALUES ('P002', 22000, 10000);
INSERT INTO PAYLEVEL VALUES ('P003', 12000, 6000);
```

QUERY 1

```
SELECT *
FROM WORKER
ORDER BY DOB DESC;
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.02 seconds CSV Export

QUERY 2

```
SELECT NAME, DESIGN
FROM WORKER
WHERE PLEVEL IN ('P001', 'P002');
```

Results Explain Describe Saved

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.01 seconds

QUERY 3

```
SELECT *
FROM WORKER
WHERE DOB BETWEEN '19-JAN-1984' AND '18-JAN-1987';
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.02 seconds CSV Export

QUERY 4

```
INSERT INTO WORKER
VALUES (19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984');
```

QUERY 5

```
SELECT COUNT(PLEVEL), PLEVEL
FROM WORKER
```

GROUP BY PLEVEL;

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.00 seconds CSV Ex

QUERY 6

SELECT MAX(DOB), MIN(DOJ)

FROM WORKER;

Results Explain Describe Savec

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.01 seconds

QUERY 7

SELECT NAME, PAY

FROM WORKER W, PAYLEVEL P

WHERE W.PLEVEL = P.PLEVEL

AND W.ECODE < 13;

Results Explain Describe S

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.00 seconds

QUERY 8

SELECT PLEVEL, PAY + ALLOWANCE

FROM PAYLEVEL

WHERE PLEVEL = 'P003';

Results Explain Describe Savec

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.02 seconds

STORE and SUPPLIERS

STORE2

```
CREATE TABLE STORE2 (
   ItemNo INT PRIMARY KEY,
   Item VARCHAR(30),
   Scode INT,
   Qty INT,
   Rate INT,
   LastBuy DATE
);
```

SUPPLIERS2

```
CREATE TABLE SUPPLIERS2 (
   Scode INT PRIMARY KEY,
   Sname VARCHAR(30)
);
```

INSERT — STORE2

```
INSERT INTO STORE2 VALUES (2005, 'Sharpener Classic', 23, 60, 8,
TO_DATE('31/06/09','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2003, 'Ball Pen 0.25', 22, 50, 25,
TO_DATE('01/02/09','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2002, 'Gel Pen Premium', 21, 150, 12,
TO_DATE('24/02/10','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2006, 'Gel Pen Classic', 21, 250, 20,
TO_DATE('11/05/09','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2001, 'Eraser Small', 22, 220, 6,
```

TO_DATE('19/01/09','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2004, 'Eraser Big', 22, 110, 8,
TO_DATE('02/12/09','DD/MM/YY'));

INSERT INTO STORE2 VALUES (2009, 'Ball Pen 0.5', 21, 180, 18,
TO_DATE('03/11/09','DD/MM/YY'));

INSERT — SUPPLIERS2

INSERT INTO SUPPLIERS2 VALUES (21, 'Premium Stationers');

INSERT INTO SUPPLIERS2 VALUES (23, 'Soft Plastics');

INSERT INTO SUPPLIERS2 VALUES (22, 'Tetra Supply');

QUERY 1

SELECT *

FROM STORE2

ORDER BY LastBuy ASC;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT ItemNo, Item

FROM STORE2

WHERE Rate > 15;

Results Explain Describe Saved S

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.01 seconds

QUERY 3

```
SELECT *
FROM STORE2
WHERE Scode = 22 OR Qty > 110;
```

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT Scode, MIN(Rate)
FROM STORE2
GROUP BY Scode;
```

Results Explain Describe Savec

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds

QUERY 5

```
SELECT COUNT(DISTINCT Scode)
FROM STORE2;
```

Results Explain Describe Saved S

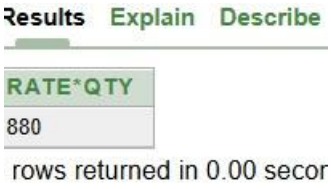
COUNT(DISTINCTSCODE)
3

1 rows returned in 0.02 seconds

QUERY 6

```
SELECT Rate * Qty
FROM STORE2
```

WHERE ItemNo = 2004;


Results Explain Describe

RATE*QTY
880

rows returned in 0.00 secor

QUERY 7

SELECT Item, Sname

FROM STORE2 S, SUPPLIERS2 P

WHERE S.Scode = P.Scode

AND ItemNo = 2006;


Results Explain Describe Saved SQL His

ITEM	SNAME
Gel Pen Classic	Premium Stationers

1 rows returned in 0.02 seconds CSV Exp

QUERY 8

SELECT MAX(LastBuy)

FROM STORE2;


Results Explain Describe Saved

MAX(LASTBUY)
24-FEB-10

1 rows returned in 0.02 seconds

GARMENT and FABRIC

CREATE TABLE GARMENT2

CREATE TABLE GARMENT2 (

GCODE INT PRIMARY KEY,

DESCRIPTION VARCHAR(30),

PRICE INT,

FCODE VARCHAR(10),

```
   READYDATE DATE
);
```

INSERT INTO GARMENT2

```
INSERT INTO GARMENT2 VALUES (10023, 'Pencil Skirt', 1150, 'F03',
TO_DATE('19/12/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10001, 'Formal Shirt', 1250, 'F01',
TO_DATE('12/01/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10012, 'Informal Shirt', 1550, 'F02',
TO_DATE('06/06/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10024, 'Baby Top', 750, 'F03',
TO_DATE('07/04/07','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10090, 'Tupik Shirt', 850, 'F02',
TO_DATE('31/03/07','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10019, 'Evening Gown', 850, 'F03',
TO_DATE('06/06/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10009, 'Informal Pant', 1500, 'F02',
TO_DATE('20/10/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10007, 'Formal Pant', 1350, 'F01',
TO_DATE('09/03/08','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10020, 'Frock', 850, 'F04',
TO_DATE('09/09/07','DD/MM/YY'));

INSERT INTO GARMENT2 VALUES (10089, 'Slacks', 750, 'F03',
TO_DATE('20/10/08','DD/MM/YY'));
```

CREATE TABLE FABRIC2

```
CREATE TABLE FABRIC2 (
   FCODE VARCHAR(10) PRIMARY KEY,
   TYPE VARCHAR(20)
);
```

INSERT INTO FABRIC2

```
INSERT INTO FABRIC2 VALUES ('F04', 'Polyester');

INSERT INTO FABRIC2 VALUES ('F02', 'Cotton');

INSERT INTO FABRIC2 VALUES ('F03', 'Silk');
```

INSERT INTO FABRIC2 VALUES ('F01', 'Terelene');

QUERY 1

```
SELECT GCODE, DESCRIPTION
FROM GARMENT2
ORDER BY GCODE DESC;
```

Results Explain Describe Saved SQ

GCODE	DESCRIPTION
10090	Tupik Shirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds

QUERY 2

```
SELECT *
FROM GARMENT2
WHERE READYDATE BETWEEN TO_DATE('08/12/07','DD/MM/YY')
AND TO_DATE('16/06/08','DD/MM/YY');
```

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.02 seconds CSV Export

QUERY 3

```
SELECT AVG(PRICE)
FROM GARMENT2
WHERE FCODE = 'F03';
```

Results Explain Describe Saved

AVG(PRICE)
875

1 rows returned in 0.00 seconds

QUERY 4

```
SELECT FCODE, MAX(PRICE), MIN(PRICE)
FROM GARMENT2
GROUP BY FCODE;
```

Results Explain Describe Saved SQL Hist

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.02 seconds CSV Expo

QUERY 5

```
SELECT SUM(PRICE)
FROM GARMENT2
WHERE FCODE = 'F01';
```

Results Explain Describe

SUM(PRICE)
2600

1 rows returned in 0.00 secon

QUERY 6

```
SELECT DESCRIPTION, TYPE
FROM GARMENT2, FABRIC2
WHERE GARMENT2.FCODE = FABRIC2.FCODE
AND GARMENT2.PRICE >= 1260;
```

Results Explain Describe Saved SQL

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.01 seconds CSV

QUERY 7

SELECT MAX(FCODE)

FROM FABRIC2;

Results Explain Describe Sa

MAX(FCODE)
F04

1 rows returned in 0.00 seconds

QUERY 8

SELECT COUNT(DISTINCT PRICE)

FROM GARMENT2;

Results Explain Describe Saved

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.02 seconds

FURNITURE AND ARRIVALS.

CREATE TABLE

CREATE TABLE FURNITURE(FNO INT,ITEMNAME VARCHAR(20),TYPE VARCHAR(30),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);

INSERT RECORDS

INSERT INTO FURNITURE VALUES (1,'WHITE LOTUS','DOUBLE BED',TO_DATE('23-02-2002','DD-MM-YYYY'),30000,25);

INSERT INTO FURNITURE VALUES (2,'PINK FEATHER','BABY COT',TO_DATE('20-01-2002','DD-MM-YYYY'),7000,20);

INSERT INTO FURNITURE VALUES (3,'DOLPHIN','BABY COT',TO_DATE('19-02-2002','DD-MM-YYYY'),9500,20);

INSERT INTO FURNITURE VALUES (4,'DECENT','OFFICE TABLE',TO_DATE('01-01-2002','DD-MM-YYYY'),25000,30);

INSERT INTO FURNITURE VALUES (5,'COMFORT ZONE','DOUBLE BED',TO_DATE('12-01-2002','DD-MM-YYYY'),25000,25);

INSERT INTO FURNITURE VALUES (6,'DONALD','BABY COT',TO_DATE('24-02-2002','DD-MM-YYYY'),6500,15);

INSERT INTO FURNITURE VALUES (7,'ROYAL FINISH','OFFICE TABLE',TO_DATE('20-02-2002','DD-MM-YYYY'),18000,30);

INSERT INTO FURNITURE VALUES (8,'ROYAL TIGER','SOFA',TO_DATE('22-02-2002','DD-MM-YYYY'),31000,30);

INSERT INTO FURNITURE VALUES (9,'ECONO SITTING','SOFA',TO_DATE('13-12-2001','DD-MM-YYYY'),9500,25);

INSERT INTO FURNITURE VALUES (10,'EATING PARADISE','DINING TABLE',TO_DATE('19-02-2002','DD-MM-YYYY'),11500,25);

QUERY 1

SELECT *

FROM FURNITURE

WHERE TYPE ='BABY COT';

Results Explain Describe Saved SQL History

FNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	pink feather	baby cot	20-JAN-02	7000	20
3	dolphin	baby cot	19-FEB-02	9500	20
6	donald	baby cot	24-FEB-02	6500	15

3 rows returned in 0.04 seconds CSV Export

QUERY 2

```
SELECT ITEMNAME
FROM FURNITURE
WHERE PRICE > 15000;
```

Results Explain Describe Saved

ITEMNAME
white lotus
decent
comfort zone
royal finish
royal tiger

5 rows returned in 0.01 seconds

QUERY 3

```
SELECT ITEMNAME,TYPE
FROM FURNITURE
WHERE DATEOFSTOCK > TO_DATE('12-02-2002','DD-MM-YYYY') ORDER BY
ITEMNAME DESC;
```

Results Explain Describe Saved SQL

ITEMNAME	TYPE
white lotus	double bed
royal tiger	sofa
royal finish	office table
eating paradise	dining table
donald	baby cot
dolphin	baby cot

6 rows returned in 0.02 seconds CSV

QUERY 4

```
SELECT ITEMNAME,DATEOFSTOCK
FROM FURNITURE
WHERE DISCOUNT > 25;
```

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
decent	01-JAN-02
royal finish	20-FEB-02
royal tiger	22-FEB-02

3 rows returned in 0.01 seconds CSV Export

QUERY 5

SELECT COUNT(*) FROM FURNITURE

WHERE TYPE='SOFA';

Results Explain Des

COUNT(*)
2

l rows returned in 0.02

CREATE TABLE

CREATE TABLE ARRIVALS(ANO INT,ITEMNAME VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);

INSERT RECORDS

INSERT INTO ARRIVALS VALUES (11, 'WOOD COMFORT', 'DOUBLE BED', TO_DATE('23-03-2003','DD-MM-YYYY'), 25000, 25);

INSERT INTO ARRIVALS VALUES(12,'OLD FOX','SOFA',TO_DATE('20-02-2003','DDMM-YYYY'),17000,20);

INSERT INTO ARRIVALS VALUES(13,'MICKY','BABY COT',TO_DATE('21-02-2002','DDMM-YYYY'),7500,15);

INSERT INTO ARRIVALS VALUES(14, 'VELVET TOUCH', 'DOUBLE BED', TO_DATE('25-03-2003','DD-MM-YYYY'), 25000,30);

QUERY 6

SELECT COUNT(DISTINCT TYPE) FROM FURNITURE;

Results Explain Describe Saved

COUNT(DISTINCTTYPE)
5

1 rows returned in 0.02 seconds

QUERY 7

SELECT MAX(DISCOUNT) FROM FURNITURE;



QUERY 8

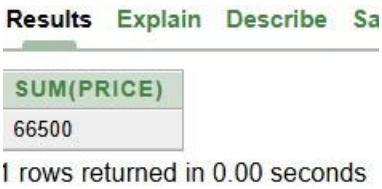
SELECT AVG(DISCOUNT) FROM FURNITURE

WHERE TYPE = 'BABY COT';



QUERY 9

SELECT SUM(PRICE) FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('12-02-2002','DD-MM-YYYY');



COVID19

CREATE TABLE COVID192

CREATE TABLE COVID192 (

PatientID INT PRIMARY KEY,

PatientName VARCHAR(30),

State VARCHAR(30),

Age INT,

Status VARCHAR(20),

```
    Vaccinated VARCHAR(10)
);
```

INSERT INTO COVID192

```
INSERT INTO COVID192 VALUES (101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');
INSERT INTO COVID192 VALUES (102, 'Priya', 'Kerala', 42, 'Active', 'Yes');
INSERT INTO COVID192 VALUES (103, 'Karthik', 'Karnataka', 28, 'Recovered', 'No');
INSERT INTO COVID192 VALUES (104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');
INSERT INTO COVID192 VALUES (105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');
INSERT INTO COVID192 VALUES (106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');
INSERT INTO COVID192 VALUES (107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');
```

QUERY 1

```
SELECT *
FROM COVID192
WHERE State = 'Tamil Nadu';
```

Results | Explain | Describe | Saved SQL | History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.02 seconds CSV Export

QUERY 2

```
SELECT PatientName
FROM COVID192
WHERE Status = 'Recovered';
```

Results | Explain | Describ

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.02 sec

QUERY 3

```
SELECT PatientName, Age
FROM COVID192
ORDER BY Age ASC;
```

Results Explain Describe Saved SQ

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds

QUERY 4

```
SELECT PatientName
FROM COVID192
WHERE Vaccinated = 'Yes';
```

Results Explain

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0

QUERY 5

```
SELECT COUNT(*)
FROM COVID192
WHERE Status = 'Active';
```

Results Explain Describe Saved SQL His

COUNT(*)
3

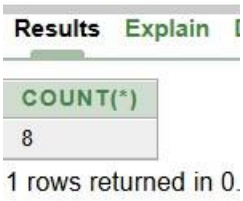
1 rows returned in 0.00 seconds CSV Exp

QUERY 6

```
INSERT INTO COVID192
VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');
```

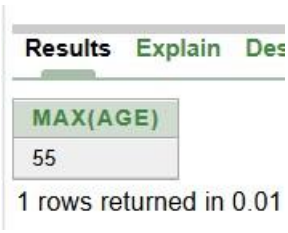
QUERY 7

```
SELECT COUNT(*)
FROM COVID192;
```



QUERY 8

```
SELECT MAX(Age)
FROM COVID192;
```



QUERY 9

```
SELECT COUNT(*)
FROM COVID192
WHERE Vaccinated = 'Yes';
```



QUERY 10

```
SELECT MIN(Age)
FROM COVID192
WHERE Status = 'Recovered';
```

MIN(AGE)
24

1 rows returned in 0.(

HOUSEPRICE

CREATE TABLE HOUSEPRICE

```
CREATE TABLE HOUSEPRICE (
   HouseID INT PRIMARY KEY,
   Location VARCHAR(30),
   Bedrooms INT,
   AreaSqft INT,
   Price INT,
   HouseType VARCHAR(20)
);
```

INSERT INTO HOUSEPRICE

```
INSERT INTO HOUSEPRICE VALUES (201, 'Chennai', 2, 1100, 5200000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES (202, 'Coimbatore', 3, 1450, 6800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES (203, 'Madurai', 2, 980, 4100000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES (204, 'Chennai', 4, 2200, 9800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES (205, 'Trichy', 3, 1650, 7200000, 'Independent');
INSERT INTO HOUSEPRICE VALUES (206, 'Salem', 2, 1200, 5000000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES (207, 'Erode', 3, 1800, 7600000, 'Independent');
```

Query 1

```
SELECT *
FROM HOUSEPRICE
WHERE Location = 'Chennai';
```

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

Query 2

```
SELECT Location, Price
FROM HOUSEPRICE
WHERE Bedrooms = 3;
```

Results Explain Describe Saved

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.01 seconds

Query 3

```
SELECT Location, AreaSqft, Price
FROM HOUSEPRICE
ORDER BY Price DESC;
```

Results Explain Describe Saved SQL

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1650	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.01 seconds CSV

Query 4

```
SELECT *
FROM HOUSEPRICE
WHERE Price > 7000000;
```

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.02 seconds CSV Export

Query 5

```
SELECT COUNT(*)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';
```

Results Explain

COUNT(*)
3

1 rows returned in 0

Query 6

```
INSERT INTO HOUSEPRICE
VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');
```

Query 7

```
SELECT MAX(Price)
FROM HOUSEPRICE;
```

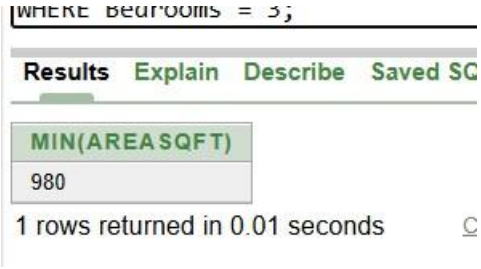
Results Explain De

MAX(PRICE)
9800000

1 rows returned in 0.01

Query 8

```
SELECT MIN(AreaSqft)
FROM HOUSEPRICE;
```



Query 9

```
SELECT AVG(Price)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';
```



Query 10

```
SELECT COUNT(*)
FROM HOUSEPRICE
WHERE Bedrooms = 3;
```



CANCERDATA

CREATE TABLE CANCERDATA

```
CREATE TABLE CANCERDATA (
   PatientID INT PRIMARY KEY,
   PatientName VARCHAR(30),
   CancerType VARCHAR(30),
   Age INT,
   Stage VARCHAR(10),
```

```
    SurvivalYears INT
);
```

INSERT INTO CANCERDATA

```
INSERT INTO CANCERDATA VALUES (301, 'Kavya', 'Breast', 45, 'II', 8);
INSERT INTO CANCERDATA VALUES (302, 'Arun', 'Lung', 60, 'III', 4);
INSERT INTO CANCERDATA VALUES (303, 'Divya', 'Breast', 38, 'I', 10);
INSERT INTO CANCERDATA VALUES (304, 'Hari', 'Colon', 52, 'II', 6);
INSERT INTO CANCERDATA VALUES (305, 'Meena', 'Lung', 57, 'IV', 2);
INSERT INTO CANCERDATA VALUES (306, 'Ravi', 'Prostate', 64, 'III', 5);
INSERT INTO CANCERDATA VALUES (307, 'Priya', 'Breast', 41, 'II', 9);
```

Query 1

```
SELECT *
FROM CANCERDATA
WHERE CancerType = 'Breast';
```

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.02 seconds CSV Export

Query 2

```
SELECT PatientName
FROM CANCERDATA
WHERE Stage = 'III';
```

Results Explain Desc

PATIENTNAME
Arun
Ravi

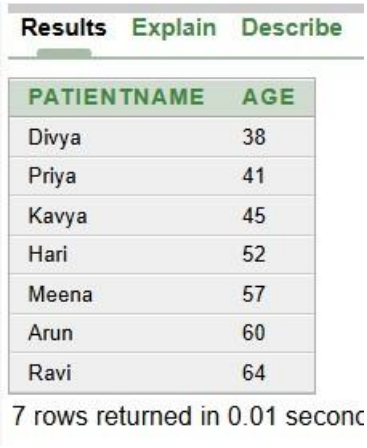
2 rows returned in 0.00 s

Query 3

```
SELECT PatientName, Age
```

FROM CANCERDATA

ORDER BY Age ASC;



Results Explain Describe

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

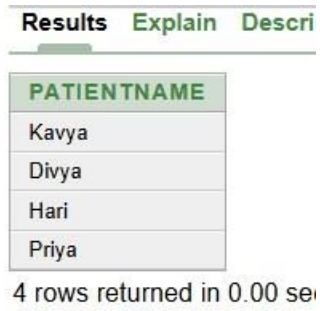
7 rows returned in 0.01 seconc

Query 4

SELECT PatientName

FROM CANCERDATA

WHERE SurvivalYears > 5;



Results Explain Descri

PATIENTNAME
Kavya
Divya
Hari
Priya

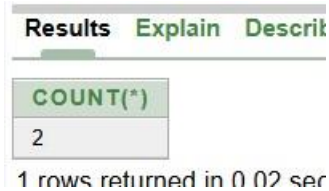
4 rows returned in 0.00 se

Query 5

SELECT COUNT(*)

FROM CANCERDATA

WHERE CancerType = 'Lung';



Results Explain Describ

COUNT(*)
2

1 rows returned in 0.02 sec

Query 6

```
INSERT INTO CANCERDATA
VALUES (308, 'Anand', 'Leukemia', 50, 'III', 7);
```

Query 7

```
SELECT COUNT(DISTINCT CancerType)
FROM CANCERDATA;
```

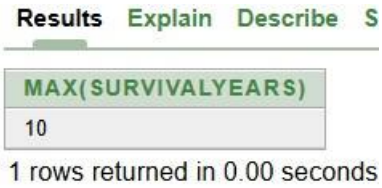


COUNT(DISTINCTCANCERTYPE)
5

1 rows returned in 0.00 seconds CSV

Query 8

```
SELECT MAX(SurvivalYears)
FROM CANCERDATA;
```



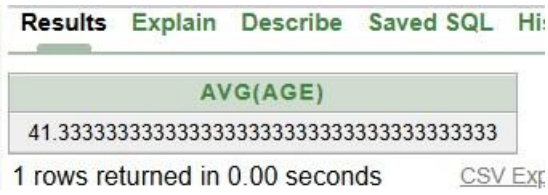
Results Explain Describe S

MAX(SURVIVALYEARS)
10

1 rows returned in 0.00 seconds

Query 9

```
SELECT AVG(Age)
FROM CANCERDATA
WHERE CancerType = 'Breast';
```



Results Explain Describe Saved SQL Hi

AVG(AGE)
41.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Exp

Query 10

```
SELECT MIN(Age)
FROM CANCERDATA
WHERE Stage = 'II';
```



ITEMS and TRADERS

CREATE TABLE ITEMS

```
CREATE TABLE ITEMS (
   CODE INT PRIMARY KEY,
   INAME VARCHAR(40),
   QTY INT,
   PRICE INT,
   COMPANY VARCHAR(30),
   TCODE VARCHAR(10)
);
```

INSERT INTO ITEMS

```
INSERT INTO ITEMS VALUES (1001, 'DIGITAL PAD12i', 120, 11000, 'XENITA', 'T01');
INSERT INTO ITEMS VALUES (1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T02');
INSERT INTO ITEMS VALUES (1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T01');
INSERT INTO ITEMS VALUES (1003, 'DIGITAL CAMERA 12X', 160, 8000, 'DIGICLICK',
'T02');
INSERT INTO ITEMS VALUES (1005, 'PEN DRIVE 32 GB', 600, 1200, 'STOREHOME',
'T03');
```

CREATE TABLE TRADERS

```
CREATE TABLE TRADERS (
   TCODE VARCHAR(10) PRIMARY KEY,
   TNAME VARCHAR(40),
   CITY VARCHAR(30)
);
```

INSERT INTO TRADERS

INSERT INTO TRADERS VALUES ('T01', 'ELECTRONIC SALES', 'MUMBAI');

INSERT INTO TRADERS VALUES ('T03', 'BUSY STORE CORP', 'DELHI');

INSERT INTO TRADERS VALUES ('T02', 'DISP HOUSE INC', 'CHENNAI');

Query 1

```
SELECT *
FROM ITEMS
ORDER BY INAME ASC;
```

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

5 rows returned in 0.01 seconds CSV Export

Query 2

```
SELECT INAME, PRICE
FROM ITEMS
WHERE PRICE BETWEEN 10000 AND 22000;
```

Results Explain Describe Sav

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds

Query 3

```
SELECT TCODE, COUNT(*)
FROM ITEMS
GROUP BY TCODE;
```

Results Explain Describ

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.00 sec

Query 4

SELECT PRICE, INAME, QTY

FROM ITEMS

WHERE QTY > 150;

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

Query 5

SELECT TNAME

FROM TRADERS

WHERE CITY IN ('DELHI', 'MUMBAI');

Results Explain Describe Sav

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.02 seconds

Query 6

SELECT COMPANY, INAME

FROM ITEMS

ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CS

Query 7

SELECT MAX(PRICE), MIN(PRICE) FROM

ITEMS;

Results Explain Describe Saved SQL

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV

Query 8

SELECT PRICE * QTY FROM

ITEMS

WHERE CODE = 1004;

Results Explain D

PRICE*QTY
1075000

1 rows returned in 0.(

Query 9

SELECT DISTINCT TCODE

FROM ITEMS;

Results Explain De

TCODE
T01
T03
T02

3 rows returned in 0.01

Query 10

```
SELECT INAME, TNAME
FROM ITEMS I, TRADERS T
WHERE I.TCODE = T.TCODE
AND QTY < 100;
```

Results Explain Describe Saved SQL

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC

2 rows returned in 0.00 seconds CSV E